

PV-PP

PROGRAMMER'S GUIDE

V2.1.167 PRODUCTIVE VALUE-PRODUCTIVE POWER
FRAMEWORK



Lance Amundsen | amundsenlance@gmail.com
<https://amundsenlance.github.io/pvpp-runtime-api/>

PV-PP Programmer's Guide

Edition 2.1, for PV-PP Runtime v2.1.167 (Framework 2.1, Build 167). 2026.

DOI: 10.5281/zenodo.23238712. Previous version, for runtime build 141: 10.5281/zenodo.23072420.

Copyright © 2026 Lance Amundsen. This work is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0): <https://creativecommons.org/licenses/by/4.0/>

PV-PP™ is a trademark of Lance Amundsen. The license covers the text of this book; it grants no rights in the trademark.

The programs in this book are also distributed in the PV-PP runtime repository and are licensed there under the Apache License, Version 2.0, the license of that repository. They are provided "as is", without warranties or conditions of any kind.

Companion volumes: PV-PP Getting Started, DOI 10.5281/zenodo.23238468; PV-PP Programmer's Reference, whose edition for runtime v2.1.167 follows this one (its edition for runtime build 141 is 10.5281/zenodo.23072506).

Runtime, programs, and all three volumes: <https://github.com/AmundsenLance/pvpp-runtime-api> and <https://amundsenlance.github.io/pvpp-runtime-api/>

Contact: Lance Amundsen, amundsenlance@gmail.com

About This Book

This book is a guide to the Productive Value–Productive Power (PV-PP) framework and to building applications with the PV-PP runtime. It explains how to understand the framework, model a productive system, write programs against the runtime, and test the results. It is the second of three volumes. PV-PP Getting Started is the short entry point, with a first application built in five steps; the PV-PP Programmer's Reference states exactly what the runtime provides, call by call. This Guide teaches; the Reference is exact. Where this book gives teaching-sized examples, the Reference gives constructors, method signatures, fields, errors, and validation contracts; a citation such as Reference 3.1 names one of its sections.

Which framework and runtime. The PV-PP framework is defined by its V2.1 specification set, maintained by the author; where this Guide refers to an owner specification or owning specification, it means one of those documents, and the specification controls where this Guide is less precise. The runtime has had four frozen releases: Runtime V1 (v0.70), Runtime V2 (v0.131), Runtime V2.1 (v0.141), and Runtime v2.1.167, which is Framework 2.1, build 167, developed under the identifier v0.167. Unless a passage says otherwise, runtime statements and code in this book refer to v2.1.167. Its decision core is build 141's, byte for byte, and is synchronous; Chapter 7 states that core's boundary. Around it, build 167 adds a supervision layer that governs an action while it runs; Chapter 12 describes it. The frozen v2.1.167 source and its tests control what the runtime actually does.

What changed from the previous edition. The previous edition described build 141. The framework did not change, the decision core did not change, and nothing in Chapters 1 through 11 that was true of build 141 has become false. This edition adds Chapter 12 and its prompt, A.12; extends Chapter 7's boundary statement to cover the supervision layer; and states, in Section 5.12, the rule for what a new actor inherits from the one that created it.

Historical material. The larger Selfish Cooperation book supplies motivation, examples, and applications; its older formulations are reconciled to the V2.1 architecture, not treated as independent authority. Teaching notation, corridor labels, and benchmark heuristics are teaching aids unless a V2.1 specification defines them formally.

Programs. Every program in this book runs against the unmodified, hash-verified runtime, and each has tests. Appendix G lists them; the repository's `programmers-guide/programs` folder holds exact copies.

Reading Paths

The chapters build on one another, but different readers can take different routes. Each path assumes Chapters 1 and 2 have at least been skimmed, since they define the terms the rest of the Guide uses.

Table 0.1 — Suggested reading paths

Reader	Path	Focus
Conceptual reader	1 → 2 → 10 → 11	What the PV-PP framework is, how it travels across domains, and what it claims

Reader	Path	Focus
Model builder	1 → 2 → 3 → 4 → 5 → 6	Specifying a model and configuring the decision cycle
Programmer	2 → 5 → 6 → 7 → 8 → 9 → 12	Building, executing, and testing against the runtime
Researcher or evaluator	1.7 → 2 → 6 → 9 → 11	The architecture, the evidence, and the proof boundaries
AI-assisted modeler	Each chapter with its Appendix A prompt	Building a model of one's own system step by step
Agent-governance integrator	1.9 → 7 → 8 → 10.5 → 11.7 → 12	Governing an agent's actions while they run, when monitors can withdraw its authority

Contents

About This Book.....	2
Reading Paths.....	2
Chapter 1 — What the PV-PP Framework Is and Why It Exists.....	10
1.1 The Historical Purpose: Cooperation and Survival.....	10
1.2 Two Questions: What Passes Between Us, and What Remains Possible Afterward?.....	10
1.3 Productive Value and Productive Power.....	11
1.4 From Interaction to Productive Systems.....	12
1.5 From Idea to Mathematics to Software.....	13
1.6 From Productive Systems to Decisions.....	14
1.7 Why One Number Is Sometimes Enough—and Sometimes Is Not.....	15
1.8 What the PV-PP Framework Preserves That a Scalar Score May Compress.....	16
1.9 Lower Bounds, Upper Bounds, and Viable Operation.....	17
1.10 One Architecture Across Very Different Domains.....	18
1.11 What the PV-PP Framework Is Not.....	19
1.12 Using This Guide with AI.....	19
1.13 From the Intuitive Model to the Formal Architecture.....	20
Chapter 2 — Conceptual Architecture and Construct Map.....	21
2.1 A Gentle Mathematical On-Ramp.....	21
2.2 Why a Construct Map Comes Before the Operator Stack.....	22
2.3 The Whole System in One Loop.....	23
2.4 Actual Persistent State: $S_i(t)$	24
2.5 Perceived Decision State: $P_i(t)$, PPP, and Perceived Viability.....	24
2.6 Objectives: $O_i(t)$	25
2.7 PV, SPV, PP, and Viability.....	25
2.8 Information, Memory, Expectation, and Transfer History.....	26
2.9 Graph, Transformations, Reachability, and the Represented Productive Envelope.....	26
2.10 Policy Construction, Completeness, Constraints, Framing, and Adequacy.....	27
2.11 Selection, Execution, Environmental Realization, and Layer-1 Transition.....	27
2.12 Cross-Cutting Rules That Keep the Architecture Honest.....	28
2.13 A Compact Walk-Through of One Governed Decision.....	28
2.14 From the Construct Map to Application Modeling.....	29
Chapter 3 — Modeling a Productive System.....	30
3.1 Start With a Bounded Scenario.....	30
3.2 Discover the Interaction Before the Decision Rule.....	30
3.3 Identify Participants and Their Roles.....	31
3.4 Classify PV, SPV, PP, Resources, and Broader State by Role.....	31
3.5 Build the Actual State Before the Perceived State.....	32
3.6 Separate Possession, Access, Permission, Control, and Operability.....	33
3.7 Model Configuration-Dependent and Joint Capability.....	33
3.8 Define Objectives Without Turning Them Into Hidden Utility.....	33
3.9 Identify Constraints, Thresholds, Corridors, and Governing Domains.....	34

3.10 Model Recovery as Structure, Not as a Hope.....	34
3.11 Decide Whether a Group Is Actually a Productive Entity.....	35
3.12 Define the Environment and the Execution Boundary.....	35
3.13 Identify What Can Change and How the Model Will Know.....	35
3.14 Keep History, Current State, and Replay Obligations Separate.....	36
3.15 Build a Scenario and Model Definition Packet Before Coding.....	36
3.16 Complete Walk-Through: The Coffee Vending Model.....	37
3.17 Second Worked Model: A Household Replacing a Failing Car.....	38
3.18 Common Modeling Errors.....	39
3.19 From Modeling to Information Governance.....	39
Chapter 4 — Information, Memory, History, and Evidence.....	41
4.1 Why Information Needs Its Own Governance.....	41
4.2 Keep the Informational Objects Separate.....	41
4.3 From Actual State to Perceived Decision State.....	42
4.4 Quality and Confidence Do Not Create Authority.....	42
4.5 Applicability and Permitted Use.....	43
4.6 Disposition, Supersession, and Contradiction.....	43
4.7 Temporal Provenance: When Was It True, and When Was It Known?.....	43
4.8 Memory Is Not the Same as History.....	44
4.9 Transfer History: Record Events Without Turning Them Into Current Truth.....	44
4.10 Historical Residue and Transition-Relevant State Sufficiency.....	44
4.11 Replay Sufficiency: Reconstruct the Decision That Was Actually Made.....	44
4.12 Prospective Sufficiency and Replay Sufficiency Side by Side.....	45
4.13 Information Can Change State, Capability, and Reachability—But Not by Declaration....	45
4.14 Information Change, Staleness, and Re-entry.....	46
4.15 Build an Information and Evidence Definition Packet.....	46
4.16 Complete Walk-Through: The Coffee Vending Information Model.....	47
4.17 Second Worked Example: Human Experience, Memory, and Expectation.....	47
4.18 Common Information-Governance Errors.....	48
4.19 From Information Governance to Change and Reachability.....	48
Chapter 5 — Representing Change, Capability, and Reachability.....	49
5.1 Why Change Needs Its Own Representation.....	49
5.2 Domains, Instances, and Transformations.....	49
5.3 Start From Function, Not Mechanism.....	50
5.4 Scan the Transformation Families.....	51
5.5 Reachability Is Not Capability, Desirability, or Authorization.....	51
5.6 Configuration, Access, and Compatibility.....	52
5.7 Compose Multi-Step Paths.....	53
5.8 From Paths to Seeds, and When the Graph Is Ready.....	53
5.9 Recovery Is a Reachability Question.....	54
5.10 Opportunity Closure, Reopening, and Delay.....	54
5.11 New Opportunities Without Silently Expanding the Model.....	55
5.12 Capability Change: From Intention to Evidenced PP.....	56
5.13 What Becomes Stale, and Where Governance Resumes.....	57
5.14 Build a Change and Reachability Definition Packet.....	58

5.15 Complete Walk-Through: The Coffee Vending Change Model.....	59
5.16 Second Worked Example: Household Transportation.....	59
5.17 Third Example: An Autonomous Agent.....	60
5.18 Common Errors in Representing Change.....	60
5.19 From Represented Change to the Decision Cycle.....	61
Chapter 6 — The Canonical Decision Cycle.....	62
6.1 The Cycle at a Glance.....	62
6.2 The Decision Interface: $P_i(t)$ and $O_i(t)$	63
6.3 Pressure: Φ	63
6.4 Collapse Horizons: H	64
6.5 Governing Domains: G	64
6.6 Regime: $\mathcal{R}$	64
6.7 From Graph to Candidates: Π and Π Completeness.....	65
6.8 Constraints.....	65
6.9 Domain Framing.....	65
6.10 Recovery in the Cycle, and the Runtime's Joint-Recovery Check.....	66
6.11 Adequacy and the Shared Projection.....	66
6.12 Selection: Σ	66
6.13 Toward Execution: ε	67
6.14 Ordering, No Downstream Repair, and Re-entry.....	67
6.15 Your First Program: Evaluating a Decision Cycle.....	68
6.16 Reading the Four Cases.....	70
6.17 The Same Cycle in Other Domains.....	71
6.18 Common Errors in the Decision Cycle.....	71
6.19 From Selection to Execution.....	72
Chapter 7 — Selection, Execution, and Environmental Realization.....	73
7.1 Four Events, Not One.....	73
7.2 Σ Selects, ε Realizes.....	74
7.3 The Execution Lineage in the Runtime.....	74
7.4 The Execution License and the Episode.....	75
7.5 Registration Is Not Authorization.....	76
7.6 Native Authorization: Single-Use and Bound.....	76
7.7 Call-Through and the Four Outcome Classes.....	77
7.8 From Result to Evidence.....	77
7.9 The Layer-1 Transition.....	78
7.10 Every Terminal Episode Reaches Layer 1.....	78
7.11 Returning Upstream.....	78
7.12 Program: Executing the Coffee Purchase.....	79
7.13 Reading the Five Cases.....	81
7.14 The Same Boundary in Other Domains.....	82
7.15 The Decision Core's Implementation Boundary.....	82
7.16 The Authorization-to-Effect Race.....	84
7.17 Extending Without a Second Decision System.....	85
7.18 A Known Interface Gap: Ending a Recovery Corridor.....	85
7.19 Common Errors at the Execution Boundary.....	85

7.20 From One Purchase to an Application.....	86
Chapter 8 — Building a PV-PP Application.....	87
8.1 From Definition Packets to Runtime Structures.....	87
8.2 The Application–Runtime Boundary.....	88
8.3 Registration and Runtime Prerequisites.....	88
8.4 The Adapter: Facts, Not Decisions.....	89
8.5 Perception from Evidence.....	89
8.6 The Store and the Layer-1 Transition.....	89
8.7 Observation Intake and Enforcement Interfaces.....	90
8.8 Reconciliation and Remedy.....	90
8.9 Traceability.....	91
8.10 Program: The Coffee Application.....	91
8.11 Reading the Day.....	94
8.12 Scalar Shortcuts and the Certificate.....	95
8.13 The AI-Assisted Development Workflow.....	96
8.14 A Second Reference: The Runtime's Small Economy Benchmark.....	97
8.15 Development History Is Not a Pattern.....	97
8.16 The Same Pattern in Other Domains.....	97
8.17 Common Errors in Building an Application.....	98
8.18 From an Application to Evidence.....	98
Chapter 9 — Testing and Benchmarking.....	99
9.1 Kinds of Test and What Each Establishes.....	99
9.2 Testing Category Boundaries.....	100
9.3 Invariant Tests.....	100
9.4 Hostile Cases.....	101
9.5 Traces That Reveal the Earliest Failure Surface.....	101
9.6 False Success.....	102
9.7 Scenario Benchmarks.....	102
9.8 Perturbation and Metamorphic Tests.....	102
9.9 Reproducibility.....	103
9.10 Regression Discipline.....	103
9.11 Replay Tests.....	103
9.12 Program: Testing the Coffee Application.....	103
9.13 The Evidence Record: Three Levels.....	105
9.14 Practices That Made the Results Credible.....	106
9.15 What the Record Shows, and What It Does Not.....	107
9.16 Framework Claims and Runtime Claims.....	108
9.17 Designing Your Own Test and Benchmark Plan.....	108
9.18 Common Errors in Testing and Benchmarking.....	108
9.19 From Evidence to Worked Examples.....	108
Chapter 10 — Worked Examples.....	109
10.1 How Each Example Is Built.....	109
10.2 Example 1: A Household Deciding Under Changing Information.....	110
10.3 Example 2: An Organization Whose Capability Depends on Configuration.....	111
10.4 Example 3: An Engineered Cooling System.....	112

10.5 Example 4: An Autonomous Operations Agent.....	113
10.6 A Teaching Benchmark: Safe Margin versus Fast Gain.....	116
10.7 What Carries Across, and What Does Not.....	117
10.8 Common Errors When Porting the Architecture.....	117
10.9 From Examples to Positioning.....	118
Chapter 11 — Positioning, Reduction, and Advanced Topics.....	119
11.1 What Positioning Means Here.....	119
11.2 Direction 1: Containment.....	119
11.3 Direction 2: Non-Reduction.....	120
11.4 The Boundaries of the Proofs.....	121
11.5 What the Benchmarks Add.....	121
11.6 Structural Comparison with Established Methods.....	122
11.7 Security and Autonomous-Agent Governance.....	123
11.8 Portability Across Disciplines.....	123
11.9 Advanced Topics.....	124
11.10 Open Questions.....	124
11.11 Common Errors in Positioning Claims.....	125
11.12 What V2.1 Establishes, What Has Been Demonstrated, and What Remains.....	125
11.13 Closing.....	126
Chapter 12 — Supervising Execution While It Runs.....	127
12.1 What Supervision Must Not Become.....	127
12.2 The Layer at a Glance.....	127
12.3 Registering an Execution.....	128
12.4 Monitors, Admission, and Governed Facts.....	129
12.5 Dependencies, Fan-Out, and Snapshots.....	129
12.6 Checkpoints and Continuation Postures.....	129
12.7 Ownership, Fencing, and Stop Requests.....	130
12.8 Enforcement, Effect, and Layer 1.....	131
12.9 Crashes, Divergence, and Recovery.....	131
12.10 Trust and Deployment.....	132
12.11 Program: Supervising the Failover.....	132
12.12 Reading the Run.....	134
12.13 What Supervision Still Cannot Do.....	135
12.14 Common Errors in Supervised Execution.....	135
Appendix A — AI Interview Prompts.....	137
A.1 Chapter 1 Prompt — Is the PV-PP Framework the Right Tool?.....	137
A.2 Chapter 2 Prompt — Sketch the Construct Map.....	139
A.3 Chapter 3 Prompt — Build the Scenario and Model Definition Packet.....	140
A.4 Chapter 4 Prompt — Build the Information and Evidence Packet.....	142
A.5 Chapter 5 Prompt — Build the Change and Reachability Packet.....	143
A.6 Chapter 6 Prompt — Configure the Decision Cycle.....	145
A.7 Chapter 7 Prompt — Plan Execution and Realization.....	147
A.8 Chapter 8 Prompt — Build the Application.....	149
A.9 Chapter 9 Prompt — Plan Testing and Evidence.....	150
A.10 Chapter 10 Prompt — Build a Worked Example in Your Domain.....	152

A.11 Chapter 11 Prompt — Position Your Application and State Its Claims.....	154
A.12 Chapter 12 Prompt — Plan Supervised Execution.....	155
Appendix B — Notation, Symbols, and Glossary.....	158
B.1 State, Perception, and Objectives.....	158
B.2 The Decision Stages.....	158
B.3 Execution and Realization Objects.....	159
B.4 Fixed Vocabularies.....	160
B.5 Graph Construction Failure Classes.....	161
B.6 Scalar-Reduction Terms.....	161
B.7 Glossary.....	162
Appendix C — Modeling Checklist.....	164
C.1 Scenario and Model (Chapter 3).....	164
C.2 Information and Evidence (Chapter 4).....	164
C.3 Change and Reachability (Chapter 5).....	165
C.4 Consistency Across the Packets.....	165
C.5 Errors to Rule Out.....	166
C.6 Sign-Off.....	166
Appendix D — Implementation Checklist.....	167
D.1 Decision-Cycle Configuration (Chapter 6).....	167
D.2 Execution Lineage (Chapter 7).....	167
D.3 Application Boundary and Adapter Discipline (Chapter 8).....	168
D.4 Supervised Execution (Chapter 12).....	168
D.5 Errors to Rule Out.....	169
D.6 Sign-Off.....	170
Appendix E — Testing and Benchmark Checklist.....	171
E.1 Claims and Kinds of Test.....	171
E.2 Application Tests.....	171
E.3 Benchmark Specification.....	171
E.4 Programs of Benchmarks.....	172
E.5 Reproducibility and Reporting.....	172
E.6 Errors to Rule Out.....	173
Appendix F — Sources and Open Items.....	174
F.1 What Controls What.....	174
F.2 Evidence Sources.....	174
F.3 Known Gaps and Open Items.....	175
Appendix G — Code Files.....	176
G.1 The Files.....	176
G.2 Running Them.....	176
G.3 Checking the Runtime.....	177
G.4 What the Programs Are Not.....	177

Chapter 1 — What the PV-PP Framework Is and Why It Exists

The Productive Value–Productive Power (PV-PP) framework describes productive systems and governs decisions within them. It begins with a simple observation: people and other productive actors do not exist as isolated decision-makers. They depend on interactions, resources, capabilities, information, relationships, institutions, technologies, and environments that make action possible. Those conditions change as interactions occur. A useful decision architecture therefore has to represent not only what an actor wants now, but what value is moving, what capability exists, what future actions remain possible, what constraints bind, and what happens after a choice is executed.

The framework can be used descriptively—to model how a productive system is structured and how it changes—and operationally—to construct, evaluate, select, authorize, execute, and reassess actions. The same architecture can be applied at different scales: a person deciding whether to enter an exchange, a family managing scarce resources, an organization committing capacity, an engineered system preserving operating margins, or an autonomous software agent whose capabilities and reachable actions must remain within governed bounds.

1.1 The Historical Purpose: Cooperation and Survival

The originating intuition behind the PV-PP framework is that people need one another. Human beings survive and develop through cooperation and interaction. No person individually recreates the food system, language, accumulated knowledge, medicine, tools, infrastructure, institutions, protection, education, and social organization on which ordinary life depends. People receive value from others before they can reciprocate, specialize because others specialize, and rely on productive structures accumulated by people they will never meet.

In its shortest historical form: people cooperate to survive, and cooperation works through interactions that move value and alter future capability.

This statement is a starting point, not a claim that all human interaction is cooperative or beneficial. The mature PV-PP framework explicitly accommodates conflict, deception, coercion, theft, protection, punishment, repair, institutional authority, and other unwilling or asymmetric interactions. The deeper point is that human continuity depends on interactions among present people and on developmental transfer into future capable people. Cooperation is one of the principal mechanisms by which individual lives and complex societies become possible.

This also explains the phrase “Selfish Cooperation” used in the larger theoretical treatment. Individuals act from within their own needs, commitments, fears, obligations, relationships, and conceptions of what matters, while the means by which they pursue life are deeply social. Cooperation does not require pure altruism. It can arise because coordinated interaction expands what participants can obtain, preserve, produce, or become capable of doing.

1.2 Two Questions: What Passes Between Us, and What Remains Possible Afterward?

The broad theory can be entered through two questions. First: what actually passes between parties when they interact? Second: what does the interaction leave them able to do afterward?

The PV-PP framework names the first side of the problem Productive Value (PV). It names the second side Productive Power (PP). Keeping those two questions separate is foundational. The idea begins from familiar economic ground: people exchange goods and services, specialize, and cooperate because exchange lets them obtain value they could not efficiently produce alone. The PV-PP framework extends that traditional exchange picture beyond priced goods and services to other value actually conveyed through interaction, including information, teaching, care, protection, and other beneficial or harmful effects. An interaction can therefore deliver something immediately valuable while strengthening, preserving, leaving unchanged, or damaging future capability. Conversely, an interaction can impose a short-term burden while building capability that matters later.

Consider education. A lesson conveys information and instruction through interaction. That is part of the value moving in the interaction. If learning occurs, the student may leave with greater capability to reason, calculate, communicate, or perform a task. The value conveyed and the capability left afterward are related, but they are not the same thing. The distinction becomes even clearer in health care, employment, family care, fraud, addiction, institutional protection, and technology: what is received now and what remains possible later can move in different directions.

1.3 Productive Value and Productive Power

Productive Value

Productive Value is value transferred, delivered, realized, imposed, or otherwise conveyed through interaction. Depending on the domain, it may include labor, care, information, teaching, money, protection, goods, services, or beneficial and harmful effects when those effects are actually conveyed or imposed. The term is broader than market price. It does not imply that every valuable human experience must justify itself by increasing economic output.

The PV-PP framework therefore does not reduce friendship, rest, affection, dignity, beauty, play, care, or other human goods to their effect on later productivity. Some value is valuable in its own right. The word productive points instead to the fact that interactions often have consequences for what can happen next.

Productive Power

Productive Power is actual capacity, under stated conditions, to generate, transform, coordinate, deploy, preserve, protect, recover, or otherwise produce Productive Value. Knowledge, skill, physical capability, organizational capability, and other capacities can constitute PP directly. Other things—money, tools, time, relationships, reputation, institutional support, technology, safety conditions, or recovery pathways—may enable, constrain, preserve, or modify capability without being ontologically identical to PP.

This distinction matters in implementation. A model should not simply relabel every useful resource or favorable condition as Productive Power. The PV-PP framework asks what role an object, state, relationship, or condition actually plays. Ownership is not the same as access. Access is not the same as control. Control is not the same as operability. Authority is not the same as capability. A resource can exist while the capability to use it does not.

PV asks what value is conveyed. PP asks what the actor or system is actually capable of doing under the relevant conditions.

The PV-PP framework also distinguishes actual Productive Power from Perceived Productive Power (PPP): the capability an actor believes or represents itself or another actor as having at decision time. PPP can be accurate, incomplete, stale, or wrong. This distinction becomes important whenever decisions are made from a representation of capability rather than direct knowledge of actual capability. Chapter 2 places PPP inside the broader perceived decision state $P_i(t)$.

1.4 From Interaction to Productive Systems

Once PV and PP are separated, the unit of analysis expands naturally. A productive system is not merely a person holding resources or a machine performing work. It is a structured environment in which actors, capabilities, resources, information, relationships, objectives, constraints, possible transformations, and realized interactions combine over time.

A productive system can be small or large. A household can be modeled as a system of people, resources, obligations, retained value, capabilities, and possible actions. A manufacturing process can be modeled in terms of equipment, material state, operator capability, transformations, quality constraints, maintenance and recovery. An institution can be modeled through collective capability, information, authority, obligations, trust, and the actions it can legitimately perform. A software-agent environment can be modeled through represented state, tools, credentials, reachable routes, evidence, authority, and execution effects.

Productive interactions can also be nested across time. The appropriate interaction boundary depends on the question being asked. A marriage, for example, can be examined through a particular conversation, a day in the relationship, a week, a year between anniversaries, a decade, or the life of the marriage. A day-level interaction can alter what becomes possible in the week or year that contains it, while the longer relationship carries forward consequences of many smaller interactions.

The higher-level interaction should not be treated as a simple arithmetic sum of the lower-level ones. Sequence, repetition, memory, thresholds, accumulated trust or damage, recovery, and path dependence can matter. A small event can have little significance in isolation yet become consequential when repeated; a later interaction can repair, amplify, or reinterpret effects of an earlier one. The PV-PP framework therefore permits the analyst to choose a time scale appropriate to the productive question while preserving the fact that interactions at one scale can change state and capability at another.

A related but distinct form of nesting occurs among productive entities. An individual may participate in a team, the team in a department, the department in an organization, and the organization in an institution. But a collection of lower-level actors does not automatically become an independent higher-level productive entity. The higher level must itself be worth modeling as a persistence-bearing productive unit—for example, because it has a meaningful boundary, retained state, transition structure, continuity, and governance or coordination that connects member action to entity-level action. Each level must therefore qualify on its own rather than inheriting entity status merely from aggregation.

These two forms of nesting—interaction nesting across episodes and time, and productive-entity nesting across organizational levels—can coexist. A team can participate in a day-long interaction nested inside a year-long project, while individual members participate in smaller interactions within the same process. The PV-PP framework can move across these temporal and organizational scales without treating the levels as interchangeable.

The commonality is structural, not superficial. The PV-PP framework does not claim that a family, a factory, and an AI agent are the same kind of thing. It claims that productive systems can often be described using a common set of questions: What is the state? What capability exists? What value is being produced or transferred? What actions are reachable? Which actions are admissible and adequate? What evidence supports the representation? What must remain true for execution authority to remain valid? What state exists after realization?

1.5 From Idea to Mathematics to Software

The PV-PP framework did not begin as a software project. It began as a conceptual attempt to describe interaction, exchanged value, and the capability left behind by interaction. As the theory became more precise, ordinary prose was no longer enough. The distinctions had to be stated mathematically: actors and their states had to be represented explicitly; Productive Value and Productive Power had to remain distinct; possible interactions and transformations had to be representable; and later decision stages had to operate on defined objects rather than intuition alone.

That formalization was developed with substantial AI assistance. The human contribution remained the originating theory, conceptual distinctions, architecture choices, interpretation, and acceptance or rejection of proposed formulations; AI tools materially assisted in turning those ideas into mathematical notation, formal specifications, proof programs, executable software, tests, and documentation. The resulting development lineage is therefore best understood as: idea → mathematical formalization → software implementation → simulation and benchmark testing.

Software became important because the PV-PP framework was intended to do more than describe a sociological idea after the fact. An executable model made it possible to represent actors, give them state and Productive Power, expose them to possible interactions, construct and govern candidate actions, execute modeled consequences, and observe how the system changed over repeated interactions. That opened the door to computational experiments involving individual behavior, exchange, cooperation, memory, addiction and recovery, multi-agent interaction, small economies, and later autonomous-agent governance.

The implementation eventually developed into a formal runtime and Runtime API. The runtime does not replace the framework; it implements a governed subset of it. This distinction remains important throughout this Guide: the framework defines the architecture, while the runtime provides executable machinery for a declared implementation scope.

The simulation and benchmark program became correspondingly broad. Internal work included toy survival worlds, ecology and spatial-viability environments, scalar-comparison tests, tradeoff gridworlds, self-sacrifice cases, memory and recall experiments, addiction/recovery and alcohol-related trajectories, cooperation-propensity and multi-agent memory studies, trade and surplus exchange, productive-maintenance exchange, and a long-running Small Economy stress test of the frozen v0.70 runtime. Public benchmark work then exposed a frozen PV-PP architecture to problems designed by others: all ten environments in DeepMind's archived AI Safety Gridworlds repository, spanning its eight safety-problem classes, and all thirty-one current registrations of the MO-Gymnasium multi-objective suite (version 1.3.2), covered through sixteen benchmark packages that include Mountain Car, Minecart, Water Reservoir, Deep Sea Treasure, Resource Gathering, Fishwood, Breakable Bottles, Fruit Tree, Four Room, Lunar Lander, Highway, Super Mario, and the MuJoCo locomotion tasks. A separate real-world tranche used three source-locked problems from the MOEA Framework RealWorldBenchmarks

repository: water-supply portfolio planning, lake-pollution control policy, and general-aviation aircraft product-family design.

These benchmarks should not be read as proof that the PV-PP framework is superior to established alternatives. Their role is more disciplined: to test whether the framework can be translated into executable models, preserve its distinctions under varied environments, expose failure modes, reproduce bounded scalar behavior where appropriate, and operate against problems not invented solely for the PV-PP framework. Across the public and real-world programs, no benchmark required a new canonical operator or state primitive; the record also contains genuine behavioral failures, source-fidelity qualifications, and one open formalization question. Coverage was not the same as full reproduction: several dependency-heavy MO-Gymnasium environments were covered through source-faithful mechanism audits rather than trained-agent runs. Chapter 9 develops the benchmark methodology and evidentiary limits in detail.

Chapter 2 begins the reverse journey for the reader: it starts with the simplest mathematical objects and builds back up to the complete architecture.

1.6 From Productive Systems to Decisions

The earliest theory concentrated heavily on interaction and exchange. As it developed, a further problem became unavoidable: people and productive systems do not merely experience exchanges; they choose and execute actions under uncertainty.

Uncertainty here includes imperfect information, and the PV-PP framework treats imperfect information as the normal condition rather than an exception. The textbook competitive-market model begins from the opposite idealization: participants know prices, qualities, and their own options. Economics has long studied what happens when that assumption fails—asymmetric information, costly search, signaling, bounded rationality—but usually as departures from an idealized baseline. The PV-PP framework builds imperfect information into every actor's decision state from the start. What an actor perceives, remembers, infers, and has been told is modeled explicitly and kept separate from what is actually true.

The consequences reach into interaction itself. A buyer may not know a seller's real capability; a lender may misjudge a borrower's reserves; a person may underestimate their own capability; an automated agent may act on a stale record. Information is also something that moves between parties—a warning, a diagnosis, a disclosure, a deception—so it can be Productive Value in its own right, and its quality can change what each party is able to do next. Chapter 2 introduces the perceived decision state $P_i(t)$ that carries this, and Chapter 4 develops the governance of information in detail.

An option can look attractive and still be non-viable. An actor may possess enough immediate capacity to begin an action but not enough to sustain it. A high-return action may destroy a non-compensable condition. An action may be technically possible but prohibited by authority or other binding constraints. A plan may work only while a particular route, resource, relationship, or assumption remains valid. An action can succeed initially yet leave no realistic recovery path if conditions deteriorate.

The PV-PP framework therefore distinguishes progressively stronger questions. Is an option represented? Is it reachable? Is it feasible under current capability? Is it permitted? Is the information adequate? Does the actor possess sufficient capability across the governing dimensions? Are relevant consequences and recovery paths represented? Is the candidate

adequate? Which candidate should be selected? Is the selected action currently authorized for execution? What actually happened after execution?

These are not merely different ways to ask whether an action has a high score. They are different governed questions. The formal architecture developed later in this Guide preserves their ordering and their distinct failure surfaces.

1.7 Why One Number Is Sometimes Enough—and Sometimes Is Not

Many familiar decision systems reduce alternatives to a scalar: a utility value, reward, weighted score, cost-benefit figure, priority number, risk-adjusted return, or other single comparator. The PV-PP framework does not reject this practice. A scalar is efficient and appropriate when the decision really is reducible to one dimension or when the surrounding structure has already resolved the conditions that would otherwise need separate treatment.

The PV-PP framework instead treats scalarization as a reduction that should be justified by the structure of the decision. The PV-PP scalar-reduction proof program, closed in September 2026, examined that question from both directions.

In one direction it proves *containment*. On a precisely defined class of decisions—roughly, those in which every candidate's consequences lie along a single line and improve on every governing domain together—a plain unweighted average recovers exactly the same maximal choice set as the PV-PP framework's full staged selection, and every condition that licenses the shortcut can be checked from the framework's own objects. On that class, a scalar is not an outside competitor; it is a certified subroutine running inside the architecture. The result is published as *When Is One Number Enough? A Containment Theorem for Scalar Comparison in Staged Decision Architectures* (Amundsen, 2026; DOI 10.5281/zenodo.22715202).

In the other direction it proves *non-reduction*. When at least two governing domains are in genuine conflict, and candidates that tie on those domains still differ elsewhere, no scalar function of any form can faithfully reproduce the PV-PP framework's staged selection process; any scalar that tries is forced to merge stages the architecture keeps separate. When the set of governing domains changes over time, scalars that ignore the system's current state select wrongly under stated conditions, and a scalar that avoids those errors does so only by consulting state information or the PV-PP framework's own architectural objects.

Both results are deliberately bounded. Containment is proved on one local class, not for scalar methods generally. Non-reduction holds under stated structural conditions; it does not show that every scalar method fails, and vector-valued utilities, approximation sequences, and nonlinear governing scalars remain open questions. Taken together, the two directions support a precise position: scalar comparison can live inside the PV-PP framework as a governed, certifiable special case, while the full architecture does not reduce to scalar comparison. Chapter 11 develops both results, and the benchmark evidence that points the same way.

The PV-PP framework does not say, “never use one number.” It asks whether one number preserves the decision structure that matters.

This gives the framework a practical relationship to ordinary scoring methods. If the problem is faithfully scalar, use the scalar method. If hard constraints, non-compensable domains, state-dependent rules, changing reachability, recovery requirements, evidence authority, or execution conditions matter independently, the PV-PP framework provides places to represent those facts without forcing them all into the score.

1.8 What the PV-PP Framework Preserves That a Scalar Score May Compress

Structure	Why the PV-PP framework keeps it explicit
State	The same candidate can have different meaning in different system states. The PV-PP framework keeps the governing state explicit rather than assuming a score is portable across state changes.
Capability	An attractive outcome does not imply the actor has the capability to produce or sustain it. PP is represented separately from desirability.
Hard constraints	Some conditions are non-compensable. A larger gain elsewhere does not automatically offset lack of authority, violated consent, a safety prohibition, or another binding constraint.
Reachability	A destination can remain desirable while the path to it disappears—or a new path can become reachable. Reachability is represented separately from preference and separately from authorization.
Information and evidence	A decision can become invalid because its evidence is stale, inapplicable, incomplete, or no longer authoritative even when the numerical attractiveness of the target has not changed.
History and memory	Past interactions can alter current state, evidence, obligations, trust, recovery, or interpretation. The PV-PP framework can preserve transition-relevant history rather than flattening all history into a current score.
Recovery	A candidate can look beneficial under its expected path while leaving the system unable to recover if the path fails. Recovery can be governed explicitly.
Selection versus execution	Choosing an action and being authorized to execute it are distinct. The PV-PP framework preserves that distinction.
Realization	The selected or authorized action is not the same as the environmental effect that actually occurs. The host environment owns the real state transition and the framework can govern re-entry from the result.

The point is not that a sufficiently elaborate scalar system can never encode these distinctions. In many cases it can. But as gates, state variables, exceptions, evidence rules, recovery logic, and execution conditions accumulate, the real architecture is no longer just the scalar formula. The surrounding structure is doing substantive decision work. The PV-PP framework makes that structure first-class rather than treating it as incidental scaffolding around the score.

1.9 Lower Bounds, Upper Bounds, and Viable Operation

Much of the PV-PP framework's original practical logic is naturally understood as lower-bound governance. A productive system needs enough of the relevant capabilities, resources, evidence, safety, recovery capacity, and other governing conditions to continue viable operation. A person may need sufficient health, money, knowledge, support, or time to sustain a commitment. A machine may need sufficient operating margin. An organization may need sufficient staffing, expertise, liquidity, or recovery capacity. Falling below a governing minimum can make an otherwise attractive action inadequate.

Later applications made the complementary upper-bound problem especially visible. In autonomous-agent and security settings, more capability or more reachability is not always better. An agent may acquire access, routes, credentials, control, or combinations of capability that exceed the conditions under which prior execution authority was granted. A productive system may therefore need ceilings, boundaries, or other non-compensable limits as well as minimum requirements.

These are not two unrelated frameworks. Both ask whether the current state and proposed action remain inside the governed conditions for viable and authorized operation. The exact form of the condition is domain-specific: a minimum reserve, a maximum exposure, a prohibited state, a required recovery path, an authority boundary, or a combination of these.

1.9.1 Corridors: Operating Between Governed Bounds

A useful way to describe governed movement through a productive system is a corridor. A corridor identifies a path, class of paths, or operating region whose actions, states, and trajectories satisfy the applicable governing conditions. In many applications this includes remaining between relevant lower and upper bounds. The lower side may represent minimum capability, reserve, evidence, safety, or recovery requirements; the upper side may represent maximum exposure, reachability, authority, resource use, risk, or other limits that must not be exceeded.

Corridors are therefore not a new decision system layered on top of the PV-PP framework, nor must every corridor reduce to one scalar interval. They are a way of expressing bounded admissible, viable, degraded, failure, recovery, substitution, or other path structure using the framework's existing state, transition, constraint, adequacy, recovery, reachability, and authority machinery. Different applications can define different corridor structures, and a system can satisfy one bound while violating another governing dimension.

For a human or organization, a recovery corridor may describe actions that preserve enough capability and optionality to return to viable operation after a setback. For an autonomous agent, a governance corridor may combine minimum conditions required to act with ceilings on capability, reachability, or authority. Later chapters show where corridor conditions enter the formal architecture rather than treating the term as a substitute for the underlying constructs.

1.10 One Architecture Across Very Different Domains

The PV-PP framework's claim to generality should be understood carefully. It does not claim that different disciplines have identical subject matter, measurements, causal models, or professional standards. Domain experts still define the authoritative world mechanics, measurements, evidence, and substantive meaning of domain-specific states. The framework supplies a common productive-system and decision architecture into which those domain facts can be represented.

Autonomous agents and security

A software agent may gain a newly reachable path, credential, tool, or external route. Reachability does not itself create authorization. A previously licensed action may need re-governance when authority-relevant facts change. The PV-PP framework can serve as a governance layer that preserves state, capability, reachability, evidence, constraints, authority, selection, execution, and realized effects as distinct governed objects. The canonical framework can represent these changes; what the runtime can detect and supervise is narrower: its decision core acts only when called, and its supervision layer acts only on what monitors report and the host checkpoints, as Chapters 7 and 12 explain.

Institutions and collective productive power

Institutions preserve knowledge, coordination, authority, trust, procedure, and long-horizon productive capacity beyond the reach of an individual. Their failure can remove capabilities or interaction opportunities even when physical resources remain. The PV-PP framework can represent the distinction between nominal institutional existence, actual productive capacity, valid authority, and the conditions under which institutional actions remain viable.

Organizations and engineered operations

A firm, production process, infrastructure system, or engineered operation may have a desirable target but insufficient configuration-dependent capability to reach it safely. Equipment, staff, information, operating state, access, transformations, margins, and recovery mechanisms jointly determine what is reachable and sustainable. The same architecture can preserve minimum operating requirements and maximum exposure or safety bounds.

Households and financial decisions

A household may be able to make a payment today while lacking the reserves to sustain the resulting obligation. Money can be retained value and a capability-enabling resource without being identical to capability itself. The decision turns on more than nominal affordability: time horizon, obligations, capability, recovery, and other governing conditions can matter.

Addiction and recovery

An immediately reinforcing interaction can convey short-term value or relief while degrading health, finances, relationships, or future alternatives. A recovery-oriented analysis therefore cannot stop at immediate preference satisfaction. It must ask what the interaction does to future capability and whether the action preserves or reopens viable paths.

Human decision-making

Human decisions are made from perceived rather than perfectly known state. A person can misjudge actual capability, act from incomplete information, construct too narrow a set of alternatives, or choose an option that is immediately attractive but structurally inadequate. The

PV-PP framework provides a way to keep actual state, perceived state, capability, objectives, candidate construction, constraints, adequacy, recovery, selection, and realized consequences distinct.

CBT and case formulation

In the CBT application, these distinctions become more specific. A person can possess actual capability while underestimating it, retrieve evidence selectively, generalize a history of failure into an expectation of futility, construct too narrow a policy set, or choose short-term relief that reduces future options. The PV-PP framework separates actual from perceived capability and viability, memory from generalized expectation, policy construction from selection, and immediate relief from recovery-bearing action. The purpose is not to replace CBT, but to provide an ordered case-formulation architecture for locating different failure surfaces.

These examples exercise different parts of the architecture. Agent governance stresses reachability, evidence authority, constraints, execution licensing, and re-entry. Institutional and operational systems stress state, collective or configuration-dependent capability, transformations, reachability, adequacy, and realization. Human decision work stresses perception, information, memory, expectation, policy construction, and recovery. The value of a common ontology is not that every domain uses every construct equally. It is that the same distinctions can be reused without redefining the entire decision architecture for each field.

1.11 What the PV-PP Framework Is Not

- The PV-PP framework is not a claim that human worth is determined by Productive Power.
- It is not a claim that all value is economic output, market price, or future productivity.
- It is not a requirement to replace every scalar score with a complex non-scalar model.
- It is not a substitute for the causal science, professional standards, or measurements of the domain being modeled.
- It is not a claim that every resource, relationship, credential, permission, or favorable condition is itself Productive Power.
- It is not a guarantee that a viable decision will produce a good outcome; uncertainty remains.
- It is not a claim that the current runtime implements every capability permitted by the framework.
- It is not, in origin or scope, a complete and all-encompassing AI security framework. The PV-PP framework can nevertheless serve as a governance layer within a broader AI or autonomous-agent productive-system architecture: it can govern represented state, capability, reachability, evidence, constraints, authority, candidate actions, selection, execution licensing, and re-entry, while surrounding security systems provide functions such as identity, network control, monitoring, sandboxing, detection, and enforcement. Agent governance is therefore a substantial application of the PV-PP framework, not the limit of its scope.

1.12 Using This Guide with AI

PV-PP models are detailed. A faithful model of even a modest productive system involves typed state, perception, objectives, information governance, reachable change, constraints, recovery, an execution boundary, and tests, and the runtime that executes such a model has a substantial programming interface. In practice, few people will build a PV-PP application entirely by hand. The simulations and benchmarks described in Section 1.5 were built with substantial AI assistance, and this Guide recommends the same approach.

The division of labor matters. AI is well suited to interviewing a modeler about a problem, proposing how its facts map onto PV-PP constructs, drafting definition packets, generating runtime code, and searching for invariant violations. It is also prone to exactly the errors this Guide warns against: relabeling a useful resource as Productive Power, treating a confident source as an authoritative one, letting a desired capability stand in for a current one, or treating a successful function call as a realized effect. The modeler—or the domain expert—therefore remains responsible for meaning: what the system is, what the facts are, which conditions are non-compensable, and whether a proposed classification is right.

The Guide is written with that division in mind. Its purpose is not to make every reader able to hand-code a PV-PP application, but to make every reader able to specify a model, recognize a category error, and review what an AI assistant produces. To support that workflow, Appendix A provides one AI interview prompt per chapter. Each prompt interviews the modeler about their own problem using that chapter's concepts, produces the chapter's structured output, and ends with a checklist for the human reviewer. The prompt for this chapter, Appendix A.1, helps decide whether the PV-PP framework is the right tool for a problem at all.

The Guide helps you specify and review. AI helps you map and build. You remain responsible for meaning.

1.13 From the Intuitive Model to the Formal Architecture

Chapter 1 has deliberately stayed mostly above the formal operator stack. The goal is to establish the problem the machinery is solving. A productive actor exists in a state. It holds or has access to capabilities and resources. It perceives that state imperfectly. It has objectives. It faces a represented set of possible changes and reachable paths. It constructs candidate actions. Those candidates must be sufficiently complete, must satisfy binding constraints, must be framed against the governing domains, and must be adequate to the conditions that matter. A candidate may then be selected, licensed for execution, realized in an external environment, and returned to governance as new state and evidence.

The formal architecture exists to keep those stages and their failure modes distinct. A problem discovered upstream should not be repaired merely by manipulating a downstream score. A missing path is not the same as a prohibited path. A prohibited path is not the same as an inadequate path. An adequate candidate is not automatically the selected candidate. A selected candidate is not automatically authorized for execution. An authorization is not the environmental effect itself.

The rest of this Guide turns the intuition of Chapter 1 into a usable architecture: first the construct map, then the modeling rules, then the canonical decision cycle, then implementation, execution, testing, and worked examples.

Chapter 2 therefore begins one step earlier than the complete conceptual map: with the smallest pieces of notation needed to represent an actor, its state and Productive Power, its possible interactions, the Productive Value conveyed through interaction, and the state that exists afterward. From there it builds the complete map—PV, PP, PPP, state, objectives, information, reachability, constraints, adequacy, selection, execution, and realization—before exposing the reader to the full operator stack.

Chapter 2 — Conceptual Architecture and Construct Map

Chapter 1 explained why the Productive Value–Productive Power (PV-PP) framework exists and how the theory moved from idea to mathematics, software, and testing. Chapter 2 now begins with the smallest mathematical building blocks before showing what the finished architecture contains and how its major constructs fit together. The purpose is orientation, not operator-level specification. Exact V2.1 owner specifications still control formal semantics.

2.1 A Gentle Mathematical On-Ramp

Before reading the full decision architecture, it helps to see how the PV-PP framework begins mathematically. The notation is not intended to make the theory more abstract than necessary. Its first job is simply to let us say precisely who is acting, what condition that actor is in, what the actor can do, what interactions are represented, what value can move through those interactions, and what changes afterward.

Start with an actor and time

We use the symbol i as an index: i simply means “the actor or entity we are talking about.” Actor i might be a person, household, organization, machine, institution, software agent, or another qualified productive entity. Time is written t . A quantity carrying both i and t therefore tells us something about actor i at time t .

Represent actual Productive Power

The actor has Productive Power. We write $PP_i(t)$ for the actual Productive Power of actor i at time t . $PP_i(t)$ should not be assumed to be one number. It can be a structured profile of capabilities relevant to the application. For a person it might include developed knowledge or physical capability; for an organization, operational capabilities; for a software agent, capabilities actually available under the current configuration and conditions.

Place capability inside a broader actual state

The actor also has a broader modeled actual state, written $S_i(t)$. $PP_i(t)$ is part of that state, but it is not all of it. A simple V2.1 state envelope is:

$$S_i(t) = \{ PP_i(t), SPV_i(t), AVS_i(t), X_i(t), \dots \}$$

Read this in ordinary language as: actor i , at time t , has an actual state containing Productive Power, Stored Productive Value, actual viability state, and whatever other typed context X is required by the model. The braces mean these are components of a structured state; they are not being added together.

Represent possible interactions

For an introductory picture, suppose the actor has a represented set of possible interactions or actions. We can temporarily write $A_i(t) = \{a_1, a_2, \dots, a_n\}$. This is teaching notation, not a new canonical PV-PP primitive. It simply gives a novice a way to picture actor i facing several represented possibilities at time t . Later, the formal architecture replaces this shorthand with the governed Graph/seed substrate and Π policy-construction machinery.

Represent Productive Value through an interaction

Suppose actor i chooses or participates in interaction a_k . That interaction may convey Productive Value. For teaching purposes, $PV_i(a_k, t)$ can denote the Productive Value realized for actor i through that interaction at that time. In a multi-actor exchange, different actors may give and receive different PV, and the relevant PV can be typed or multidimensional rather than a single quantity.

Represent what changes afterward

The interaction can change what exists afterward. The simplest state-transition picture is:

$$S_i(t) \text{ -- interaction / realized effect --> } S_i(t+1)$$

The important idea is not the arrow notation. It is that an interaction has consequences. A transfer may change Stored Productive Value. Learning may change Productive Power. Injury may reduce capability. A new obligation may change state without being PP. A successful repair may restore a recovery capability. The next decision therefore begins from a potentially different state.

Distinguish actual state from perceived state

The actor normally does not decide from perfect knowledge of $S_i(t)$. The PV-PP framework therefore distinguishes actual state from perceived decision state $P_i(t)$. Perceived Productive Power, $PPP_i(t)$, can appear inside $P_i(t)$. This lets the mathematics represent a simple but important fact: what an actor is actually capable of doing and what the actor believes it is capable of doing can differ.

At this point the novice has the core pieces: an actor i ; time t ; actual state $S_i(t)$; actual capability $PP_i(t)$; retained value $SPV_i(t)$; a represented set of possible interactions; Productive Value conveyed through interaction; a later state $S_i(t+1)$; and a perceived decision state $P_i(t)$ that may differ from reality. The rest of Chapter 2 explains why a serious governed decision requires additional structure around those pieces.

2.2 Why a Construct Map Comes Before the Operator Stack

The PV-PP framework contains more than a sequence of decision operators. It contains state, capability, value, perception, objectives, information governance, history, reachable transformation structure, candidate policies, admissibility, adequacy, selection, execution, and realized state change. If those objects are not separated first, the formal stack can look like a long pipeline of symbols rather than a set of answers to distinct questions.

The map in this chapter therefore separates three layers of understanding. First is what exists or is represented: actual state, retained value, capability, perception, objectives, information, history, and the transformation substrate. Second is governance: construction, completeness, constraints, framing, adequacy, and selection. Third is realization: execution, external effects, authoritative state transition, and feedback. Cross-cutting rules such as evidence authority, state sufficiency, replay, and dependency invalidation connect the layers without erasing their distinctions.

2.3 The Whole System in One Loop

Actual state → perceived decision state + objectives → canonical governance → selected policy → execution transition → environmental realization → updated actual state → governed information, history, and perception updates.

In compact V2.1 form, the decision interface begins with perceived decision state $P_i(t)$ and adjacent objective state $O_i(t)$, then proceeds through the canonical stack: $(P_i(t), O_i(t)) \rightarrow \Phi \rightarrow H \rightarrow G \rightarrow \mathcal{R} \rightarrow \text{Graph} / \text{seed substrate} \rightarrow \Pi \rightarrow \text{completeness} \rightarrow \text{Constraints} \rightarrow \text{Domain Framing} \rightarrow \text{Adequacy} \rightarrow \Sigma \rightarrow \epsilon$. After attempted execution and environmental realization, Layer 1 updates actual persistent state $S_i(t+1)$ from the realized effect and admissible context or evidence.

Each symbol in that sequence names a distinct canonical stage. Table 2.1 gives each stage's owner-specification name and the single question it answers. The table is orientation only: Chapter 6 walks through the stages in order, and the owning V2.1 specifications control their exact semantics.

Table 2.1 — Canonical V2.1 decision stages

Stage	Canonical name	Question the stage answers
Φ	Pressure Field operator	How strongly are the relevant domains constraining behavior now?
H	Collapse Horizon operator	Under baseline continuation, how long until each domain would breach its threshold?
G	Governing Domain Identification	Which dependency-closed set of domains actually governs this decision?
$\mathcal{R}$	Regime Classification	Given pressure, horizons, and governing domains, what behavioral posture applies?
Graph / seed	Graph Layer / seed substrate	Which transformations, compositions, and routes are represented as reachable?
Π	Policy Construction operator	Which bounded candidate policies can be built from that represented substrate?
Π completeness	Π Completeness Validation	Is every materially relevant, feasible policy class actually present?
Constraints	Constraint System	Which candidates are infeasible or impermissible under binding conditions?
Domain Framing	Domain Framing	What preservation target must the surviving candidates be judged against?
Adequacy	Adequacy	Does the policy preserve viability across the governing set until recovery becomes reachable?
Σ	Selection operator	Which adequate policy is selected—without collapsing the governing structure into a score?
ϵ	Execution	How does the selected policy move toward attempted execution, and when must it

Stage	Canonical name	Question the stage answers
		continue, stage, or abort?

The frozen runtime also evaluates joint-recovery feasibility for recovery obligations that are already active: after Π completeness it binds any mandatory current-period recovery actions into every candidate before Constraints, and it checks joint feasibility after Domain Framing, before Adequacy. Some supporting documents draw that step into the stack itself. It is an implemented runtime stage serving Adequacy rather than a separate operator in the V2.1 owner stack; Chapter 6 explains how the two relate.

This loop is deliberately asymmetric. The framework reasons from a representation of the world, but the representation is not the world. Selection is not execution. Execution is not guaranteed realization. A successful function call is not automatically proof that the intended environmental effect occurred. The authoritative state transition belongs to the application or domain model that owns the world mechanics.

2.4 Actual Persistent State: $S_i(t)$

Actual persistent state is the modeled state that presently exists for actor or entity i . A common V2.1 envelope is $S_i(t) = \{PP_i(t), SPV_i(t), AVS_i(t), X_i(t), \dots\}$. Productive Power is therefore a component of state, not the whole state. Stored Productive Value, actual viability state, resources, obligations, institutional standing, environmental conditions, and other typed variables retain their own roles.

This prevents a common modeling error: treating every favorable condition as capability. A person may own a tool without access to it. An organization may have access without authority to use it. A software agent may possess a credential that is revoked or inapplicable. A machine may contain a component that is present but inoperable. State must be rich enough to preserve distinctions that can change what transitions, policies, execution, or governance are possible.

V2.1 calls this transition-relevant state sufficiency. For the declared modeling purpose and admissible context, current state must contain or reference every presently material consequence capable of changing modeled transition structure, reachability, policy construction, execution, or governance. Complete causal history does not have to remain live state; only the historical residue that is still causally material must be represented in current state or referenced appropriately.

State can also be represented at more than one qualified scale. An individual, team, department, organization, or institution may each have its own $S_i(t)$ when that level independently qualifies as a persistence-bearing productive entity. Likewise, a short interaction can occur inside a longer episode or process. These levels should not be collapsed into one another: member state is not automatically collective state, and the state consequence of a long-running interaction is not necessarily the arithmetic sum of its smaller episodes. Chapter 3 supplies the modeling tests for collective qualification and nested structure.

2.5 Perceived Decision State: $P_i(t)$, PPP, and Perceived Viability

Decisions are made from represented state, not from omniscient access to actual state. $P_i(t)$ is the decision-time representation of relevant state. The modeler does not simply invent a second version of reality. Instead, the application defines the informational pathways by which the actor can form a representation: direct observation, sensors, reports, records, retrieved memory or evidence, expectation or model state, confidence and uncertainty, bounded attention, and other

licensed informational inputs. $P_i(t)$ is formed from what is actually available through those pathways at decision time.

Perceived Productive Power (PPP) is one component that may appear inside $P_i(t)$. PPP represents perceived capability, while PP represents actual capability. The framework deliberately preserves the possibility that they diverge. A person may underestimate a real capability; an organization may overestimate operational readiness; an agent may represent a route as unavailable when it is actually reachable. Perceived viability can likewise diverge from actual viability.

That divergence is often a consequence of interaction over time rather than a deliberately constructed false state. Suppose a coffee machine physically has no cups left but its inventory record still reports twelve. Actual dispensing PP may be absent while the decision process still represents dispensing as available. Or a buyer may see a "Ready" display and reasonably perceive the machine as able to deliver coffee even though an internal actuator has failed. In both cases, the perceived state follows from the information available to the decision-maker; it is simply no longer aligned with actual state.

Perceived state is therefore dynamic. Interaction produces observations and evidence; memory and records retain some of them; later events can confirm, supersede, or invalidate earlier representations. A failed dispense, a maintenance report, a new sensor reading, or a remembered prior event can change $P_i(t)$ without the underlying actual state changing at that same moment. Conversely, actual state can change before the actor receives evidence of the change. Modeling $P_i(t)$ therefore requires asking not only "What does the actor currently represent?" but also "What information produced that representation, how current is it, and what could make it wrong?"

$P_i(t)$ is broader than PPP. This is important because a decision can be wrong even when capability is perceived correctly: the error may concern a constraint, a resource, an external condition, an obligation, an authority fact, reachability, or another part of represented state. Chapter 4 develops the information, memory, evidence, temporal provenance, and history mechanisms that govern how such representations are formed, retained, challenged, superseded, and updated.

2.6 Objectives: $O_i(t)$

V2.1 carries objective state $O_i(t)$ adjacent to perceived decision state. Objectives describe what an actor or application is currently trying to achieve, preserve, prevent, recover, or develop. They are explicit inputs to the decision interface, but they do not become hidden utility functions or override the rest of the architecture.

An objective does not create reachability, capability, authority, adequacy, or execution permission. Conflicting objectives may coexist without the framework inventing weights between them. A capability-development objective is especially important: wanting or planning to acquire a capability does not make that capability current PP. The capability must be realized and evidenced through an actual state transition before it becomes current capability.

2.7 PV, SPV, PP, and Viability

Chapter 1 introduced PV and PP. The complete map adds Stored Productive Value (SPV) and viability. PV is value actually transferred, delivered, realized, imposed, or otherwise conveyed through interaction. SPV is realized PV retained as a holding for later expenditure, transfer,

exchange, consumption, or conversion. PP is actual capability under stated conditions to produce or preserve PV. These are related but not interchangeable.

A cash payment can be PV when transferred and SPV when retained by the recipient. That retained money can enable later capability, but the money is not automatically PP. A skill may constitute PP even when it is not presently producing a transfer. A viability condition can govern whether a capability may be safely or legitimately exercised without itself being PV or PP.

The same discipline separates knowing from being able. A novice can hold correct declarative knowledge of a procedure without yet possessing the practiced capability to perform it reliably; an experienced practitioner can possess capability that no explicit statement of their knowledge fully captures. Knowledge and evidence can inform state and decision-making, and repeated realized interaction can build PP, but neither should be counted as the other. What ordinary language calls wisdom may combine retained knowledge, experience-built capability, memory, expectation, and judgment; the PV-PP framework does not treat it as a single canonical primitive unless an application defines it explicitly.

Viability asks whether operation remains within the governing conditions that matter for the modeled system. Those conditions may include lower bounds, upper bounds, hard constraints, recovery requirements, evidence requirements, authority, and other domain-specific conditions. Corridors are a useful way to describe regions in which those conditions remain jointly satisfied, but the corridor is defined by the underlying governed structure rather than replacing it.

2.8 Information, Memory, Expectation, and Transfer History

The PV-PP framework keeps several informational objects separate because they answer different questions. Information can be retrieved or observed. Memory concerns retained or reconstructed decision-relevant material. Expectation or model state concerns what is inferred or predicted. Confidence concerns uncertainty. Source authority concerns who or what is qualified to establish a fact. Applicability concerns where the information applies. Permitted use concerns which governance surfaces may use it.

Transfer History is a structured event-history substrate. It can record interactions, transfers, commitments, failures, recoveries, or other relevant events. It is not automatically current belief, current truth, current PP, current state, or authority. Historical events can explain why current state exists without being duplicated as live-state primitives.

This separation supports two different sufficiency problems. Prospective state sufficiency asks whether current state contains what is presently material to future governance. Replay sufficiency asks whether enough decision-time material was retained to reconstruct a prior governed decision without projecting later knowledge backward into the historical basis.

Chapter 4 develops each of these distinctions in turn: current decision-use information versus historical record, source authority versus confidence, applicability versus permitted use, and prospective state sufficiency versus replay sufficiency.

2.9 Graph, Transformations, Reachability, and the Represented Productive Envelope

The PV-PP framework represents possible change through Graph and transformation structure. At a high level, the Graph or seed substrate represents governed mechanisms, opportunities, and constructible paths within the represented productive envelope. A transformation describes a modeled way in which state may change under stated conditions.

This Guide uses the phrase *represented productive envelope* for the whole body of represented transformations, routes, compositions, and successor configurations that the governed model currently admits. It is a teaching label for the Graph/seed substrate and the represented structure around it, not an additional canonical object.

Reachability answers whether a represented path or destination is available under the current modeled conditions. It is distinct from capability, desirability, and authorization. Productive Power can affect reachability because changed capability can make transformations possible or impossible, but reachability is not itself a PP magnitude. Likewise, a technically reachable action can remain prohibited by constraints or authority.

This distinction is central in dynamic systems. If a route disappears, a candidate may cease to be constructible even though its destination remains desirable. If a new route appears, the represented Graph may become incomplete until the new path is admitted and governed. In autonomous-agent applications, newly reachable network or tool paths are therefore governance-relevant without being treated as automatic permission to use them.

2.10 Policy Construction, Completeness, Constraints, Framing, and Adequacy

The framework does not jump directly from “what is reachable?” to “which action scores highest?” Π constructs candidate policies from the represented substrate. Completeness asks whether the relevant candidate surface is sufficiently represented for the declared decision. Constraints then apply binding admissibility, authority, permission, legal, technical, or other conditions.

Domain Framing preserves the consequence and governing-domain structure that must remain visible during comparison. Adequacy asks whether a surviving policy satisfies the governing target and required conditions. These stages are intentionally distinct. A candidate can be reachable but inadmissible, admissible but inadequately represented, or admissible and complete yet inadequate to the governing requirements.

The no-downstream-repair rule follows from this structure. Missing state, missing candidate families, invalid authority, or defective evidence should be repaired at the surface that owns the problem. A downstream selector should not silently compensate for an upstream omission by changing weights or scores.

2.11 Selection, Execution, Environmental Realization, and Layer-1 Transition

Σ selects from the surviving structured policy set. ε governs the transition from selected policy toward attempted execution. They are not interchangeable. Selection does not itself mutate the environment, and ε is not a second selector.

When the current runtime uses native call-through, it can invoke a registered synchronous function after the required governance checks and return structured results or errors. The application and external systems remain responsible for the actual environmental effect and authoritative state update. A callable can return successfully while the intended real-world effect fails, only partially occurs, or produces an unexpected consequence.

Layer 1 therefore updates actual state from realized effects under the application's transition rules: $S_i(t+1) = T(S_i(t), \text{realized action/effect, admissible context/evidence})$. After that transition, information, history, memory, expectation, confidence, and perceived state may update through their licensed interfaces. Later knowledge does not retroactively become part of the earlier decision state.

2.12 Cross-Cutting Rules That Keep the Architecture Honest

Several V2.1 rules apply across the entire map. First, heterogeneous governing domains should not be collapsed into one scalar merely because they are measurable. Second, actual state, perceived state, objectives, memory, expectation, confidence, information authority, Transfer History, and replay evidence remain distinct. Third, PV, SPV, PP, viability, resources, constraints, objectives, and consequences retain their assigned roles.

Fourth, capability cannot be laundered forward from intention. The path is intention → discovery/training/investment/action → realized state transition → evidenced capability → current PP. Fifth, framework capability and runtime implementation must be reported separately. The framework can represent governance requirements that the current runtime does not independently implement.

Finally, authority is time- and dependency-sensitive. If authority-relevant facts change, previously valid downstream decisions may become stale and require governed re-entry at the earliest invalid stage. The canonical framework can represent asynchronous observation, invalidation, and renewed governance when an implementation supplies the instrumentation, event delivery, and execution supervision to feed them. The runtime's decision core is synchronous: it provides substantial pre-call dependency and authorization machinery, and Chapter 7 states its boundary in full. Runtime v2.1.167 adds part of the rest around that core—monitor intake, in-flight checkpoints, and governed stop requests—without creating a parallel decision system; Chapter 12 shows how, and what it still leaves to the host.

2.13 A Compact Walk-Through of One Governed Decision

Consider an autonomous service agent asked to perform an external operation. Actual state records the currently known tools, credentials, system conditions, retained resources, capability, and other relevant facts. $P_i(t)$ represents what the agent or governing application currently believes about that state. $O_i(t)$ identifies the requested objective.

The upstream architecture prepares the relevant representation, horizon, governing domains, and regime. Graph represents the currently admitted routes and transformations. Π constructs candidate policies. Completeness asks whether the candidate surface is sufficient. Constraints remove actions that are not permitted or authorized. Framing preserves the domains that matter, and Adequacy checks whether the surviving candidates satisfy the required conditions. Σ selects a surviving policy. ϵ governs whether and how it may proceed toward execution.

Suppose that after authorization a new network route becomes externally reachable. That fact does not automatically become authority. With suitable instrumentation, an external observation can update the relevant represented state or evidence, invalidate the artifacts that depend on it, and cause re-entry at the earliest affected stage. Whether the current runtime can notice such a change while a call is already in flight is a separate, implementation-level question, taken up in Chapter 7.

The same pattern can be translated to a human, household, organization, or engineered system. The domain facts change, but the questions remain recognizable: What state actually exists? What does the actor believe? What is it trying to accomplish? What paths are represented and reachable? What candidates are constructed? Which are admissible and adequate? What is selected? What is authorized for execution? What actually happens, and what state follows?

2.14 From the Construct Map to Application Modeling

At this point the reader has the complete conceptual loop but has not yet been asked to build a domain model. That is deliberate. The next step is not to memorize Φ , H , G , $\mathcal{R}$, Π , Σ , and ϵ . It is to learn how to decide what belongs in state, what counts as PV or PP, which resources merely enable capability, how to represent actors and collective entities, how to identify objectives and constraints, and how to define the environment that owns actual state change.

Appendix A.2 contains an AI interview prompt that sketches this construct map for your own problem, as a first pass before the detailed modeling of Chapter 3.

Chapter 1 answered why the PV-PP framework exists. Chapter 2 answered what the architecture contains. Chapter 3 turns that map into a modeling method.

Chapter 3 — Modeling a Productive System

Chapter 2 supplied the conceptual map. Chapter 3 turns that map into a modeling method. The task is not to force a domain into Productive Value–Productive Power (PV-PP) terminology. It is to identify the real productive structure of a bounded situation and assign each material fact the role it actually plays.

A useful PV-PP model begins before code and before scoring. The modeler first defines the scenario boundary, discovers the interactions that matter, identifies participants and productive roles, classifies PV, PP, retained value and broader state, and states the conditions under which paths are viable, recoverable, authorized, and consequential. Only after that structure is explicit should it be mapped into the runtime.

Model the productive system first. Implement the runtime representation second.

3.1 Start With a Bounded Scenario

The first modeling decision is scope. A productive system can be described at almost any scale, but an executable model needs a bounded episode, transaction, workflow, operating interval, or other declared unit of analysis.

For the scenario, state what starts it, what ends it, what is deliberately excluded, and what time horizon matters. Also ask whether capability or represented structure can change inside the boundary and whether an external consequential action can outlive a single decision step.

Running example: coffee vending

Consider a buyer attempting to purchase one coffee from an automated machine. The example is deliberately ordinary. The buyer, machine, payment service, inventory, dispensing mechanism, refund mechanism, and evidence of delivery can all matter, but they do not all occupy the same ontological role. The modeling exercise is to preserve those differences.

The scenario begins when the purchase attempt is initiated and ends when coffee is delivered, the transaction is refused, recovery completes, or the episode reaches a declared unresolved/failure condition. That boundary is already more useful than saying simply, "maximize successful sales."

3.2 Discover the Interaction Before the Decision Rule

An interaction is a bounded event, transaction, exchange, operation, or workflow step in which PV flows and PP or other consequential state may change. Start by asking what actually happens between participants, not what score should be optimized.

Question	Modeling purpose
Trigger	What causes the interaction to begin?
Participants	Who or what acts, receives, supplies, governs, reviews, recovers, or is affected?
PV flow	What is actually transferred, delivered, realized, imposed, or conveyed?
Required capability	What PP and configuration must exist for the

	interaction?
Authority and evidence	What permission, control, provenance, or evidence is required?
Bounds and corridors	What can block, redirect, degrade, recover, or terminate the path?
Execution	Does a selected action invoke a real external function or effect?
Result	What state, capability, obligation, relation, evidence, or represented structure changes afterward?

Nested interactions should be added when they materially change governance. In the coffee example, payment authorization, payment capture, dispensing, delivery confirmation, and refund may need separate representation because failure at one step changes state, evidence, recovery, or authority at another.

3.3 Identify Participants and Their Roles

A participant may be a person, software agent, machine, institution, subsystem, tool, service, or relevant environmental component. Participation alone does not make every object an independent productive entity.

Useful roles include actor, recipient or affected party, source of PV or evidence, reviewer or approver, recovery actor, governing authority, execution target, and qualified productive unit. One participant may occupy several roles, and a role can change across interactions.

In the vending example, the buyer initiates the purchase and receives coffee; the machine participates in payment and dispensing; a payment service may authorize or realize a monetary transfer; a refund subsystem may be the recovery mechanism; and an external service may provide evidence or execution infrastructure. Treating all of these simply as “agents” would discard useful structure.

3.4 Classify PV, SPV, PP, Resources, and Broader State by Role

This is one of the most important modeling disciplines in the PV-PP framework. Do not classify something by what it is called in ordinary language. Classify it by the role it occupies in the modeled interaction.

Productive Value (PV)

Classify something as PV when it is transferred, delivered, realized, imposed, or otherwise conveyed through the interaction. In the vending episode, money can be PV when transferred to the seller; coffee can be PV when delivered to the buyer; a refund can be PV when returned. Information can also be PV when it is actually conveyed. For example, if the machine informs the buyer that it is out of coffee, that information is conveyed through the interaction and can immediately alter what the buyer does next even though no coffee changes hands.

Stored Productive Value (SPV)

Realized PV can later occupy a retained-value role. Money received and held, inventory retained for future exchange, or another stored claim may be SPV when the application represents it as retained value available for later expenditure, transfer, consumption, or conversion.

Productive Power (PP)

Classify something as PP only when it constitutes actual capability, under stated conditions, to generate, transform, coordinate, deploy, preserve, protect, recover, or otherwise produce PV. Purchasing capability, payment-acceptance capability, dispensing capability, refund capability, and temperature-maintenance capability may therefore be PP in the vending model.

Resources, mechanisms, and context

A resource can enable PP without itself being PP. A cup, network connection, credential, machine component, tool, or retained reserve may be necessary for a capability to exist or be usable, but the resource and the capability should remain distinct. The cash reserve is a useful example. Cash held for later use is ordinarily SPV. If the machine must be able to issue a cash refund, that SPV may be one input to refund PP, but it is not identical to the refund capability. Actual refund PP may depend jointly on sufficient cash being present, the refund subsystem having access to it, valid permission to release it, control of the relevant mechanism, and a mechanically operable dispenser or payment rail. Availability, evidence, obligations, environment, and other context can likewise be material state without being PV, SPV, or PP. The same object can occupy different roles across time or relationships.

Do not solve ontology problems by putting every useful thing into PP.

3.5 Build the Actual State Before the Perceived State

Chapter 2 introduced the actual-state envelope $S_i(t) = \{PP_i(t), SPV_i(t), AVS_i(t), X_i(t), \dots\}$. Chapter 3 now asks what must actually be placed in that state for the declared scenario.

Include or reference every presently material consequence that can change modeled transition structure, reachability, policy construction, execution, governance, or viability. This is transition-relevant state sufficiency. It does not require complete causal history to remain live state.

For the vending machine, relevant actual state might include inventory, payment state, stored funds, power, cup availability, actuator status, temperature condition, refund reserve, network availability, active obligations, and evidence state. Some belong to PP, some to SPV, and some to broader context.

Only after the model has identified the relevant actual-state structure should it specify how the decision-making actor forms perceived decision state $P_i(t)$. The modeler does not invent a second arbitrary version of reality. Instead, the application identifies the informational pathways available to the actor: direct observation, sensors, records, reports, retrieved memory, prior interactions, inferred expectations, confidence, uncertainty, and other licensed inputs. $P_i(t)$ is the decision-time representation produced from those inputs.

Actual and perceived state can therefore diverge naturally over time. Suppose the machine physically has no cups left, but its inventory record still reports twelve. The actual dispensing capability may be absent while the governing process still represents dispensing as available. Or the buyer may see a "Ready" display and reasonably believe coffee can be delivered even though an internal actuator has failed. No one had to deliberately create a false perceived state; the information available to the decision-maker simply failed to track the current actual state.

A useful modeling question is therefore not merely "What does the actor believe?" but "By what observation, memory, evidence, or inference did that representation arise, how current is it, and what could make it wrong?" Later interactions can correct or further distort $P_i(t)$. A successful test, failed dispense, maintenance report, new sensor reading, or remembered prior event can

update the representation. This makes the actual/perceived distinction dynamic rather than a one-time comparison.

3.6 Separate Possession, Access, Permission, Control, and Operability

These dimensions often coincide in simple examples, which makes them easy to collapse. They must remain separate whenever they can diverge.

Dimension	Question	Coffee example
Possession	Is the resource held or present?	The machine contains a refund reserve.
Access	Can the relevant subsystem reach it?	The refund subsystem can reach the reserve.
Permission	May it be used for this action?	Transaction rules permit a refund.
Control	Can the actor command the relevant mechanism?	The subsystem has command authority over the payment or dispensing rail.
Operability	Will the mechanism function now?	The network, actuator, or refund mechanism is actually functioning.

A resource can therefore be possessed but inaccessible, accessible but unauthorized, authorized but uncontrolled, or controlled but inoperable. None of those should silently become current PP.

3.7 Model Configuration-Dependent and Joint Capability

Productive Power is often configuration-dependent. A capability may exist only when several resources, roles, mechanisms, and conditions are jointly present. This is why PP should not automatically be treated as an additive inventory.

In the vending scenario, dispensing PP exists only when the relevant configuration is jointly available: coffee inventory, a cup, power, adequate temperature, an operable actuator, and a usable control path. A model that records each component as present but never checks the configuration can overstate actual capability.

The same issue appears in organizations. A team may possess all required skills across its members but still lack collective capability because coordination, authority, timing, tooling, or role complementarity is missing. Configuration belongs in the model whenever it changes what can actually be done.

3.8 Define Objectives Without Turning Them Into Hidden Utility

Where purpose materially affects discovery, interpretation, or evaluation, define objective state $O_i(t)$ separately from actual state $S_i(t)$ and perceived state $P_i(t)$. An objective can describe what the actor or application is trying to achieve, preserve, prevent, recover, or develop.

In the vending scenario, an objective might be to complete the requested purchase, preserve correct payment state, or recover from failed delivery. None of those objectives makes coffee available, creates refund authority, or turns an inoperable mechanism into current PP.

An objective does not create capability, reachability, authority, adequacy, or execution permission. Wanting the machine to refund a failed purchase does not make the refund mechanism operable. Wanting an agent to acquire a future capability does not make that capability current PP.

If objectives conflict, do not invent weights merely to force them into one score. Record their identities, sources, scope, lifecycle, temporal bounds, relations, and authority where those facts matter.

3.9 Identify Constraints, Thresholds, Corridors, and Governing Domains

Next ask what conditions define viable or admissible operation. Thresholds are structural boundaries, not merely score adjustments. Hard constraints should remain explicit.

In the vending scenario, one lower bound might require sufficient inventory and operable dispensing capability before payment is finalized. A recovery condition might require a reachable refund path if delivery fails. A transaction that captures payment while neither delivery nor recovery is viable can therefore be rejected as structurally inadequate even if the payment call itself succeeds.

A governing domain is one whose failure changes viability, admissibility, defensibility, recoverability, or structural adequacy for the scenario. The model should say why the domain governs rather than hiding priority in tuple order or incidental sorting.

Corridors provide a useful path-level view. A normal corridor may represent the expected viable path; a degraded corridor reduced sufficiency; a failure corridor blocked intended flow; a recovery corridor restoration or compensation; a fallback corridor substitution; and a negative corridor movement toward unrecovered loss. The names are useful only when the underlying state, bounds, constraints, and transition structure are explicit.

For coffee vending, “payment accepted” is not by itself adequate success if coffee cannot be delivered and refund is unreachable. A locally successful step can coexist with system-level false success.

3.10 Model Recovery as Structure, Not as a Hope

Recovery should be decomposed. Identify the reserve available for recovery, the recovery capability, the concrete mechanism that realizes it, the prerequisites on which it depends, whether the recovery-capable region is currently reachable, and the obligation or function that recovery must restore.

In the vending scenario, the refund reserve, refund capability, payment or cash-dispensing mechanism, network availability, authorization, and current reachability of the refund path are separate modeling facts. A refund reserve is not refund PP; refund PP is not the payment rail or dispenser that realizes it; and the presence of all three does not prove that the refund is currently reachable, because a network outage, revoked authority, or failed mechanism can break the path. This is why “the machine has money” is not enough to establish that it can presently refund the buyer.

This decomposition becomes especially important in human, institutional, and autonomous-agent applications, where apparent backup capacity may disappear precisely when the primary path fails.

3.11 Decide Whether a Group Is Actually a Productive Entity

The PV-PP framework can model collective and nested entities, but it does not grant entity status merely because a group has a name, legal label, shared objective, or registered collection of members.

The vending machine itself might be modeled as one engineered productive unit for this scenario, while the merchant, payment processor, and service organization remain separate entities or contextual systems. If the model instead treats a fleet of machines as one productive entity, that fleet-level entity must independently satisfy the qualification conditions rather than inheriting entity status merely because its machines share an owner.

For the modeled purpose, a persistence-bearing collective should have a meaningful boundary, persistent state not adequately represented as an unstructured list of member states, a transition structure, entity-level continuity or viability conditions, state retention across membership turnover, governance or coordination connecting member action to entity action, and an analytically useful reason to treat the aggregate as a unit.

Where those conditions are not met, represent the group as an interaction network, coalition, institution, contextual structure, or collection of participants rather than inventing an independent entity.

Collective PP is not the arithmetic sum of member PP. Entity capability can depend on member configuration, roles, tools, information, authority, commitments, and coordination. Member and entity viability can also diverge: an organization can remain viable while a member leaves, and members can remain viable after an organization dissolves.

3.12 Define the Environment and the Execution Boundary

A PV-PP application must state who owns authoritative world mechanics. The framework and runtime can govern represented actions, but the external environment or host application owns the actual consequential transition.

In the vending scenario, the runtime may authorize and invoke a dispense function, but the machine and its host environment determine whether a cup actually drops, coffee actually flows, and delivery actually occurs. Completion evidence must therefore come back from the world-owning layer rather than being inferred solely from the fact that a function returned successfully.

For each consequential action, identify the represented action, required execution authority, actual callable or mechanism, expected external effect, possible failure ambiguity, completion evidence, and Layer-1 consequence. A function returning success is not automatically proof that the intended real-world effect occurred.

This boundary also prevents a common implementation mistake: treating the runtime as the entire world model. Domain measurements, authoritative state, physical or institutional mechanics, external services, and real effects normally remain application responsibilities and feed governed observations/results back into the PV-PP framework.

3.13 Identify What Can Change and How the Model Will Know

Static models are easier, but many important productive systems are dynamic. Before implementation, identify which facts can change during an episode: capability, configuration, Graph reachability, recovery routes, evidence authority, objectives, constraints, relationships, or other represented structure.

In the vending scenario, inventory can fall, the network can disappear, a payment authorization can expire, an actuator can fail, or a refill can restore capability. The model should state which observations establish those changes and which earlier governance artifacts become stale when they occur.

Then identify the evidence that establishes each material change: source, provenance, freshness, uncertainty, authority, applicability, permitted use, dependencies, and invalidation conditions. A changed fact should invalidate the governance artifacts that actually depend on it rather than triggering arbitrary downstream patching.

The framework can represent asynchronous observations and governed re-entry when an implementation supplies event intake and supervision; Chapter 7 describes what the runtime's decision core can and cannot do here, and Chapter 12 the intake and supervision that runtime v2.1.167 adds.

3.14 Keep History, Current State, and Replay Obligations Separate

A productive-system model may depend on prior transfers, failures, commitments, trust, learning, or expectation update. Historical information can explain why current state exists without becoming current state itself.

In the vending scenario, a prior payment capture, failed dispense, attempted refund, or maintenance event may matter later. The current model should carry forward the consequences that remain material while retaining historical event evidence separately when replay or dispute reconstruction is required.

Transfer History can record relevant events. Current state should contain or reference the consequences that remain presently material. If the application must later reconstruct why a prior decision was made, separately retain enough decision-time evidence for replay. Do not use later knowledge to repair the historical basis.

This is also where experience can matter without becoming a vague catch-all variable. Repeated realized interactions may change skill, expectations, confidence, memory, relationships, or other state. The model should assign each lasting consequence to the state surface that owns its present effect.

3.15 Build a Scenario and Model Definition Packet Before Coding

The output of this chapter is a structured scenario definition that can be handed to implementation. It does not need to be bureaucratic, but it should make the model's commitments inspectable before code hides them.

For the vending example, the packet would not simply say "sell coffee." It would record the purchase episode boundary; buyer, machine, payment and recovery roles; money, coffee and informational PV flows; retained inventory and funds; payment, dispensing and refund PP; actual and perceived state; required configurations; constraints and recovery conditions; evidence sources; consequential calls; and the authoritative evidence that establishes delivery, failure, or refund.

Packet element	What to record
Scenario boundary	Start, end, exclusions, horizon, episode semantics.
Interactions	Triggers, participants, PV flows, required PP,

	consequences and nested interactions.
Participants and entities	Roles, boundaries, qualification, membership and governance where collective.
State basis	PP, SPV, viability, resources, obligations, context and authoritative references.
Perceived state	PPP and other represented beliefs, uncertainty, source and freshness.
Objectives	Identity, scope, lifecycle, source and dependencies.
Capability/configuration	PP holders, enabling resources, access/control/operability, joint requirements.
Graph/change structure	Transformations, reachable paths, dynamic structure and admission conditions.
Governance conditions	Constraints, thresholds, governing domains, corridors, adequacy and recovery.
Information/evidence	Source, provenance, authority, applicability, permitted use, invalidation.
History/replay	Transfer History needs, current historical residue, replay-retention obligation.
Execution boundary	Action-to-effect mapping, authority, callable/mechanism, completion evidence, Layer-1 semantics.
Open assumptions	Unresolved semantics, deliberate exclusions, uncertainties and external dependencies.

Appendix A.3 contains an AI interview prompt that builds this packet for your own problem by working through the sections of this chapter in order. Review its output against the walk-through below and the common errors in Section 3.18.

3.16 Complete Walk-Through: The Coffee Vending Model

The informal problem is simple: a buyer wants coffee from a vending machine. The PV-PP model begins by bounding one purchase episode rather than optimizing an abstract sale score. The episode starts with a purchase attempt and ends in delivery, refusal, completed recovery, or a declared unresolved/failure state.

The participants are the buyer, vending machine, payment service, and any recovery or service mechanism that materially affects the episode. Money is PV when transferred; coffee is PV when delivered; an out-of-coffee notice is informational PV when conveyed; retained funds and inventory can occupy SPV roles. Purchasing, payment acceptance, dispensing, temperature maintenance, and refunding can be PP when those capabilities actually exist under the required configuration.

Actual state includes inventory, funds, power, network condition, cup availability, actuator status, temperature, payment state, obligations, and evidence. Perceived state is formed from the records, sensors, displays, reports, and other informational inputs available to the relevant

decision process. If inventory records say cups remain after the physical supply is exhausted, $P_i(t)$ can diverge from $S_i(t)$ and PPP can overstate actual dispensing PP.

The model then identifies configuration and governance. Dispensing may require inventory, cup, power, temperature, actuator, and control path jointly. Payment or delivery rules may impose hard constraints. Adequacy may require either successful delivery or a viable recovery path after payment. A captured payment with no coffee and no reachable refund is false success, not merely a low-scoring outcome.

Recovery is decomposed into reserve, capability, mechanism, prerequisites, reachability, and obligation. Execution is also separated from realization: authorization to call dispense is not the same as coffee being delivered. The host machine/environment owns the actual transition and supplies evidence of what occurred. The resulting state and evidence then become inputs to later governance.

The finished model is therefore not “choose whether to sell coffee.” It is a typed representation of value flow, retained value, capability, state, perception, objectives, configuration, constraints, recovery, evidence, execution, and realized state change. That structure is what is handed forward to implementation.

3.17 Second Worked Model: A Household Replacing a Failing Car

A second example tests whether the modeling method travels beyond the vending-machine domain. Consider a household whose primary car is becoming unreliable. One household member needs dependable transportation to work, the household has limited savings, and repair, replacement, public transportation, borrowing, and temporary rides are possible but not equally available.

Scenario boundary. The modeled episode begins when the household must decide how to preserve transportation after evidence of serious vehicle unreliability and ends when a transportation plan is adopted and its immediate financing or repair commitment is realized. The model deliberately excludes the household's entire financial life except where obligations, reserves, income, credit, or other conditions materially affect this decision.

Interactions and PV. A repair shop may convey diagnostic information and repair service. A seller may convey a replacement vehicle in exchange for money or a financed obligation. A lender may convey financing. A family member may convey transportation assistance. These are different PV flows; the model should not reduce all of them to purchase price.

State, SPV, and PP. Household savings are ordinarily SPV. Employment income, obligations, vehicle condition, available time, credit terms, and access to alternative transportation are broader state. Transportation PP is the actual capability to get household members where they need to go under the relevant conditions. Savings can enable that capability without being identical to it. Driving skill may be PP; a car may be an enabling resource whose availability and operability help realize transportation PP.

Actual and perceived state. The car may be mechanically worse than the household realizes, or a repair estimate may make the household believe replacement is unavoidable when a second diagnosis would reveal a viable repair. $P_i(t)$ is formed from observations, mechanic reports, prior breakdowns, financial records, remembered experience, lender information, and uncertainty. The model therefore records not only the household's belief but the informational basis for it.

Objectives and constraints. Objectives might include preserving dependable work transportation, avoiding an unaffordable obligation, retaining an emergency reserve, and restoring a stable transportation path. A lender's approval does not prove household adequacy. A vehicle may be purchasable yet violate a minimum-reserve or payment-sustainability condition. Conversely, a cheap repair may be immediately affordable but inadequate if expected reliability remains below the required transportation threshold.

Reachability and recovery. Repair may be technically represented but unreachable if the shop cannot obtain parts in time. Replacement may be reachable only if financing, down payment, insurance, and delivery conditions all hold. A temporary ride arrangement can form a recovery or bridging path. If that fallback disappears, a previously adequate plan may require re-governance.

Execution and realization. Selecting "repair the car" does not repair it. Payment authorization, shop acceptance, parts availability, completed work, and post-repair operability are distinct. Likewise, selecting a replacement vehicle does not mean financing closes or the vehicle is delivered. The environment supplies the realized effects that update actual state.

The household example demonstrates the same modeling method without relying on software-agent concepts. The domain facts differ sharply from coffee vending, but the PV-PP framework still asks the same structural questions: what value moves, what capability actually exists, what state and perception govern the decision, what paths are reachable, what conditions are non-compensable, what recovery remains, and what actually happens after execution.

3.18 Common Modeling Errors

- Starting with a reward or utility function before identifying the productive structure.
- Treating every useful resource, permission, relationship, credential, or favorable condition as PP.
- Treating PP as the complete state.
- Equating possession, access, permission, control, and operability.
- Assuming collective PP is the sum of member PP.
- Treating an objective as evidence that a capability or reachable path exists.
- Treating technical call success as adequate environmental realization.
- Treating historical records as automatically current truth or current state.
- Inventing scalar weights for domains whose compensability has not been established.
- Allowing missing semantics to be supplied implicitly by code.
- Treating what a new actor inherited—configuration, tools, a prompt, permissions—as its current PP (5.12).

3.19 From Modeling to Information Governance

A well-formed productive-system model tells us what facts matter, where they belong, and which transitions or decisions depend on them. It does not yet tell us in sufficient detail how information becomes admissible evidence, how memory and Transfer History differ, how later evidence changes current representation without rewriting the past, or how source authority and temporal provenance are governed.

Chapter 2 mapped the architecture. Chapter 3 defined the productive system. Chapter 4 asks how that system knows what it thinks it knows.

That is the next step because even a perfectly structured domain model can produce bad governance if its information is stale, misattributed, inapplicable, unauthorized for use, or historically reconstructed with knowledge that was not available at decision time.

Chapter 4 — Information, Memory, History, and Evidence

Chapter 3 defined the productive system: actors, state, Productive Value, Productive Power, objectives, constraints, recovery, environment, and the execution boundary. Chapter 4 now asks a different question: how does that system know what it thinks it knows?

The Productive Value–Productive Power (PV-PP) framework does not treat information as one undifferentiated pool. A statement can be accurate but unauthorized for a particular use, relevant but outside its applicability scope, historically valid but no longer current, remembered but uncertain, or recorded in history without being current belief. These distinctions matter because a decision can fail even when the underlying data looks plausible.

The practical rule is simple: information becomes decision material through governed pathways. Its source, authority, applicability, permitted use, disposition, time, provenance, and relationship to state must remain visible whenever those facts can change governance.

4.1 Why Information Needs Its Own Governance

A productive-system model can be structurally correct and still make a bad decision because its informational basis is wrong. The problem is not limited to false data. Information may be stale, superseded, applicable to a different actor or configuration, prohibited for the proposed use, or learned only after the decision being reconstructed.

The PV-PP framework therefore separates evidentiary quality from evidentiary governance. Confidence answers one kind of question; authority answers another. Relevance is not permission. Recency is not proof of applicability. A historical record is not automatically current state.

Running example: coffee vending. Suppose the machine database reports twelve cups remaining. The record may be genuine and high-confidence, yet physically stale because the last twelve cups were removed during maintenance without an inventory update. The decision process may reasonably form $P_i(t)$ from the record, but that does not make the represented cup count identical to actual state $S_i(t)$.

4.2 Keep the Informational Objects Separate

Several concepts that ordinary language blends together occupy different roles in the PV-PP framework. The application should preserve those distinctions whenever collapsing them could change a decision or a later reconstruction.

Object	Question it answers
Observation or retrieved information	What item was sensed, received, or retrieved?
Memory	What retained or reconstructed material is available to the actor or decision process?
Expectation / model state	What does the actor infer, predict, or generalize beyond directly established facts?
Confidence / uncertainty	How uncertain is the representation or inference?
Source authority	Who or what is qualified to establish this fact?

Applicability	To which actor, state, domain, interval, configuration, or decision does it apply?
Permitted use	On which governance surfaces may it be used?
Disposition	Is it current, rejected, superseded, defective, prohibited, historically valid but noncurrent, unresolved, contradicted, or otherwise governed?
Transfer History	What relevant events, transfers, commitments, failures, or signals occurred?
Replay evidence	What decision-time material was retained so a prior governed episode can be reconstructed?

None of these distinctions says that every application needs a large information bureaucracy. A simple model may use only a few. The requirement is proportional: preserve the distinctions that are decision-material for the declared scenario.

4.3 From Actual State to Perceived Decision State

Chapter 3 introduced $P_i(t)$ as the decision-time representation formed from informational pathways rather than as an arbitrary second reality. Chapter 4 makes that construction more explicit.

A useful conceptual flow is:

authoritative or observed world facts → observation / retrieval / memory / inference →
information governance → admitted decision-time representation $P_i(t)$

This does not mean every element of $P_i(t)$ must come from a single database or evidence package. A person may combine direct observation, memory, testimony, expectation, and uncertainty. A software application may combine sensor readings, records, service responses, and model output. What matters is that the application knows which inputs were admitted and under what governance conditions.

Coffee example. The machine may combine a cup-count record, a door sensor, a payment-service response, and a maintenance flag. If the cup-count record says twelve but the maintenance flag says the cup cassette was removed after that count was recorded, the application has a temporal and applicability conflict to govern rather than a reason to average the two records into a confidence score.

4.4 Quality and Confidence Do Not Create Authority

A central V2.1 rule is that information can look excellent and still be unusable for the proposed decision. Source authority asks whether the source is qualified to establish the fact. Confidence asks how strongly the information or inference is supported. Those are not interchangeable.

A weather forecast may be high quality but have no authority to establish whether a bank transfer was completed. A buyer may be completely confident that a vending machine charged a card, while the authoritative payment service reports that authorization failed. A language model

may produce a highly plausible statement about a current account balance without being an authorized source of that balance.

The modeling consequence is important: do not convert confidence, plausibility, relevance, or model certainty into source authority unless the domain independently defines that relationship.

4.5 Applicability and Permitted Use

Even authoritative information can be wrong for the present use. Applicability identifies the actor, state, domain, interval, configuration, or decision to which an item applies. Permitted use identifies the governance surfaces on which the item may be used.

For example, a maintenance certificate may validly establish that a vending machine passed inspection yesterday, but it may not establish that the dispenser is operable after a failure this morning. A personnel record may be authoritative for employment status while being prohibited from use in an unrelated decision. A credential may establish identity without establishing permission to execute a particular action.

This separation prevents a common shortcut: “the information is real, therefore we can use it here.” The PV-PP framework requires the second half of that sentence to be justified.

4.6 Disposition, Supersession, and Contradiction

Information should not disappear merely because it stops governing the present decision. V2.1 uses disposition to preserve the status of an item: current, rejected, superseded, defective, prohibited for use, historically valid but noncurrent, unresolved, contradicted, or another explicitly governed state.

Supersession is especially important. A later record can replace an earlier record for current governance without erasing the earlier record from history. The old item may remain essential to explaining why an earlier decision was reasonable.

Coffee example. At 9:00 a.m., inventory record R1 reports twelve cups and is current. At 9:15, a maintenance event establishes that the cassette was emptied. R1 can become superseded or noncurrent for present dispensing decisions while remaining historically valid as the record that existed at 9:00. Overwriting R1 with zero would destroy the historical distinction between what was known then and what is known now.

4.7 Temporal Provenance: When Was It True, and When Was It Known?

Where time can change governance, the PV-PP framework distinguishes valid time from record or knowledge time. Valid time concerns when a fact holds in the world or modeled domain. Record/knowledge time concerns when the system recorded, received, or learned it.

Those times can differ. A payment reversal may be effective at 10:03 but not arrive in the application until 10:05. A medical test may describe a condition at collection time but become known only after laboratory processing. A maintenance failure may occur before the monitoring system reports it.

This distinction is one reason later knowledge must not be projected backward into an earlier decision basis. The fact that we now know an actuator had failed does not mean the decision process knew that when it authorized the earlier dispense attempt.

4.8 Memory Is Not the Same as History

Memory is decision-relevant retained or reconstructed material available to an actor or process. Transfer History is an event-history substrate. The two can interact, but they are not the same object.

A history event can support later memory, expectation, confidence, trust, or capability change. But appending an event to Transfer History does not automatically change any of those things. Likewise, a person or agent can remember something inaccurately even though the event record is accurate, or can fail to remember an event that remains preserved in history.

Human example. A driver experiences three breakdowns in one year. Transfer History can preserve the three events. Memory may retain all three, only the most recent, or a distorted impression of frequency. Expectation may generalize from them into "this car will fail again." The events, remembered material, and expectation are related, but a model that collapses them loses the ability to locate where the representation changed.

4.9 Transfer History: Record Events Without Turning Them Into Current Truth

Transfer History is useful when the sequence of interactions matters: transfers, attempted transfers, failures, signals, commitments, recoveries, or revealed effects can be recorded with identity and provenance. It remains separate from current state, current PP, current belief, expectation, and execution authority.

If a historical event creates a presently material consequence, that consequence must enter current state through the appropriate state-transition path. The event itself is not a substitute. A completed payment may create a current obligation to deliver or refund; that obligation belongs in current state if it remains material. The historical payment event can still remain in Transfer History.

This gives the model two useful views at once: what happened, and what remains true because it happened.

4.10 Historical Residue and Transition-Relevant State Sufficiency

The distinction between history and current state leads to a prospective sufficiency test. For the declared modeling purpose and admissible context, current state must contain or reference every presently material consequence capable of changing transition structure, reachability, policy construction, execution, or governance.

Complete causal history does not have to remain live. What must remain is the residue that still matters now. If two systems are declared to have the same complete current state and admissible context but their modeled futures differ only because an omitted historical consequence is still active, the declared state basis is incomplete.

Coffee example. The fact that a payment was captured five minutes ago need not remain a special "past event" inside live state. But if that capture creates an unresolved delivery-or-refund obligation, the obligation is a current consequence and must remain represented until resolved.

4.11 Replay Sufficiency: Reconstruct the Decision That Was Actually Made

Replay sufficiency answers a different question from prospective state sufficiency. If an application claims it can reconstruct a prior governed decision, it must retain enough decision-

time material to reproduce the basis on which that decision was made without importing later knowledge.

Depending on the claimed reconstruction, the replay package may need the state basis; $P_i(t)$; applicable $O_i(t)$ identity and version; admitted retrieval and information-governance material; rules and configuration; Graph, candidate, completeness, constraint, framing, adequacy, and selection traces as applicable; execution artifacts; and Layer-1 or environmental evidence.

Execution-complete and replay-sufficient are separate properties. A transaction can complete successfully while leaving an inadequate audit record. Conversely, a failed transaction can be replay-sufficient if its decision-time basis and failure evidence were preserved.

The temporal-integrity rule is strict: later evidence may explain why the earlier decision was wrong, but it must not be inserted into the earlier $P_i(t)$ or evidence basis as though it had been available then.

4.12 Prospective Sufficiency and Replay Sufficiency Side by Side

Question	Prospective state sufficiency	Replay sufficiency
Purpose	Support correct governance now and forward	Support reconstruction of a prior governed episode
Primary object	Current transition-relevant state basis	Retained decision-time evidence package
History requirement	Only presently material historical residue must remain live or referenced	Enough historical material must be retained for the reconstruction actually claimed
Later knowledge	May update current state through governed paths	May not be projected backward into the historical decision basis
Failure example	An unresolved obligation was omitted from current state	The prior selector result is retained but the evidence and rules that produced it are missing

Current capability and current state concern what exists now; replay concerns what can be established about the basis of a past decision. Neither should be used as a substitute for the other.

4.13 Information Can Change State, Capability, and Reachability—But Not by Declaration

Information often causes downstream change, but the change must occur through the surface that owns it. Learning that a resource exists may update $P_i(t)$. Verified discovery of a new route may cause Graph or reachability refresh. Training evidence may support a capability transition after the relevant realized change. A newly received instruction may create an objective or obligation if the application recognizes it through the proper authority path.

The information item itself does not automatically become current PP, current reachability, authority, or environmental truth. This is the informational form of the no-downstream-repair and anti-laundering rules: evidence supports governed transitions; it does not skip them.

Agent example. A scanner reports a newly reachable network route. The observation can be admitted as evidence and trigger representation/reachability re-governance. It does not itself authorize the agent to use the route. Reachability and authority remain separate.

4.14 Information Change, Staleness, and Re-entry

When an information item on which downstream governance depends becomes stale, superseded, contradicted, or otherwise invalid, the application should invalidate the artifacts that actually depend on it and re-enter at the earliest affected stage. It should not simply patch the final score or selector output.

In a synchronous implementation, this can occur when new information is presented on a later governance entry. The V2.1 framework can also represent asynchronous observation and invalidation while an external system supervises execution; the runtime's decision core does not provide that supervision itself (Chapter 7), and runtime v2.1.167 provides it only through a supervision layer that the host drives (Chapter 12). That is an implementation boundary, not a reason to collapse information governance.

4.15 Build an Information and Evidence Definition Packet

Before implementation, extend the Chapter 3 scenario packet with an explicit information/evidence section. The goal is not paperwork. It is to make the decision basis inspectable before code turns assumptions into hidden behavior.

Packet element	What to record
Information item / class	What fact, signal, report, retrieval, memory, or inference is represented?
Source and provenance	Where did it come from, and what identity/version is preserved?
Quality / confidence	What uncertainty or evidentiary-quality measure applies?
Source authority	What is the source qualified to establish?
Applicability	Actor, state, domain, interval, configuration, or decision scope
Permitted use	Which governance surfaces may consume it?
Disposition	Current, rejected, superseded, defective, prohibited, historical, unresolved, contradicted, etc.
Temporal provenance	Valid time and record/knowledge time where material
Dependencies	Which state, Graph, policy, authority, adequacy, or execution artifacts depend on it?

Invalidation / re-entry	What change makes the item or dependent artifacts stale, and where does governance resume?
Transfer History	Which events must be recorded separately from current state or memory?
Replay obligation	What decision-time material must be retained for the historical claim being made?

Appendix A.4 contains an AI interview prompt that builds this packet from your Chapter 3 packet, item by item, and flags the information-governance risks listed in Section 4.18.

4.16 Complete Walk-Through: The Coffee Vending Information Model

Return to the Chapter 3 coffee-vending model. The machine begins with an inventory record R1 stating that twelve cups are available. R1 has a source, version, record time, applicability to this machine and cassette, and a current disposition. The decision process uses R1 as one input to $P_i(t)$.

At 9:15 a maintenance worker removes the cassette but the inventory database is not updated. Transfer History can record the maintenance event if the application captures it. The actual state has changed: cup availability is now zero. Yet $P_i(t)$ may still represent twelve because the decision process has not received admissible evidence of the change.

A buyer then initiates a purchase. Payment authorization succeeds. The machine attempts to dispense and receives completion evidence that no cup dropped. That new observation can update the represented state, invalidate the assumption that dispensing PP is currently available, and activate the recovery path. If payment was captured, the resulting refund or delivery obligation becomes current state; the earlier payment event remains historical evidence.

Later, the maintenance event is reconciled into the information system. It may explain why the dispense failed and supersede R1 for current inventory governance. It must not be inserted retroactively into the earlier decision basis. A replay of the failed purchase should show that R1 was the admitted inventory record at decision time, that the maintenance fact was learned later, and that the failed-dispense evidence triggered the subsequent update and recovery governance.

The example shows why the PV-PP framework separates actual state, perceived state, information governance, Transfer History, current historical residue, and replay. Without those distinctions, a postmortem can easily rewrite the past into a decision process that never actually occurred.

4.17 Second Worked Example: Human Experience, Memory, and Expectation

Consider the household from Chapter 3 deciding what to do about a failing car. Three breakdowns have occurred over the previous year. Those events can be retained in Transfer History or another authoritative service record. The household members may remember them differently. One remembers every breakdown vividly; another remembers only the most recent. A mechanic provides a new diagnosis. An online forum offers anecdotal reports about the same model year.

The mechanic may be an authoritative source for the diagnosed condition of this particular vehicle within the scope of the inspection. The forum may be useful background but not

authoritative evidence that this car has the same defect. Memory may influence $P_i(t)$ and expectations, but confidence in a frightening breakdown memory does not make the remembered frequency exact.

Suppose the mechanic later discovers that the initial diagnosis was incomplete. The corrected diagnosis can supersede the earlier report for the current decision while the original report remains part of the historical basis for choices made before the correction. If the household had already authorized an expensive replacement on the basis of the first diagnosis, a faithful replay must preserve what was known and admitted then, not rebuild the decision from the corrected diagnosis.

The same architecture therefore applies to machine records and human cognition: distinguish event history, retained memory, current representation, expectation, source authority, uncertainty, and later correction. The domain mechanisms differ, but the information-governance questions remain recognizable.

4.18 Common Information-Governance Errors

- Treating confidence as authority.
- Treating relevance as applicability or permission.
- Treating the newest record as automatically authoritative.
- Overwriting superseded evidence and thereby destroying historical provenance.
- Treating Transfer History as current state, current belief, or current PP.
- Keeping a historical event but failing to carry its still-material consequence into current state.
- Using replay evidence as a substitute for missing prospective state.
- Reconstructing an earlier decision with information learned only later.
- Allowing an information item to create capability, reachability, or authority without the required governed transition.
- Invalidating everything after an evidence change rather than following explicit dependencies to the earliest affected stage.

4.19 From Information Governance to Change and Reachability

At this point the Guide has established the productive-system model and the informational basis from which that model is governed. The next question is how represented change itself is structured: what transformations exist, what configuration-dependent capability makes them possible, which paths are reachable, how new paths are admitted, and how capability and reachability change over time.

Chapter 3 defined what the system is. Chapter 4 defined how its decision-relevant information is governed. Chapter 5 turns to the structure of possible change.

Chapter 5 — Representing Change, Capability, and Reachability

Chapter 3 defined the productive system. Chapter 4 defined how that system knows what it thinks it knows. Chapter 5 asks what can change: which transitions are represented as possible, what makes them reachable now, how new routes and capabilities enter the model, and what happens to earlier governance when any of that shifts.

The central idea comes from the V2.1 Graph Layer specification: policies are not static objects. A policy is a bounded path through reachable instance space, and that path may preserve, degrade, or destroy the viability the model cares about. Before the framework can ask which path should be taken, it needs a disciplined representation of which paths exist. That representation is the Graph or seed substrate—what Chapter 2 called the represented productive envelope.

The Graph structures possibility. It does not judge adequacy, rank candidates, or select. Chapter 6 takes the represented paths through construction, constraints, framing, adequacy, and selection; this chapter makes sure there is something honest for those stages to work on.

Represent what could change before deciding what should.

5.1 Why Change Needs Its Own Representation

Without an explicit representation of possible change, candidate actions come from intuition. Intuition is reliably biased: it favors continuing what is already happening, overlooks exit and substitution, and forgets staged recoveries that take several steps. Early PV-PP traces showed exactly this pattern. When the same scenarios were rebuilt on an explicit transformation layer, the layer forced reachability assumptions into the open, generated policy families more systematically than intuition, and surfaced a missing restructuring family before any downstream evaluation ran.

The architectural reason is stricter than convenience. Policy construction Π builds candidates only from what the Graph represents; it may not improvise missing transformation classes. If a materially relevant option is never represented, no later stage can recover it. Constraints can only remove candidates, Adequacy can only test them, and Σ can only choose among them. An omission upstream therefore becomes an invisible error downstream: the selector confidently chooses the best of an incomplete set.

Coffee example. Suppose the model represents “dispense” and “refund,” but not “decline the sale before charging.” When inventory is doubtful, the safest action is to refuse the purchase before any money moves. If that transformation is absent, the decision cycle can only choose between dispensing into a probable failure and refunding after one. No amount of downstream sophistication creates the missing option.

5.2 Domains, Instances, and Transformations

The Graph Layer separates four levels, and most modeling errors in this chapter come from placing a rule on the wrong one.

- Domains are function-bearing viability structures: transportation viability, housing stability, financial solvency, delivery of a paid-for good. A domain says what must persist, what

threshold matters, how viability drifts without maintenance, how substitutable the function is, and whether damage is reversible. Domain rules describe viability physics, not action options.

- Instances are concrete realizations that support or burden domains now: a particular car, a lease, a savings balance, a cup cassette, a card terminal, a standing obligation. Instance rules describe what exists now—its condition, ownership, control, and support strength.
- Transformations are reachable transitions from one instance-state to another: car to cash by sale, captured payment to refunded payment, failing actuator to repaired actuator. Transformation rules describe what transitions are reachable and under what conditions.
- Policies are bounded paths through the transformation graph. Π builds them from seeds; they are the subject of Chapter 6.

The practical test is where a question belongs. “Can it be liquidated?” is not a domain rule and not merely an instance label. It is a transformation feasibility rule, and it belongs on the transformation that would carry out the liquidation.

Table 5.1 — Where rules belong

Rule type	Question it answers	Coffee vending	Household car
Domain rule	What must persist, and what counts as failure?	A paying buyer must receive coffee or be made whole.	Household members must be able to reach work reliably.
Instance rule	What exists now, in what condition?	Cup cassette holds twelve cups; actuator status unknown.	The car has a failing transmission.
Transformation rule	What transition is reachable, and under what conditions?	A refund can be issued if the payment rail is reachable and refund authority is current.	The car can be sold within the week if a buyer exists.

Burden instances matter as much as support instances. A fixed obligation, a costly maintenance commitment, or a captured payment that has not yet been matched by delivery shapes which transformations are needed. Leaving burdens out produces false adequacy later.

5.3 Start From Function, Not Mechanism

Graph construction begins by declaring, provisionally, what function or capability the graph must be able to preserve or recover. The declaration should be made at the functional level —“transportation viability of the household,” not “this car must keep running”—unless the identity of the mechanism is itself part of the problem.

The reason is substitution. If the preservation object is stated as a mechanism, every path that preserves the function through a different mechanism disappears before it can be scanned. A household locked onto “keep this car running” never represents selling it, sharing a ride, or taking transit. A vending model locked onto “this dispenser must succeed” never represents making the buyer whole through a refund or voucher.

This declaration is preliminary. It scopes graph construction; it does not settle what the decision will finally be judged against. That authority belongs to Domain Framing in Chapter 6, which may confirm, refine, or reject the preliminary scope.

5.4 Scan the Transformation Families

Transformations are discovered by systematic scan, not brainstorming. V2.1 defines eight canonical families, and every relevant instance is scanned across all of them. The current regime changes which families are most likely to be required—a stable operating posture emphasizes continuation and maintenance, while a survival posture makes exit, substitution, and escape routes essential—but the scan itself is never skipped.

Table 5.2 — The eight transformation families

Family	What it does	Coffee vending	Household car
Continuation	Keep operating as now.	Dispense normally.	Keep driving the car as is.
Maintenance / local adjustment	Adjust within the current mechanism.	Restock cups; recalibrate heater.	Drive conservatively; top up fluids.
Corrective / repair	Restore a degraded mechanism.	Reset or replace the actuator.	Repair the transmission.
Structural reconfiguration	Change how the function is organized.	Switch to authorize-then-capture payment.	Reorganize schedules around one shared car.
Exit / transfer / liquidation	Leave, convert, or hand off an instance.	Decline the sale before charging.	Sell the car for cash.
Substitution	Preserve the function through another mechanism.	Offer a different beverage; issue a voucher.	Take transit; borrow a relative's car.
Release / restructuring	Remove or reshape a burden.	Void an authorization; convert a charge to credit.	Refinance or restructure the car loan.
Escape / emergency	Act immediately to stop governing damage.	Lock out the machine after a fault.	Stop driving an unsafe car at once.

Two families receive explicit attention because they are the ones intuition drops. Exit must be scanned whenever an instance can be converted into a different viability-supporting form, whenever keeping it imposes a continuing burden, or whenever leaving would bypass critical damage. Substitution must be scanned whenever the function could continue through a different instance or recovery depends on a temporary bridge. Transformations are classified by their primary structural effect, each into exactly one family; there is no miscellaneous category.

5.5 Reachability Is Not Capability, Desirability, or Authorization

Every transformation that enters the Graph receives an explicit reachability assessment. Structural reachability asks whether the represented transition can actually be attempted from the current state: whether the mechanism is physically executable; whether the required instances, capability, access, control, operability, and counterparties are present or obtainable; whether it can be carried out within the relevant horizon; and whether its prerequisites hold. The

status is recorded as reachable, unreachable, or uncertain—the same three values the runtime accepts when a transformation is registered, admitted, or refreshed—and an uncertain status must carry a note saying why.

Legal, institutional, and other binding conditions are recorded on the transformation as enabling or blocking conditions, but the Graph does not decide them. Whether a policy that uses the transformation is permitted is a Constraints question, taken up in Chapter 6. The one exception is a condition that cannot be causally satisfied within the modeled horizon at all: a transformation whose required resource or access cannot exist in time is structurally unrealizable and does not enter the Graph, because admitting it produces imagined options. Difficulty is a different matter. A hard, expensive, or unpleasant transformation stays in the Graph with its costs recorded rather than being quietly excluded.

Reachability then has to be kept apart from three things it is easily mistaken for.

- **Capability.** Productive Power can make a transformation reachable, and losing PP can close one, but reachability is a structural fact about represented transitions, not a PP magnitude. A skilled driver without a working car has capability and no reachable driving transformation.
- **Desirability.** An objective can direct attention toward part of the transformation space, but it cannot make a transformation reachable. Wanting a refund path does not create one.
- **Authorization.** A reachable transformation may still be impermissible. The Graph records enabling and blocking conditions; Constraints later decides whether a constructed policy is feasible under the binding conditions of the decision; and execution authority is a further, separate question (Chapter 7). The Graph must not pre-filter a path merely because Constraints might reject it, and Constraints must not treat admission to the Graph as proof of feasibility.

Table 5.3 — Four different questions about the same action

Question	Owned by	Coffee example
Is the transition represented at all?	Graph construction	Is “refund via card rail” in the model?
Is it structurally reachable now?	Graph reachability	Is the payment network up and the refund mechanism operable?
Is a policy using it feasible and permitted?	Constraints (Chapter 6)	Do transaction rules permit a refund for this purchase?
Is this exact execution authorized now?	Execution licensing (Chapter 7)	Is there a current, single-use authorization to call the refund function?

5.6 Configuration, Access, and Compatibility

Chapter 3 separated possession, access, permission, control, and operability. In the Graph those distinctions become enabling and blocking conditions on specific transformations. The dispense transformation in the vending model is reachable only when a cup is present, inventory is available, power and temperature are adequate, the actuator is operable, and a control path exists. Record each as a condition. A model that records only “machine has coffee” will represent dispensing as reachable when it is not.

Real-world facts about reachability are supplied through a bounded configuration handoff. Configuration may state that a transformation is reachable, unreachable, or uncertain, and

whether its enabling instances are available, with bounded notes on timing, access, or prerequisites and the evidence for each. Permission facts—a rule, a license, a freeze, a credential—are recorded too, but as conditions for Constraints, not as reachability values; a reachable transformation may still be prohibited. Configuration must not encode which policy should be preferred, which loss matters more, whether a policy is adequate, or which option “feels best.” Those judgments belong to later stages, and slipping them into configuration is one of the quietest ways to smuggle a decision into a model.

Configuration-dependent capability appears here too. Some transformations become reachable only when several components are jointly present—agents, tools, permissions, counterparties, infrastructure. V2.1 treats this as configuration activation: the realized capability is represented as PP, while the enabling components keep their own roles. There is no separate “relational PP” object, and no assumption that joint capability is the sum of individual capabilities.

When a transformation involves another party, compatibility becomes a condition. Joint action requires aligned counterparties, terms, timing, roles, resources, authority, and communication. Another party's capability does not imply willingness. A car buyer who could afford the car has not agreed to buy it; a relative who owns a second car has not agreed to lend it.

5.7 Compose Multi-Step Paths

Many recoveries are not single steps. A sale converts a car to cash, but the household still needs transportation until a replacement arrives. A failed dispense after payment capture may need a refund, and if the payment network is down, a voucher now and a refund later. The Graph must therefore construct multi-step paths explicitly: ordered sequences of reachable transformations in which each step is reachable from the state the previous step leaves behind.

Composition is mandatory, not optional, whenever recovery, substitution, or exit requires staging. Depth must be bounded, but not truncated so far that the only viable path disappears. A bias toward short paths is a construction defect: it makes situations look more hopeless, or more dangerous, than they are. Family coverage is checked after composition, because a graph can look complete one step at a time and still be missing the staged path that matters.

5.8 From Paths to Seeds, and When the Graph Is Ready

The Graph is reduced to a bounded set of policy seeds before it is handed to Π . A seed represents a structurally distinct policy class. Reduction may merge parameter variants, near-duplicates, and cosmetic differences. It must not merge materially distinct classes. Five differences are never collapsed for simplicity: retaining versus selling, repairing versus substituting, direct versus staged recovery, continuing a burden versus releasing it, and preserving a mechanism versus preserving its function.

Before handoff, the Graph is checked as a construction artifact and receives one of three results: PASS, FAIL, or CONDITIONAL PASS. A conditional pass is allowed only when uncertainty is explicitly recorded and does not change which families are present or whether the only viable path exists. Failures are attributed to exactly one layer. If the Graph lacks required structure, the failure is the Graph's; if the Graph contains it and Π fails to represent it, the failure is Π 's. The V2.1 protocol names the Graph failure classes so that tests and traces can report them precisely.

Table 5.4 — Graph construction failure classes

Code	Failure	What it means in practice
G-001	Taxonomy violation	A transformation is unclassified or placed outside the eight families.
G-002	Coverage failure	A materially required transformation family is missing.
G-003	False structural reachability	An unrealizable transformation was admitted: an imagined option.
G-004	Unverified exclusion	A transformation was left out without a recorded reachability basis.
G-005	Missing multi-step path	A required staged recovery was never constructed.
G-006	Composition collapse	Depth truncation removed the only viable corridor.
G-007	Explosion	Equivalent variants were not reduced, overwhelming construction.
G-008	Coverage loss in reduction	Distinct policy classes were merged into one seed.
G-009	Graph- Π boundary violation	Π invented structure, or the Graph tried to construct policies.
G-010	Execution-order violation	Construction steps ran out of order, e.g. seeds before reachability.

5.9 Recovery Is a Reachability Question

Chapter 3 decomposed recovery into reserve, capability, mechanism, prerequisites, current reachability, and obligation. In the Graph those pieces have homes. The reserve is an instance or SPV holding; the recovery capability is PP; the mechanism is an instance and the transformation that uses it; the prerequisites are enabling conditions; current reachability is the transformation's status; and the obligation is the domain the recovery must restore. A reserve without a reachable recovery transformation is not a recovery path.

Some support is strictly necessary for recovering another domain—a working payment rail for refunds, or bridge transportation for keeping a job while a car is replaced. The runtime lets an application register that fact explicitly as recovery necessity, so it becomes represented structure rather than an assumption buried in code.

One boundary matters here. The Graph owns current structural reachability at decision time. The shared projection service, introduced in Chapter 6, projects how following a candidate policy would change future reachability—which recovery routes it would close or reopen. A projected closure does not rewrite the current Graph. Realized changes enter the Graph through the next state update.

5.10 Opportunity Closure, Reopening, and Delay

Opportunities close. A payment authorization expires; a repair shop's parts window lapses; a buyer's offer is withdrawn; skill or health degrades until a route can no longer be used. V2.1 defines opportunity closure as a change in the set of reachable viable futures: a trajectory that

was reachable earlier is no longer reachable later, with the policy, environment, and information basis held fixed except for the passage of time.

Closure is not a cost-of-delay number. It is not subtracted from a score, does not aggregate domains, and does not select a policy. It is a change in what is structurally possible, and it is represented in the pathways it actually affects. The architecture distinguishes projected closure—a current estimate that delay will remove paths, which may be wrong—from realized closure, which updates the Graph for the next decision cycle.

Closure is also not monotonic. A new counterparty, restored capability, additional reserves, or a changed rule can reopen an equivalent or substitute route. Waiting, deferring, and information-seeking are therefore ordinary candidate policies: sometimes waiting closes options, and sometimes it preserves or improves them. The model should say which, not assume that acting sooner is always safer.

5.11 New Opportunities Without Silently Expanding the Model

Dynamic systems discover things. A scanner finds a new network route; a vendor offers a mobile-refund interface; a relative mentions a spare car. The framework welcomes discovery but insists on one discipline: discovering something is not the same as admitting it to the governed model, and admitting it is not the same as authorizing its use.

An opportunity is first a representation—perceived or actual availability of an interaction, with its parties, possible flows, conditions, and open questions. It is not yet a policy. Objectives may direct where the model looks, and bounded search is legitimate, but those are search controls, not preferences. A missed opportunity should be classifiable as a detection, retrieval, or representation failure rather than blamed on selection.

The runtime makes the distinction concrete by treating three kinds of structural change differently.

Table 5.5 — Three kinds of change to represented structure

Kind of change	What it may change	What it may not do
Reachability refresh	The status of an already-represented transformation: reachable, unreachable, or uncertain, with source and provenance.	Introduce any new transformation, action, instance, family, or composition.
Structural-coverage evidence	Report that represented structure is sufficient, incomplete, or uncertain, naming missing families, path classes, or compositions.	Discover or admit the missing structure. Incompleteness invalidates prior Graph authority and blocks re-entry until the gap is defined and admitted.
Controlled admission or revision	Add a new transformation, or replace or remove an existing one, under an explicit configuration version, source, provenance, and rationale.	Happen implicitly. A new consequential function is admitted together with its first transformation, in one atomic step.

Each of these operations is bound to the identity of the Graph it expects to modify. If the Graph changed after the evidence was gathered, the operation is rejected rather than applied to a

structure it was not about. A rejected operation creates no governance work; a material change creates an explicit invalidation and a plan to re-enter at the Graph stage.

5.12 Capability Change: From Intention to Evidenced PP

Capability changes through learning, practice, maintenance, repair, damage, illness, reconfiguration, and acquisition of tools or help. The Productive Value–Productive Power (PV-PP) framework allows all of these, under one rule: a desired, planned, or predicted capability is not current PP until a realized state transition produces it and evidence shows it. The path is:

intention → action or investment → realized transition → evidenced capability → current PP

The framework does not permit the first term to stand in for the last. A household member enrolled in driving lessons does not yet have driving capability; a technician scheduled for certification cannot yet perform the certified repair; an agent that has requested a credential does not yet hold it. A capability-development policy is represented only when a currently reachable path can build, acquire, restore, or activate the capability.

In the runtime, capability change enters as explicit evidence. An observation names the entity, the capability, and the kind of change—gained, lost, increased, decreased, reconfigured, or uncertain—with its source and provenance. Definite changes must identify the capability snapshots before and after, so the claim can be audited. The runtime validates the evidence but deliberately does not compute a PP magnitude from it; capability remains non-scalar at this boundary.

Three further mechanisms connect capability to reachability without letting capability rewrite the model on its own. Ordered observations can form a capability trajectory, a structured history rather than a score. An application can declare that a specific, already-represented transformation's reachability depends on a capability snapshot, so that when the capability changes, that transformation's reachability refreshes through the ordinary path. And an application can record what capability it expected to see by a given time; when later evidence differs, that prediction error invalidates the governance artifacts that depended on the expectation.

Experience matters, but it is not PP by itself. Repeated interaction can build skill, confidence, and reliability, and those changes are represented in PP when they are realized. The same interaction may also update memory and expectation, which Chapter 4 keeps separate. Capability can also change during execution: an action entered with enough capability to begin may reveal hidden burdens or latent competence as it unfolds. Chapter 7 covers how execution may adapt within its licensed corridor and when it must return upstream instead.

Inheritance follows the same rule. Productive systems create other productive systems: an organization opens a branch, a fleet commissions a replacement unit, an agent starts a sub-agent or is copied into a new run. What passes from the one that creates to the one created is governed as follows.

- **Configuration and information can pass.** A design, settings, a prompt, code, tools, and a record of what worked can be inherited.
- **Permissions and standing state pass only by declaration.** Credentials, commitments, memory, expectations, and current context are reset unless the application declares a valid transfer explicitly, with its authority.

- **Current PP never passes.** The parent's current Productive Power is not copied into the child. The child's PP is realized after it is constructed and must be established from evidence about the child itself, by measurement, qualification, or observation, before anything that depends on it relies on it.

The rule is not bookkeeping. A child that receives a calculator tool and permission to use it has not thereby acquired arithmetic: the interface may fail, the model may choose the wrong tool, the permission may be withdrawn. Inherited configuration is a possible basis for capability, the first term of the path above, not its last. Controlled experiments with AI-agent populations made the distinction falsifiable: configuration was strongly inherited while measured capability was not, a result that would have been impossible to observe had inherited configuration been labeled PP (Amundsen, Before AI Agents Evolve in the Wild, Version 2.0, 2026, DOI 10.5281/zenodo.23175664).

In the runtime this is a host duty, not a runtime check. When application code creates an actor from another, it carries the chosen definitions into the new actor's registry and forms the new actor's perceived state from evidence about the new actor. It never copies the parent's ProductivePowerState values. The rule adds nothing to Framework V2.1; it states how an application must handle inheritance under the framework as it stands.

5.13 What Becomes Stale, and Where Governance Resumes

Every change in this chapter can invalidate earlier governance. The rule from Chapter 4 applies unchanged: invalidate what actually depends on the change, and re-enter at the earliest affected stage. Do not patch the final choice.

Table 5.6 — Change, invalidation, and re-entry

Change	What becomes stale	Where governance resumes
Realized reachability change of a represented transformation	Graph/seed substrate and everything built on it	Graph/seed stage
Structural coverage found incomplete or uncertain	Prior Graph authority	Blocked until the missing structure is defined and admitted; then the Graph stage
Transformation admitted, replaced, or removed	Graph/seed substrate and downstream	Graph/seed stage
Capability change bound to a transformation's reachability	That transformation's reachability and downstream	Graph/seed stage
Capability prediction error	Only artifacts bound to the failed expectation	The earliest stage among those artifacts
Realized opportunity closure or reopening	Next-cycle Graph	The next governance entry
Only a constraint fact changed	Constraint results and downstream	Constraints, if upstream artifacts remain valid

The runtime's decision core supports this machinery synchronously: changes are presented to it and processed between governance entries. What it can and cannot notice while an action is already executing is covered in Chapter 7, and what the supervision layer adds in Chapter 12.

5.14 Build a Change and Reachability Definition Packet

The output of this chapter extends the Chapter 3 and Chapter 4 packets with the structure of possible change. It records what could change, what makes each change reachable, how the model will learn of changes, and what those changes invalidate.

Table 5.7 — Change and Reachability Definition Packet

Packet element	What to record
Preservation object	The function or capability the graph must be able to preserve or recover; mechanism-independence note.
Domains	Threshold, maintenance need, drift, dependencies, substitutability, recovery structure, irreversibility.
Instances	Support and burden instances, condition, ownership and control, support strength.
Transformations	Family, source and destination, domains affected, enabling and blocking conditions, reachability status and basis, horizon, irreversibility, uncertainty note.
Compositions	Staged paths, with why each is structurally distinct.
Family coverage	Required, present, and missing families; reasons for any exclusion.
Seeds	Seed list, reduction basis, and what was merged.
Graph validity	PASS, FAIL, or CONDITIONAL PASS; failure classes; open uncertainty.
Recovery routes	Reserve, capability, mechanism, prerequisites, reachability, obligation; any registered recovery necessity.
Closure watch list	Windows, deadlines, and degradations that can close routes; possible reopenings.
Dynamic structure	Which reachability can refresh, where coverage is uncertain, and the admission path for new transformations or functions.
Capability change	Capabilities that can change, the evidence that would show it, trajectories, reachability dependencies, and expectations.
Invalidation map	For each kind of change, what becomes stale and where governance resumes.

Appendix A.5 contains an AI interview prompt that builds this packet from your Chapter 3 and Chapter 4 packets, following the construction order of this chapter and checking the result against Section 5.18.

5.15 Complete Walk-Through: The Coffee Vending Change Model

Return to the vending machine. The preliminary preservation object is stated as a function: a paying buyer receives the requested coffee or is made whole. It is not “the dispenser must succeed,” which would erase every path that satisfies the buyer another way.

The domains are delivery (the buyer's requested product or its equivalent) and transaction integrity (money moved only against delivery or a completed remedy). Support instances include coffee inventory, the cup cassette, the heater, the actuator, the control path, the card terminal, the payment network, the cash reserve, and the refund rail. Burden instances include an authorization hold and, once it occurs, a captured payment not yet matched by delivery.

The family scan produces continuation (dispense normally), maintenance (restock cups), repair (reset the actuator), reconfiguration (switch to authorize-then-capture), exit (decline the sale before charging), substitution (offer another beverage or a voucher), release (void the authorization or refund the charge), and emergency (lock out the machine after a fault). Each transformation carries its conditions. Dispensing requires a cup, inventory, power, temperature, an operable actuator, and a control path. A card refund requires a reachable network and a working rail, and it carries a refund-authority condition that Constraints will decide.

Composition makes the staged routes explicit: authorize, dispense, capture; authorize, fail to dispense, void; capture, fail to dispense, refund to card; and, when the network is down, capture, fail, issue a voucher, refund later. The last path is the one a short-path bias would drop, and it is the only way to make the buyer whole during an outage. The graph passes validation with one recorded uncertainty: actuator status after the last service is unknown.

At 9:15 the cassette is removed without an inventory update, as in Chapter 4. When the failed-dispense observation is admitted, the dispense transformation's cup condition fails and its reachability refreshes to unreachable. That invalidates the Graph/seed substrate and everything downstream, and governance re-enters at the Graph stage. The exit transformation—decline before charging—now carries the decision for the next buyer, while the refund path carries the obligation to the buyer already charged. The authorization hold is on the closure watch list: if it expires before it is voided, the release transformation changes form.

Later, the payment vendor offers a mobile-refund interface. The model does not simply start calling it. A new consequential function is admitted together with its first transformation, under a configuration version, source, and rationale; the Graph is re-validated; and execution authority for the new function is established separately. Meanwhile, the service company schedules its technician for actuator certification. That intention does not make certified repair reachable. The repair transformation's reachability depends on the certification capability, and it refreshes only when evidence shows the certification has been earned.

5.16 Second Worked Example: Household Transportation

For the household with the failing car, the preservation object is transportation viability, not the car. The transformations include continuing to drive, driving conservatively, repairing the transmission, selling the car, buying a replacement with cash or financing, taking transit, and borrowing a relative's car. The staged path that matters most is sell, hold cash, bridge with transit or a borrowed car, then buy. Without the bridge, selling looks impossible, because transportation would fail between sale and purchase.

Reachability conditions carry the real constraints. Repair is reachable only if parts arrive within the horizon; financing only if credit is approved; borrowing only if the relative agrees, which is a

compatibility condition, not a property of the relative's car. The parts back-order and the buyer's offer are on the closure watch list. A new used-car listing can reopen a replacement route that had closed.

Capability development follows the anti-laundering path. A second household member begins driving lessons so the household can share one car. Until that member holds a license—a realized, evidenced change—the shared-driving configuration is not reachable. An objective to own a new SUV does not make financing reachable either; it only directs attention toward financing routes, whose reachability still depends on credit.

5.17 Third Example: An Autonomous Agent

For an autonomous agent, the same distinctions become governance controls. Suppose a scanner reports a newly reachable network route. If the route corresponds to an already-represented transformation, the observation can refresh that transformation's reachability, with source and provenance, and trigger re-entry. If it is a genuinely new path, the model does not grow on its own: coverage evidence can report the gap, and the new transformation must be defined and admitted explicitly. If the agent acquires a new tool, that is a new consequential function, admitted atomically with its first transformation.

In every case, reachability is not permission. A reachable route still faces Constraints, and a permitted policy still needs current execution authority. This is how the PV-PP framework limits capability accumulation from above. Within a PV-PP-governed execution path, discovered reach does not itself become authorized action: the route must pass through the applicable governance stages and obtain current execution authority. Keeping the agent on that path, so that it cannot simply use the route directly, is an integration and enforcement responsibility of the surrounding system, not something the framework guarantees on its own.

5.18 Common Errors in Representing Change

- Generating candidate actions from intuition rather than a family scan, and so omitting exit, substitution, or staged recovery.
- Stating the preservation object as a mechanism, which removes substitution before it can be scanned.
- Placing a transformation rule on a domain or an instance, such as recording "can be sold" as a property of the car.
- Treating reachability as capability, as desirability, or as authorization.
- Excluding a transformation because it is difficult, rather than because it is unreachable.
- Admitting imagined options: transformations whose required conditions cannot be met within the horizon.
- Encoding preferences, adequacy, or "which loss matters more" in configuration.
- Truncating paths so short that the only viable staged recovery disappears.
- Merging structurally distinct seeds, such as repair and substitute, during reduction.
- Letting Π , or the application, invent transformations the Graph does not contain.
- Treating projected closure as realized, or treating closure as a scalar cost.
- Letting a discovered route or tool enter the model without controlled admission.
- Counting desired or predicted capability as current PP.
- Recomputing everything after a change instead of re-entering at the earliest affected stage.

5.19 From Represented Change to the Decision Cycle

The productive system is now modeled (Chapter 3), its information is governed (Chapter 4), and its possible changes are represented as a validated graph of reachable transformations, paths, and seeds (Chapter 5). What remains is to decide.

Chapter 6 walks through the canonical decision cycle that consumes this substrate: the upstream stages that establish pressure, horizons, governing domains, and regime; policy construction and its completeness gate; Constraints; Domain Framing; Adequacy; selection; and the transition toward execution. It is also where the Guide's code begins.

Chapter 6 — The Canonical Decision Cycle

Chapters 3 through 5 produced a model: a bounded productive system, a governed information basis, and a validated representation of what can change. This chapter takes that model through the canonical V2.1 decision cycle—the ordered sequence of stages that turns a represented situation into a selected policy ready for execution.

The operator sequence is easy to memorize and easy to misread. Read as a string of symbols, it looks like a pipeline that computes a score. It is better read as a series of distinct questions, each owned by one stage, each with its own kind of failure, and each forbidden from repairing a defect that belongs to another stage. That discipline is what keeps a missing option from being mistaken for a bad choice, or an infeasible action from being mistaken for an inadequate one.

This is also where the Guide's code begins. Section 6.15 builds the coffee-vending model as a small program against the frozen runtime and evaluates one decision cycle with execution not requested, so every stage's result can be inspected without changing anything in the host's world.

Each stage answers one question. No stage repairs another stage's defect.

6.1 The Cycle at a Glance

The canonical V2.1 sequence begins with the perceived decision state and its adjacent objectives and ends with the transition toward execution:

**$(P_i(t), O_i(t)) \rightarrow \Phi \rightarrow H \rightarrow G \rightarrow \mathcal{R} \rightarrow \text{Graph / seed substrate} \rightarrow \Pi \rightarrow \Pi \text{ completeness}$
 $\rightarrow \text{Constraints} \rightarrow \text{Domain Framing} \rightarrow \text{Adequacy} \rightarrow \Sigma \rightarrow \epsilon$**

The order is a dependency order. Each stage consumes what earlier stages produced and adds one kind of structure. Table 6.1 summarizes the question each stage answers, what it produces, and what it must not do. The owning V2.1 specifications control the exact semantics; the sections that follow explain each stage in turn.

Table 6.1 — The canonical decision stages

Stage	Question it answers	Produces	Must not
Φ Pressure Field	How strongly is each domain demanding correction now?	Domain-local pressure	Rank domains or become a caring weight
H Collapse Horizon	Under baseline continuation, how long until each domain fails?	Baseline horizons	Evaluate policies or set priority
G Governing Domains	Which domains must be preserved for recoverable viability?	Governing set	Shrink because a domain is being neglected
$\mathcal{R}$ Regime	What behavioral posture does the situation call for?	Mission, Stabilization, Survival, or Existential	Judge feasibility, adequacy, or selection

Stage	Question it answers	Produces	Must not
Graph / seed	Which transitions and paths are represented and reachable?	Validated seeds	Construct, rank, or judge policies
Π Policy Construction	Which bounded candidate policies can be built?	Candidate set	Invent missing transformation families
Π completeness	Are all materially required policy classes present?	Pass, fail, or qualified pass	Be satisfied by downstream stages
Constraints	Which candidates are executable under binding conditions?	Feasible set	Hide prohibitions as score penalties
Domain Framing	What preservation target are candidates judged against?	Validated frame	Recompute governing domains
Adequacy	Does the policy open a real recovery corridor for every governing domain?	Adequate set	Rank, or let one domain compensate for another
Σ Selection	Which adequate policy is selected?	Selected policy	Reduce the structure to one score
ϵ Execution	How does the selected policy move toward attempted execution?	Execution episode and status	Reselect, or treat a returned call as a realized effect

6.2 The Decision Interface: $P_i(t)$ and $O_i(t)$

The cycle starts from the perceived decision state $P_i(t)$, not from actual state. Every stage reasons from what the actor or application currently represents, which is why Chapters 3 and 4 insisted on knowing how that representation was formed. Each operator consumes only the components of $P_i(t)$ its owner specification authorizes; none receives an unrestricted view of memory or of actual state.

Objectives $O_i(t)$ sit beside $P_i(t)$ rather than inside the sequence. They may direct bounded discovery—where the model looks for candidates—and they may be described in relation to each candidate as supporting, conflicting, neutral, or unresolved. They never make a candidate feasible, adequate, or selected. In the runtime, objective responsiveness is evaluated after Π completeness as a descriptive record, not as a stage that can remove or rank anything.

Actual state stays with the host. In this chapter's example the host hands the runtime a represented state directly. Chapter 8 shows the integrated form, in which the host's adapter maps actual persistent state into the perceived decision state before the cycle begins.

6.3 Pressure: Φ

The Pressure Field operator builds a domain-by-domain picture of present corrective demand: how strongly each domain presses for protection, repair, or restoration. Pressure in a domain grows from a shrinking margin to its threshold, a worsening trajectory, shocks, persistence, and propagation from other domains.

Three invariants govern it. Pressure must not decrease as the margin to failure shrinks. It must not flatten or reverse near the threshold, where urgency matters most. And it stays local to each domain: Φ never aggregates domains into a single welfare score.

One distinction is easy to miss. Pressure is not governing force. A domain can be registered as under severe strain while later behavior still fails to protect it—the pattern V2.1 calls domain devaluation. Φ may represent the pressure correctly even when the actor does not honor it. In the runtime, the host supplies the domain-specific pressure factors and function; the runtime enforces the operator's order and boundaries.

6.4 Collapse Horizons: H

The Collapse Horizon operator estimates, for each domain, how long it would take to fail under baseline continuation—if nothing new were done. Its output is a set of baseline horizons. A larger margin must never yield a shorter horizon under the same deterioration. Horizons express time to failure, not importance or priority.

Baseline continuation needs a representation, which is why the runtime requires every registry to contain an action named "steady." The horizons H computes are baseline-only. The policy-conditioned horizons that later stages compare—how long each domain would last under each candidate—come from the shared projection service in Section 6.11, and the two must never be substituted for each other.

6.5 Governing Domains: G

G identifies the domains that must be preserved if recoverable viability is to remain possible. It works in two steps. The domains nearest collapse form a seed—in the runtime, every domain whose baseline horizon lies within a configured tolerance of the shortest one. Then dependency closure adds any domain whose loss would eliminate every recovery path for a domain already governing.

The governing set is never empty, always includes the domain nearest collapse, and is closed under recovery necessity. It is structural, not a record of current concern: neglecting a domain does not remove it from governance. In the coffee example both delivery and transaction integrity govern, because their baseline horizons fall within the configured tolerance of each other.

6.6 Regime: $\mathcal{R}$

Regime classification assigns one of four postures—Mission, Stabilization, Survival, or Existential—from the governing domains' shortest horizon, their highest pressure, how many domains are governing, and any explicit contextual interruption. Conditions are checked from most to least severe, and the first satisfied condition governs. Hysteresis rules keep the classification stable across adjacent decisions: a sharp horizon drop must hold or escalate the regime, de-escalation requires clear improvement, and no hysteresis rule may suppress a valid Existential assignment.

Regime is posture, not a score. It shapes how broadly Π searches, which policy families are required, and how far conditional constraints may relax under necessity. It does not decide feasibility, adequacy, or selection.

6.7 From Graph to Candidates: Π and Π Completeness

The Graph and seed substrate from Chapter 5 arrives here. Policy construction Π builds a bounded, regime-shaped candidate set from the validated seeds. Mission favors continuation and local adjustment; Stabilization adds selected structural alternatives; Survival materially expands structural and escape families; Existential expands across all families within bounded feasibility. Π may combine seeds, but it may not invent a transformation family the Graph does not contain.

Π completeness is a hard gate. Π need not be exhaustive, but it must include every feasible policy class that would represent a materially different trajectory for the governing domains under the current regime—including exit, liquidation, transfer, and substitution classes where they matter. The test is relative to the substrate Π received: it can detect that Π dropped something the Graph contained, but it cannot restore a class the Graph never built. When the substrate itself is uncertain or contaminated, the result is qualified rather than a clean pass.

This is the first place the chapter's principle becomes concrete. In case D of Section 6.16, the exit family was never represented. The cycle stops at the Graph with failure code G-002 and never reaches selection. A missing candidate family is not a selection problem, and the fix belongs upstream.

6.8 Constraints

Constraints is the authoritative decision-stage feasibility operator. It sorts restrictions into three classes. Hard constraints express impossibility or absolute prohibition, and any policy that violates one is removed. Conditional constraints are normally binding but may relax under an elevated regime—and then only when retaining them would block every executable candidate needed to preserve the governing domains and no compliant candidate remains. Soft constraints—comfort, convenience, minor social expectations—never remove a policy.

Two boundaries protect this stage. The Graph's reachability metadata is not a feasibility verdict, so a policy built on a reachable transformation may still be infeasible here. And no objective, however important, can make a blocked transformation permitted or relax a hard constraint. In the runtime, if Constraints leaves no executable candidate at all, the cycle enters a terminal selection mode that chooses among the infeasible candidates by severity—hard before conditional before soft—rather than pretending an ordinary choice exists.

6.9 Domain Framing

Domain Framing states what the surviving candidates will be judged against: the preservation target for each governing domain. It confirms or refines the preliminary preservation object declared during graph construction in Chapter 5.

The central rule is to frame functions rather than mechanisms, but only as far as real substitution allows. A domain framed as "this car provides transport" will reject every substitute; one framed as "the household retains transportation above threshold" admits them. Expansion must be minimal but sufficient, and a substitute counts only if it preserves the function above threshold within a reachable, sustainable corridor. An emergency bridge is not durable continuity. In the runtime, the frame is validated against the governing set already computed: every governing domain must be framed, and Framing never recomputes G.

6.10 Recovery in the Cycle, and the Runtime's Joint-Recovery Check

In the framework, recovery is not a separate decision stage. It lives in Adequacy's recovery-corridor standard (Section 6.11) and in the recovery transformations the Graph represents.

The runtime adds one implementation mechanism that readers of the runtime documentation will meet as “Joint Recovery.” When recovery obligations are already active—corridors the system has committed to from earlier cycles—the runtime checks whether they can all still be carried out against a host-supplied calendar of shared capacity and deadlines. Mandatory current-period recovery actions are bound into every candidate before Constraints, and the joint-feasibility result is checked after Domain Framing and before Adequacy. The runtime describes this as a structural precondition for Adequacy, not a ranking rule, and it uses no queue order or sacrifice rule to resolve conflicts. When no recovery obligations are active, the check does not appear in the trace at all, as in this chapter's example.

The Guide therefore presents the canonical stack as the owner specifications define it, and treats joint recovery as an implemented runtime stage that serves Adequacy rather than as an additional framework operator.

6.11 Adequacy and the Shared Projection

Adequacy asks whether a policy opens a real recovery corridor. A policy is adequate only if, for every governing domain, it preserves viability until recovery becomes reachable, enters a recovery-capable region, and supports joint sustainment across the governing domains.

Adequacy is conjunctive: no governing domain may be traded for another, and no gain in a non-governing domain can compensate for inadequacy in a governing one. It is a filter, not a ranking.

If no candidate is adequate, recovery is unavailable under the current candidate structure. That is not the same as action being impossible—executable behavior may still exist—and the architecture routes the decision to a separately governed fallback structure rather than treating the best inadequate candidate as an ordinary choice.

Adequacy and Σ both need to know what each candidate is projected to do. That comes from the shared policy-conditional projection service, $Q_t(\pi)$: for each candidate, projected domain trajectories, projected future reachability including closure and reopening, recovery-corridor facts, and each domain's projected time to failure under that policy. Projections that run past the licensed horizon are marked as right-censored rather than given invented exact values. The projection service is not a decision stage. It generates, filters, and ranks nothing; Adequacy and Σ consume the same authoritative records, and neither recomputes them. In the runtime, the host supplies these records through a projection service attached to the runtime, separate from its adapter. The runtime licenses each request—a finite horizon, the decision-time state, the candidate—and validates each record against it. An older route, a projection method on the adapter itself, still works but receives none of those checks; new programs should not use it.

6.12 Selection: Σ

Σ selects from the adequate set by three stages, none of which aggregates domains into a score. Each compares policy-conditioned projected horizons with a censor-aware relation that treats right-censored horizons honestly.

- Stage 1, governing-domain non-domination. Retain every policy that no other policy beats on all governing domains at once. Only governing domains count here.

- Stage 2, global dominance refinement. Among the Stage 1 survivors, drop any policy that another survivor strictly beats across all active domains. A policy is never dropped merely because another improves a non-governing domain.
- Stage 3, deterministic tie resolution. If several policies remain, choose by a fixed, state-independent order index. This order breaks ties; it must never act as a hidden preference.

Σ consumes an already-licensed set. It does not repair omitted candidates, relax constraints, redo adequacy, or recompute projections. When the ordinary admissible set is empty, Σ does not pretend a choice exists; the separately governed fallback structure applies. The two-stage structure of Stages 1 and 2 is also the object of the scalar-reduction proofs discussed in Chapter 11: where governing domains are in genuine conflict, no single score can reproduce both stages faithfully.

6.13 Toward Execution: ϵ

ϵ realizes the selected policy. It instantiates an execution episode, monitors it, allows bounded adaptation within the licensed corridor—pacing, assistance, staging—and ends with an auditable status such as completed, partial realization, aborted and returned upstream, failed, or emergency return. It distinguishes entry sufficiency (enough to begin), continuation sufficiency (still licensed after execution reveals more), and completion sufficiency. It must not reselect, invent a rival policy class, invoke fallback on its own, or repair framing or adequacy locally.

In the runtime, a decision cycle evaluated without an execution request stops after Σ . An execution license can be built only from a completed cycle, and every step from license to realized effect is the subject of Chapter 7.

6.14 Ordering, No Downstream Repair, and Re-entry

The stage order gives every failure a home. When something goes wrong, the question is which stage owns it, and the fix belongs there. Table 6.2 lists common symptoms, the stage that owns each, and the tempting downstream repair that the architecture forbids.

Table 6.2 — Where failures belong

Symptom	Owning stage	Forbidden downstream repair
A sensible option never appears among candidates	Graph or Π (Π completeness)	Adjusting the Σ order or a score to favor what remains
A candidate is executable but leaves no recovery path	Adequacy	Treating it as a constraint violation
A candidate violates a binding rule	Constraints	Keeping it and hoping Adequacy rejects it
The decision was based on stale or unauthorized evidence	Perception and information governance	Changing a weight or score to compensate
The preservation target was mechanism-locked	Domain Framing	Relaxing Adequacy
A domain under strain is being ignored	G (structural governance)	Removing the domain from the governing set
Execution revealed the policy cannot finish	ϵ , returning upstream	Letting execution quietly switch to another policy class

When something changes after a decision—new evidence, a refreshed reachability, a capability change, a revised objective—earlier results may become stale. The rule from Chapters 4 and 5 applies: invalidate the artifacts that actually depend on the change and re-enter at the earliest affected stage. The runtime supports this directly. It identifies the earliest invalidated stage, preserves only upstream artifacts that remain valid, and resumes through stage-specific executors covering every step from fresh perception through Σ . A change that touches only an objective invalidates only what depends on that objective; it does not pretend that state or perception changed.

6.15 Your First Program: Evaluating a Decision Cycle

The coffee-vending model now becomes code. The program is deliberately small: one purchase episode, two domains, and two one-step transformations. Its purpose is to show where each part of the model lives and how to read what the runtime returns. The complete file, `coffee_decision.py`, accompanies the Guide together with a test file; both run unchanged against the frozen runtime: written for build 141, they run on v2.1.167 because its decision core is build 141's, byte for byte.

To run it, point the environment variable `PVPP_RUNTIME_PATH` at the `runtime-v2.1.167` folder (the folder that contains `pvpp_runtime`) and run the file with Python 3.11 or later. The program has four parts: a registry that declares the governed structure, a host adapter that supplies domain semantics, a request that states what the cycle must preserve and cover, and a call that evaluates one cycle.

The registry declares what exists for governance: domains with thresholds, actions (including the required “steady” baseline), the governing tolerance and regime thresholds, a hard constraint, the Graph's instance and transformations, and a Σ order used only to break ties.

Listing 6.1— The registry

```
def build_registry(include_exit=True):
    reg = PVPPRegistry()
    reg.register_domain(DomainDefinition("D", "delivery: buyer receives coffee or is made whole", 5.0))
    reg.register_domain(DomainDefinition("T", "transaction integrity: money moves only against
        delivery", 5.0))

    # Actions. "steady" is required by the runtime: baseline continuation, used by H.
    reg.register_action(ActionDefinition("steady", "baseline: machine idles, buyer waits", ("D", "T")))
    reg.register_action(ActionDefinition("sell", "authorize, dispense, then capture payment", ("D",
        "T")))
    reg.register_action(ActionDefinition("decline", "refuse the sale before any charge", ("D", "T")))

    reg.register_governing_configuration(GoverningConfiguration(epsilon_h=10.0))
    reg.register_regime_configuration(RegimeConfiguration(
        tau_h_existential=1, tau_h_survival=2, tau_h_stabilization=4,
        tau_phi_existential=100, tau_phi_survival=50, tau_phi_stabilization=10))
    reg.register_constraint_rule(ConstraintRuleDefinition("card_network", "hard",
        description="a card sale requires a reachable payment network"))

    # Graph: one instance (the purchase episode) and one transformation per family.
    reg.register_graph_instance(GraphInstanceDefinition("purchase", "episode", ("D", "T"), "active"))
    reg.register_graph_transformation(GraphTransformationDefinition(
        "sell", "purchase", "purchase", "continuation", ("D", "T"), "reachable", "sell",
        ("continuation",)))
    if include_exit:
        reg.register_graph_transformation(GraphTransformationDefinition(
            "decline", "purchase", "purchase", "exit_transfer_liquidation", ("D", "T"), "reachable",
            "decline", ("exit",)))

    # Sigma order: deterministic tie resolution only (Stage 3), never a preference score.
    reg.register_sigma_order(SigmaOrderDefinition("graph:sell", 10))
    reg.register_sigma_order(SigmaOrderDefinition("graph:decline", 20))
```

```
return reg
```

The host adapter is where domain meaning lives. For this example the most instructive adapter method is the constraint profile, which reports whether each candidate violates the card-network rule. Projected consequences come from a separate projection service, `CoffeeProjection`, which tells the runtime what each policy is projected to do to each domain and whether it opens a recovery corridor. It echoes the request's state, horizon, and candidate, and marks horizons at the limit as right-censored, so the runtime can check each record. Notice what is absent: neither the adapter nor the service decides feasibility, adequacy, or selection. They report facts; the runtime applies the stages.

Listing 6.2 — Constraint facts from the host adapter, and the projection service

```
# Constraints: typed observations over the represented state.
def constraint_profile(self, state, policy_id, action_ids, regime, governing):
    network_down = not state.metadata.get("network_up", True)
    violated = "sell" in action_ids and network_down
    return PolicyConstraintProfile(policy_id, (ConstraintObservation("card_network", violated),))

def compare_constraint_violation_severity(self, a, pa, b, pb, cls):
    return 0

class CoffeeProjection:
    """Shared projection Q_t(pi), supplied as a PolicyProjectionService.

    It says what each policy is projected to do to each domain. It projects; it never
    filters, ranks, or selects. The runtime licenses each request and validates each record.
    """

    def project(self, req):
        state, action_ids = req.represented_state, req.action_ids
        cups = state.metadata.get("cups", 0)
        corridors, horizons = [], {}
        if "sell" in action_ids:
            ok = cups > 0
            # Delivery succeeds only if a cup is available; a failed dispense after
            # capture leaves the buyer charged and unserved.
            horizons = {"D": 30.0 if ok else 0.5, "T": 30.0 if ok else 0.5}
            for d, f in (("D", "delivery"), ("T", "transaction")):
                corridors.append(RecoveryCorridorProjection(d, f, ok, ok, ok, 1.0 if ok else None,
                                                            None if ok else 0.5))
        else: # decline: nobody is charged; the buyer is made whole but not served
            horizons = {"D": 12.0, "T": 30.0}
            for d, f in (("D", "delivery"), ("T", "transaction")):
                corridors.append(RecoveryCorridorProjection(d, f, True, True, True, 1.0))
        # Echo the request's identity and horizon; horizons at the limit are right-censored.
        censored = tuple(d for d, h in horizons.items() if h >= req.projection_horizon)
        return PolicyProjectionRecord(
            req.policy_id, True, state, horizons, tuple(corridors),
            projection_horizon=req.projection_horizon, right_censored_domain_ids=censored,
            state_id=req.state_id, state_time=state.time, candidate_mode=req.candidate_mode,
            model_version="coffee-q1", projection_input_trace=dict(req.projection_input_trace),
        )
```

The request states the preliminary preservation object, the domain frame, and the policy families and classes that must be present. The final call evaluates one cycle through Σ . Execution is not requested, so the host's state is not touched—the accompanying test confirms that the state after the cycle is identical to the state before it.

Listing 6.3 — The request and one decision cycle

```
def request(required_families=("continuation", "exit_transfer_liquidation"),
            required_classes=("continuation", "exit")):
    return CanonicalDecisionCycleRequest(
        preservation_object=PreliminaryPreservationObject(
            "buyer-whole", "a paying buyer receives coffee or is made whole"),
```

```

        domain_frame=DomainFrame((DomainFrameTarget("D", "delivery"), DomainFrameTarget("T",
"transaction"))),
        required_graph_family_ids=required_families,
        materially_required_policy_class_ids=required_classes,
        projection_horizon=30.0,
    )

def make_runtime(include_exit=True):
    return PVPPRuntime(build_registry(include_exit), CoffeeWorld(),
        projection_service=CoffeeProjection())

def decide(cups, network_up=True, include_exit=True):
    rt = make_runtime(include_exit)
    state = WorldState(0, {"D": 10.0, "T": 10.0}, {"cups": cups, "network_up": network_up})
    return rt.evaluate_canonical_decision_cycle(state, request())

```

Running the file evaluates four situations. The output is reproduced exactly:

Listing 6.4 — Program output

```

== A. cups available
trace:      PPP -> Phi -> H -> G -> R -> Graph/Seed -> Pi -> Pi Completeness -> Constraints ->
           Domain Framing -> Adequacy -> Sigma
status:      sigma_standard_policy_selected | stopped at: Sigma
governing:   ('D', 'T') | regime: Mission
graph:       PASS ('graph:sell', 'graph:decline')
feasible:    ('graph:sell', 'graph:decline') | infeasible: ()
adequate:    ('graph:sell', 'graph:decline') | inadequate: ()
selected:    graph:sell | Stage 1 frontier: ('graph:sell',)

== B. cups exhausted (evidence admitted)
trace:      PPP -> Phi -> H -> G -> R -> Graph/Seed -> Pi -> Pi Completeness -> Constraints ->
           Domain Framing -> Adequacy -> Sigma
status:      sigma_standard_policy_selected | stopped at: Sigma
governing:   ('D', 'T') | regime: Mission
graph:       PASS ('graph:sell', 'graph:decline')
feasible:    ('graph:sell', 'graph:decline') | infeasible: ()
adequate:    ('graph:decline',) | inadequate: ('graph:sell',)
selected:    graph:decline | Stage 1 frontier: ('graph:decline',)

== C. payment network down
trace:      PPP -> Phi -> H -> G -> R -> Graph/Seed -> Pi -> Pi Completeness -> Constraints ->
           Domain Framing -> Adequacy -> Sigma
status:      sigma_standard_policy_selected | stopped at: Sigma
governing:   ('D', 'T') | regime: Mission
graph:       PASS ('graph:sell', 'graph:decline')
feasible:    ('graph:decline',) | infeasible: ('graph:sell',)
adequate:    ('graph:decline',) | inadequate: ()
selected:    graph:decline | Stage 1 frontier: ('graph:decline',)

== D. exit family never represented
trace:      PPP -> Phi -> H -> G -> R -> Graph/Seed
status:      FAIL | stopped at: Graph/Seed
governing:   ('D', 'T') | regime: Mission
graph:       FAIL () ('G-002',)

```

Three simplifications are deliberate. The domain values are host-defined margins, not measurements the runtime understands; the program passes the represented state straight to the cycle as if already perceived, which skips the perception and objective-set checks a snapshot performs; and each policy is a single step. Chapter 8 replaces these with a fuller adapter, perceived-state formation through the validated snapshot route, and staged policies.

6.16 Reading the Four Cases

Each case stops or decides at a different stage, which is the point of running them together.

Table 6.3 — What decided each case

Case	Where it was decided	Why
A. Cups available	Σ Stage 1	Both policies are feasible and adequate; selling beats declining on both governing horizons, so it is the only policy on the governing frontier.
B. Cups exhausted	Adequacy	The sale is executable—Constraints passed it—but its projection opens no recovery corridor: a charged buyer would go unserved. Declining is the only adequate policy.
C. Network down	Constraints	The card sale violates the hard network rule and is removed before Adequacy runs. Declining remains.
D. Exit never represented	Graph (G-002)	The required exit family is missing, so the cycle stops before Π . No candidate is selected, and none should be.

Cases B and C reach the same selection for different reasons, and the difference matters. In B the sale could be attempted but would fail the buyer; in C it cannot be attempted at all. A trace that records only “declined” would lose that distinction, and with it the ability to fix the right thing: restock the machine in one case, restore the network in the other. Case B also completes the story from Chapter 4: once the failed-dispense evidence is admitted, the represented cup count changes, the projection changes, and Adequacy does the rest. No score was adjusted.

Case D is the most important of the four. A model without an exit option would otherwise let the machine sell into a probable failure and call it the best available choice. The runtime refuses to produce that choice at all, and it names the stage and failure class that need fixing.

6.17 The Same Cycle in Other Domains

The stages ask the same questions everywhere, although the causal mechanics behind each answer are domain-owned. For the household with the failing car, pressure and horizons express how long transportation and solvency would last; G may add bridge transportation through recovery necessity; Survival posture requires exit and substitution families; Constraints removes financing the household cannot obtain; Framing holds the target at transportation rather than this car; and Adequacy rejects a cheap repair that restores the car for a month but leaves no recovery if it fails again.

For an organization, governing domains might be liquidity and staffed capacity, and Adequacy may reject a profitable contract whose delivery would exhaust both. For an engineered system, horizons are operating margins, and Framing distinguishes the function—cooling, power, throughput—from the specific unit providing it. For an autonomous agent, Constraints carries authority and permission rules, Adequacy asks whether a successor configuration keeps recovery and control available, and Σ selects only among candidates that survived both.

6.18 Common Errors in the Decision Cycle

- Treating the operator sequence as a scoring pipeline rather than a series of distinct questions.

- Feeding actual state, or raw memory, directly into the cycle instead of the perceived decision state.
- Letting an objective remove, rank, or authorize a candidate.
- Confusing pressure with governing force, or dropping a neglected domain from the governing set.
- Substituting baseline horizons from H for policy-conditioned horizons from the projection service, or the reverse.
- Using regime as a utility or priority score.
- Letting Π invent a transformation the Graph does not contain, or accepting a Π completeness failure as a selection problem.
- Encoding a hard prohibition as a penalty, or letting an objective relax a constraint.
- Framing a mechanism instead of a function, or accepting a substitute that does not preserve the function above threshold.
- Treating Adequacy as ranking, or letting a non-governing gain offset a governing failure.
- Recomputing projections in Adequacy or Σ instead of consuming the shared projection records.
- Using the Σ order as a preference rather than a deterministic tie-breaker.
- Treating an empty adequate set as an ordinary choice instead of routing to fallback.
- Implementing Σ , Adequacy, or completeness logic in host code instead of calling the runtime.
- Fixing a failure at the stage where it became visible rather than the stage that owns it.

6.19 From Selection to Execution

A selected policy is a decision, not an event. Nothing in this chapter moved money, dispensed coffee, or changed the host's state. Chapter 7 takes the selected policy across the boundary the framework treats most carefully: building an execution license from the completed cycle, obtaining a single-use native authorization, invoking the host function through the runtime, translating what actually happened into truthful evidence, and applying the host-owned Layer-1 state transition. It is also where the Guide states, once and in full, what the runtime's synchronous decision core can and cannot do while an action is under way; Chapter 12 adds what the supervision layer of runtime v2.1.167 does around it.

Chapter 7 — Selection, Execution, and Environmental Realization

Chapter 6 ended with a selected policy. Nothing had happened yet: no coffee was dispensed, no money moved, and the host's state was unchanged. This chapter carries the selected policy across the boundary where implementations most often go wrong—the boundary between deciding, being authorized to act, attempting the act, and finding out what the world actually did.

Those four things are routinely collapsed into one in ordinary software. A function is called; it returns; the program records success and updates its state. The Productive Value–Productive Power (PV-PP) framework keeps them apart because each can fail independently. A selection can be sound while the authority behind it has gone stale. An authorized call can return normally while the world did something else. A call can fail after it has already changed the world. Treating any of these as the others is how a governed system quietly loses its governance at the last step.

The chapter first explains the framework's execution operator, ϵ , and then the runtime machinery that implements the path from selection to realized state: the execution license, the execution episode, single-use native authorization, authorized call-through, execution evidence, and the host-owned Layer-1 transition. Section 7.12 extends the coffee program to execute a purchase under five conditions. Section 7.15 is the Guide's full statement of what the runtime's decision core can and cannot do; other chapters refer to it rather than restating it, and Chapter 12 states the supervision layer's own boundary.

Selected is not authorized. Authorized is not attempted. Returned is not realized.

7.1 Four Events, Not One

Between a decision and a changed world lie four separate events, each owned by a different part of the architecture and each proving something narrower than it appears to.

Table 7.1 — The four events between decision and realized state

Event	Owned by	What it establishes	What it does not establish
Selection	Σ	This policy is the governed choice among the adequate candidates at this decision state.	That it may be executed now, that the facts it relied on still hold, or that it will succeed.
Current authority	The execution lineage (license, episode, authorization)	This exact action, under this exact binding, may be attempted once, now.	That the world will cooperate, or that the attempt produced any effect.
Attempt	ϵ , through the authorized call-through	The call was made, and it ended in one of four classified outcomes.	What changed in the world. A normal return is technical completion only.
Realization	The host and its Layer-1 transition	What actually changed, recorded as the actor's next actual state.	That the change was the intended one.

The table's last column is the discipline of the chapter. Each event must be checked on its own evidence. None may be inferred from the one before it.

7.2 Σ Selects, ϵ Realizes

In the owning V2.1 specification, ϵ is the bounded realization operator. It receives the selected policy from Σ together with the upstream licensing envelope—the governing-domain structure, the fixed preservation target, the adequacy classification, and the feasibility posture already established—and instantiates the policy as a concrete execution episode. It monitors that episode, allows bounded local adaptation inside the licensed corridor (pacing, sequencing, assistance uptake, licensed staging), records what actually occurred, and returns upstream when execution can no longer remain licensed.

The specification states ϵ 's outputs as a set of distinct objects: the instantiated episode $x_{\text{exec}}(t)$; the bounded within-execution state $E_{\text{exec}}(t, \tau)$; the realized execution path $\text{ExecPath}(t)$; an auditable status $\text{status}_{\text{exec}}(t)$; the realized Productive-Value bundle or staged sequence $B_{\text{real}}(t)$; and execution-generated information $\text{info}_{\text{exec}}(t)$. The status classifies the outcome, the path records what occurred, and the bundle records the realized value consequence. The three must not be used interchangeably.

The canonical status classes are completed, completed_staged, partial_realization, aborted_return, failed, and emergency_return. ϵ tracks three sufficiency conditions: entry sufficiency (licensed strongly enough to begin), continuation sufficiency (still licensed after execution reveals more), and completion sufficiency (able to finish as structured without violating the upstream envelope).

Two boundaries define ϵ . Upward, it is not a second selector: it must not invent a rival policy class, invoke fallback on its own, redefine governing domains, repair framing or adequacy locally, or treat an upstream-invalid policy as valid because execution happened to go well. A material change of policy class requires abort and return upstream. Downward, ϵ does not change actual state. Layer 1 owns the ontological update $S_i(t+1) = T_i(S_i(t), B_i(t), \dots, \text{ExecPath}_i(t))$. A selected policy therefore never changes actual state directly, and neither does a completed execution episode.

The runtime objects introduced below—licenses, native authorizations, bindings, and call-through results—are implementation machinery serving ϵ . The owning specification is explicit that implementation detail between the selected policy and the execution episode must remain subordinate to ϵ and may not acquire independent selection authority. They are not new operators and they are not a separate executable-policy object.

7.3 The Execution Lineage in the Runtime

The frozen runtime implements the path from a selected policy to realized state as a lineage of runtime-issued objects. Each link can be created only from the one before it, and each checks something the next link relies on.

Table 7.2 — The execution lineage

Step	What it checks or produces	What it does not confer
Execution license	Built only from a completed cycle resolved by Σ . Rejects stale or identity-inconsistent cycle artifacts. Records the selection mode and the upstream licensing posture.	Execution of any particular action.
Execution episode	Instantiates ϵ from one current license, once. Active only if the license authorizes entry and the host declares entry sufficiency.	Authority to call any function.
Binding registration	Associates a host callable with a registered action, under a binding identity and implementation version.	Any execution authority. Registration is representation.
Native authorization	Single-use, runtime-ledger authority for one attempt of one licensed action through one binding version.	Validity after consumption, invalidation, or a control-state change.
Authorized call-through	Consumes the authorization, calls the bound function, and classifies the outcome.	Knowledge of what the world did.
Execution evidence and ϵ step	Converts the classified result and the host's realized-effect evidence into an ϵ observation and a terminal ϵ status.	A state change.
Layer-1 handoff and transition	The host transition produces the next actual state; the runtime validates attribution, identity, time, and declared invariants.	Interpretation of what PP, SPV, AVS, or X now mean.

Exact method signatures, field lists, and result-normalization tables belong to the PV-PP Programmer's Reference. The sections that follow explain what each link is for and what mistakes it prevents.

7.4 The Execution License and the Episode

An execution license is the runtime's record that ϵ may be entered for the selected policy. It is built from a completed canonical cycle, never assembled by hand. Before building it, the runtime re-audits the cycle's artifacts to confirm that they all refer to the same state, governing set, candidate set, feasible set, and projection records; a cycle whose parts have drifted apart is rejected rather than licensed.

The license preserves how Σ reached its selection, because the three selection modes carry different execution authority:

- **Standard.** The selected policy passed Π completeness, Constraints, Domain Framing, and Adequacy. Entry is authorized.
- **Recovery-unavailable fallback.** No policy was adequate, and the governed fallback structure selected a feasible, framed policy. Entry is authorized, but the license keeps the policy

marked as recovery-inadequate. Fallback authority comes from the fallback selection itself, not from relabeling the policy as adequate.

- Terminal infeasibility. Constraints left no feasible candidate, and Σ 's terminal mode chose among infeasible ones by severity. The selection is preserved for traceability, but entry is blocked: ε may not turn upstream infeasibility into executable validity.

The license is then used once to instantiate an execution episode. The host states whether entry is sufficient under current conditions and how many bounded steps the episode may take. If the license does not authorize entry, or the host reports that entry is not sufficient, the episode is created inactive and returns upstream immediately. Nothing is called.

7.5 Registration Is Not Authorization

A host function becomes callable through the runtime only after it is registered in a binding registry against a registered action, with a binding identity, an implementation version, and provenance. Registration answers “what function would carry out this action?” It does not answer “may this action be carried out now?” The registry's own direct invoke route is fail-closed: calling it raises an error, because a registration carries no authority.

The implementation version matters. Authorization binds the exact version that was registered when authority was issued. If the function behind an action is replaced, an authorization issued for the earlier version no longer matches and the call is refused.

7.6 Native Authorization: Single-Use and Bound

A native execution authorization is the runtime's grant of authority for exactly one attempt. It is issued only for a current, active episode instantiated from a current license, only for an action the license covers, and only when that action has a registered binding. It binds the episode, the decision-cycle identity, the selected policy, the action, the binding and its implementation version, the configuration identity, and the attempt number.

Authority lives in the runtime's private ledger, not in the object the host holds. When the host presents an authorization, the runtime checks that it is the very record the ledger issued—a reconstructed copy with matching field values is refused—and that its status is still issued. It then marks the authorization consumed immediately before entering the function. A second presentation of the same authorization is refused, and two concurrent attempts with one authorization produce at most one call.

Authority can also end before it is used. The host can invalidate outstanding authorizations by episode, decision cycle, or action, stating a reason; an invalidation by episode or decision cycle also retires the matching active episode. The host can also place an attempt under a control state—paused, aborted, or reauthorization required—which fails the call closed before the function is entered. A reauthorization request produces a handoff whose only meaning is “return to governance for fresh authority.” None of these mechanisms decides where re-evaluation should begin; that remains a governance question (Section 7.11).

This protects the PV-PP runtime-interface authority boundary. It is not a sandbox, an operating-system or network control, or an identity-and-access system, and it cannot stop host code from calling its own function directly. Within a correctly integrated PV-PP execution path, an action does not run without current authority; making that path the only path is an integration responsibility.

7.7 Call-Through and the Four Outcome Classes

The authorized call-through invokes exactly the bound function and classifies how the attempt ended. The classification is about what the attempt can prove, not about whether the host is satisfied.

Table 7.3 — Call-through outcome classes

Outcome	Meaning	External effect possible?	ϵ terminal status
succeeded	The function returned normally. This is technical completion only.	Yes	completed
failed_cleanly	The function reported failure together with evidence that no material effect occurred.	No	failed, returning upstream
indeterminate	The function failed and an effect cannot be ruled out: a timeout, a lost connection, a declared unknown, or any unclassified error.	Yes	partial_realization if an effect was observed; otherwise aborted_return
completed_invalid	The function finished, but the realized result was invalid.	Yes	partial_realization if an effect was observed; otherwise aborted_return

The asymmetry in the table is deliberate. A function may claim a clean failure only by saying so explicitly, and by doing so it asserts that nothing material happened. Every other failure—including an ordinary exception the programmer did not anticipate—defaults to indeterminate, because a function that failed partway may already have acted. Absence of a response is not evidence that nothing happened. Classifying an unknown effect as a known failure would let the next cycle proceed as if the world were unchanged, which is exactly the error the classification exists to prevent.

A succeeded result is equally limited. The function returned; that is all. Whether the intended change occurred is a question for the host's evidence and the Layer-1 transition, not for the return value.

7.8 From Result to Evidence

The call-through result enters ϵ as an execution observation. The host supplies, alongside the classified result, its own evidence of what was realized—drawn from the records the host treats as authoritative, such as sensors, ledgers, confirmations, or counterparties—not from the function's return value. The runtime translates the result and evidence into the observation and advances the episode, which becomes terminal with one of the canonical statuses. The episode's authorizations are retired when it becomes terminal.

Two details matter in practice. First, whether a realized-effect bundle is present changes the status: an indeterminate attempt with an observed effect is a partial realization; without one, it is

an abort and return. Reporting an effect you did not observe, or omitting one you did, therefore changes what governance sees. Second, the observation step uses an execution context that carries the attempt's identity. The call-through does not return that context, so the host reconstructs it from the result and the authorization, as the runtime's own tests do. The context is an identity and correlation record; it carries no authority.

The binding registry also offers a direct Layer-1 handoff builder that bypasses the ϵ trace, and its status mapping differs—for example, it classifies a completed-but-invalid attempt as failed. This Guide uses the ϵ path, which the runtime documents as the single canonical execution-evidence chain. The difference is recorded for the Reference.

7.9 The Layer-1 Transition

A terminal ϵ result produces a Layer-1 handoff: the episode and selected-policy identity, the ϵ status, the execution path, the realized bundles, the execution information, whether governance must be re-entered, and the actions the license covered. The handoff contains no next state. It is the input to the host's transition, not its result.

The host owns the transition. In the runtime it is a service the host attaches, and it is called only when the host explicitly requests it; neither the decision cycle nor ϵ ever calls it. The service receives the current actual state—an envelope whose outer structure mirrors $S_i(t) = \{PP_i(t), SPV_i(t), AVS_i(t), X_i(t)\}$ —and returns the next state together with any invariant checks the host declares.

The runtime validates the transition without interpreting it. It checks that the result is attributed to the same episode and policy, that it starts from the current state, that the actor is unchanged, that a new state identity was produced, that time did not move backward, that all four state components are present, and that every declared invariant held. It does not decide what PP, SPV, AVS, or X now contain; those meanings belong to the host and the Layer-1 authority.

Two optional records can follow a validated transition. An execution-transition provenance record links the prior state, the episode, and the next state for audit and for later cycles; it confers no authority. And where recovery obligations are active, the host confirms which registered recovery actions were actually realized. The runtime does not infer that from event names, path labels, or policy membership.

7.10 Every Terminal Episode Reaches Layer 1

It is tempting to apply a Layer-1 transition only after success. The shortcut is unsafe, and this Guide sends every terminal episode to the host's transition. A partial realization changed the world. A completed-but-invalid attempt changed it in the wrong way. An indeterminate attempt may have changed it in a way the host must still reconcile. Even a clean failure is a recorded event: its next state is a new state, even when nothing material changed. The host's transition decides what actually changed in each case, and the next decision cycle starts from that state rather than from the state the decision assumed.

7.11 Returning Upstream

Several ϵ outcomes request a return upstream. That request is not an instruction to retry, and ϵ does not choose what happens next. Returning upstream means that the next decision is made by governance from the updated state: perception forms a new $P_i(t)$ from the new actual state, and the cycle runs again, or—when the host can identify exactly which earlier artifacts the change invalidated—re-enters at the earliest affected stage, as Chapter 6 described.

The same applies to stale authority. When a fact the decision relied on changes after authorization, the host invalidates the outstanding authority. The call-through then refuses before the function runs, and the matter goes back to governance. The native layer deliberately does not infer which stage must be re-entered; that determination belongs to the re-entry machinery, which Chapter 8 integrates into a complete application.

7.12 Program: Executing the Coffee Purchase

The Chapter 6 program selected a policy. This program, `coffee_execute.py`, carries the selection through to realized state. It imports the Chapter 6 registry, adapter, and request unchanged and adds three things: a stand-in for the external world, a host-owned Layer-1 transition, and the execution lineage itself. It runs against the unmodified frozen runtime; the accompanying test file checks each behavior described below.

The machine is the world. Its `sell` function is the registered native function, and a fault setting injects one realistic failure: an issuer decline before anything happens, a capture timeout after the cup is dispensed, or a delivery sensor that sees no cup after the charge. The machine's records—not the function's return value—are the evidence the host uses.

Listing 7.1 — The world and its native function

```
class CoffeeMachine:
    """The external world. `fault` injects one realistic failure into a sale."""
    def __init__(self, cups=12, network_up=True, fault=None):
        self.cups, self.network_up, self.fault = cups, network_up, fault
        self.revenue, self.capture, self.unserved, self.calls = 0.0, "none", 0, 0

    def sell(self, context):
        # the registered native function
        self.calls += 1
        if self.fault == "card_declined":
            # nothing happened, and we know it
            raise CleanExecutionFailure("issuer declined; nothing held or dispensed")
        self.cups -= 1
        # the cup is dispensed
        if self.fault == "capture_timeout":
            # money may or may not have moved
            self.capture = "unknown"
            raise TimeoutError("no response from payment network during capture")
        self.revenue += PRICE; self.capture = "captured"
        if self.fault == "no_delivery":
            # finished, but the result is wrong
            self.unserved += 1
            raise CompletedInvalidExecution("charge captured; sensor saw no cup")
        return {"receipt": context.execution_id}

    def records(self):
        # what the machine's sensors report
        return {"cups": self.cups, "revenue": self.revenue,
                "capture": self.capture, "charged_unserved": self.unserved}
```

The Layer-1 transition builds the next actual-state envelope from the machine's records. Unresolved buyers—a capture whose outcome is unknown, or a buyer charged and not served—appear in actual viability state, where the next cycle will see them.

Listing 7.2 — The host-owned Layer-1 transition

```
def open_items(r):
    items = ["reconcile capture"] if r["capture"] == "unknown" else []
    return items + ["refund owed: charged, not served"] * r["charged_unserved"]

def envelope(machine, state_id, time):
    r = machine.records(); items = open_items(r)
    return ActualPersistentStateEnvelope(
        actor_id="coffee-machine", state_id=state_id, time=time,
        pp={"dispense": "operational",
            "payment": "up" if machine.network_up else "down"},
        spv={"revenue_captured": r["revenue"]},
        avs={"unresolved_buyers": len(items)},
        context={"cups": r["cups"], "open_items": tuple(items)})
```

```

class CoffeeLayer1:
    """Next actual state from the machine's records, attributed to the episode."""
    def __init__(self, machine): self.machine = machine
    def transition(self, current, handoff):
        nxt = envelope(self.machine, f"{current.state_id}+1", current.time + 1)
        return Layer1TransitionResult(
            handoff.episode_id, handoff.selected_policy_id, current.state_id,
            nxt, True, {"cups_nonnegative": nxt.context["cups"] >= 0})

```

The purchase function is the execution lineage from Table 7.2 in order: evaluate the cycle, build the license, instantiate the episode, register bindings, obtain one authorization, call through, gather evidence, advance ϵ , and apply the Layer-1 transition. The stale-authority case changes a fact after authorization and invalidates the authority before the call.

Listing 7.3 — One governed purchase, from cycle to realized state

```

def purchase(machine, actual, *, stale_before_call=False):
    rt = PVPPRuntime(build_registry(), CoffeeWorld(),
                     projection_service=CoffeeProjection(),
                     layer1_transition_service=CoffeeLayer1(machine))
    req = request()
    cycle = rt.evaluate_canonical_decision_cycle(perceive(actual), req) # Ch. 6
    license = rt.build_execution_license_from_cycle(cycle, req.domain_frame)
    episode = rt.instantiate_execution("purchase-1", license,
                                     entry_sufficient=True, max_steps=1).episode
    action = license.action_ids[0]

    bindings = ExecutionBindingRegistry(tuple(rt.registry.actions)) # not authority
    bindings.register(ExecutionBindingIdentity("sell-v1", "sell", "1.0"),
                     machine.sell)
    bindings.register(ExecutionBindingIdentity("decline-v1", "decline", "1.0"),
                     lambda context: {"declined": True})
    auth = rt.issue_native_execution_authorization(
        episode, action, bindings, decision_cycle_id="cycle-1")

    if stale_before_call: # a fact the decision relied on has changed
        machine.network_up = False
        rt.invalidate_native_execution_authorizations(
            decision_cycle_id="cycle-1", reason="payment network reported down")
    try:
        result = rt.invoke_authorized_native(auth, bindings) # consume, then call
    except RuntimeError as refused:
        return {"selected": license.selected_policy_id, "refused": str(refused),
              "calls": machine.calls}
    try:
        rt.invoke_authorized_native(auth, bindings) # single use
        reuse = "accepted"
    except RuntimeError:
        reuse = "refused"

    after = machine.records() # host evidence, not result
    dispensed = actual.context["cups"] - after["cups"]
    realized = {"cups_dispensed": dispensed, "capture": after["capture"]} \
        if dispensed else {}
    ctx = ExecutionContext(result.execution_id, auth.action_id, auth.binding_id,
                           auth.decision_cycle_id, auth.attempt)
    obs = bindings.epsilon_observation(ctx, result, realized_pv_bundle=realized)
    step = rt.advance_execution(episode, obs) # epsilon
    handoff = rt.build_layer1_transition_handoff(step)
    new_actual = rt.apply_layer1_transition(actual, handoff).next_state # Layer 1
    return {"selected": license.selected_policy_id, "native": result.status,
          "epsilon": step.status, "return_upstream": step.return_upstream,
          "reuse": reuse, "calls": machine.calls, "actual": new_actual}

```

Running the file executes five purchases and one follow-up cycle. The output is reproduced exactly:

Listing 7.4 — Program output

```

== A. normal sale
selected:      graph:sell
native result: succeeded | authorization reused: refused
epsilon:       completed | return upstream: False
actual state:  cups=11 revenue=2.50 unresolved=0

== B. card declined
selected:      graph:sell
native result: failed_cleanly | authorization reused: refused
epsilon:       failed | return upstream: True
actual state:  cups=12 revenue=0.00 unresolved=0

== C. capture timed out
selected:      graph:sell
native result: indeterminate | authorization reused: refused
epsilon:       partial_realization | return upstream: True
actual state:  cups=11 revenue=0.00 unresolved=1 ['reconcile capture']

== D. no cup delivered
selected:      graph:sell
native result: completed_invalid | authorization reused: refused
epsilon:       partial_realization | return upstream: True
actual state:  cups=11 revenue=2.50 unresolved=1 ['refund owed: charged, not served']

== E. network fails after authorization, before the call
selected:      graph:sell
call-through:  refused before the function ran
reason:        native execution authorization lifecycle status is invalidated
function calls: 0

== E'. next cycle, from the updated facts
selected:      graph:decline
native result: succeeded | authorization reused: refused
epsilon:       completed | return upstream: False
actual state:  cups=12 revenue=0.00 unresolved=0

```

The example simplifies in the same ways as Chapter 6—perception is exact and each policy is one step—and in one more: the Layer-1 transition reads the machine directly, where a production host would read its own persistent records. Chapter 8 replaces these with a fuller adapter and application structure.

7.13 Reading the Five Cases**Table 7.4 — What each case shows**

Case	Outcome	What it shows
A. Normal sale	succeeded → completed	Only now does actual state record the sale: one cup fewer, revenue captured. The same authorization presented again is refused.
B. Card declined	failed_cleanly → failed	The function declared a clean failure, so no effect is asserted. State is unchanged apart from time, and the decision returns upstream.
C. Capture timed out	indeterminate → partial_realization	The cup was dispensed but money may or may not have moved. The timeout is not treated as a failure; the unknown is carried into actual state as a capture to reconcile.

Case	Outcome	What it shows
D. No cup delivered	completed_invalid → partial_realization	The function finished and the charge was captured, but the result was wrong. Actual state records a refund owed; completion is not laundered into success.
E. Network fails after authorization	refused before the call	The authority was invalidated when the fact it relied on changed. The function never ran. The next cycle, from the updated facts, declines (E ').

Cases C and D leave the machine in states no ordinary success-or-failure flag can express: an unserved buyer who may or may not have been charged, and a charged buyer who was not served. Because both are recorded as actual state, the next decision cycle starts from them. That closes the loop the Guide has been building since Chapter 3, where the purchase was modeled as nested interactions precisely because authorization, capture, dispensing, delivery confirmation, and refund can fail separately.

Case E is the authorization-to-call boundary working as designed. It is not a mid-flight cancellation. The change was reported before the call began; had the network failed during the call, the result would have been Case C. Section 7.16 explains why that distinction matters.

7.14 The Same Boundary in Other Domains

For a household, selecting a repair is not paying for it, paying is not having the car fixed, and a mechanic's "done" is not a working car; the next decision starts from the car's actual condition. For an organization, a board approval is not a signed contract and a sent payment instruction is not a settled payment; an unacknowledged wire is indeterminate until reconciled. For an engineered system, a commanded valve position is not the measured one. For an autonomous agent, a tool call that times out after submitting an order has an unknown effect, and treating the timeout as a clean failure invites a duplicate order. In each case the same four events apply, and the same rule decides what the next cycle may assume.

7.15 The Decision Core's Implementation Boundary

This section is the Guide's full statement of what the runtime's decision core implements: the core of frozen Runtime V2.1 v0.141, carried byte for byte into runtime v2.1.167. It separates four things that are easy to run together: what the framework permits, what the runtime does, what the host must supply, and what remains prospective work. Build 167's supervision layer, which works around this core rather than inside it, appears in Table 7.5 and is described in Chapter 12.

What the decision core implements. The core is synchronous. It implements the canonical decision cycle through Σ , the execution lineage described in this chapter, pre-call authorization with single-use consumption, ledger-backed lifecycle and invalidation, binding-version and configuration matching, synchronous pre-invocation control states, outcome classification with a conservative default, ϵ status and trace handling, Layer-1 handoff and validation, execution-transition provenance, recovery bookkeeping from validated transitions, and the bounded re-entry machinery Chapter 6 described. Changes reported to it between governance entries are processed synchronously.

What the decision core does not implement. It does not sense the world on its own. It has no asynchronous sensing, no external instrumentation hooks, no autonomous detection of

environmental change while a call is in flight, no way to inject an observation into a call already under way, no concurrent or distributed execution framework, no continuous supervision of distributed execution, and no guaranteed mid-flight cancellation. Its control states are checked before the function is entered and have no effect on a call already running. Build 167's supervision layer supplies some of this around the core: registered monitors as sources of evidence, checkpoints of an action while it runs, and governed requests to stop it. It does not supply autonomous sensing, governance of a native call already entered, or physical cancellation (Chapter 12).

What else it does not claim. The decision core is not a persistent runtime database (the supervision layer's durable store holds supervisory records, not the host's world), an autonomous semantic-discovery engine, a mechanism that invents PP domains, an LLM semantic adapter, a learning algorithm, an unrestricted self-modification mechanism, or a replacement for operating-system, identity-and-access, network, credential, or sandbox enforcement. Reachability at scale is an engineering concern outside its governance semantics.

What the host owns. The host supplies authoritative actual state and world mechanics, domain measurements and their interpretation, the semantic completeness of transition-relevant state, objective content, external persistence, sensors and instrumentation, empirical and capability evidence, external enforcement, environmental effects, and the authoritative Layer-1 transition.

Table 7.5 — Framework, decision core, supervision layer, and host

Concern	Framework V2.1	Decision core (since v0.141)	Supervision layer (v2.1.167)	Host responsibility
Change during execution	ϵ may monitor, revise continuation, and return upstream mid-execution	Processes changes reported between steps and before calls; no in-flight intake	Admits reports from registered monitors, finds the affected executions, and governs each at a checkpoint the host calls	Register sources and dependencies; checkpoint every affected execution
Stopping an action	Abort and return when continuation is no longer licensed	Refuses calls under invalidation or control state; cannot stop a running call	Records a governed stop request, translates it for the controller, and records the outcome; cannot govern a native call already entered	Carry out the stop; make stoppable actions stoppable in the world
Unknown outcomes	Unknown effects are not treated as failure	Defaults unclassified failures to indeterminate	Keeps unknown and partial effects unresolved; a timeout never becomes no effect	Reconcile against authoritative records

Concern	Framework V2.1	Decision core (since v0.141)	Supervision layer (v2.1.167)	Host responsibility
State after execution	Layer 1 owns the transition	Validates attribution, identity, time, and declared invariants	Records a Layer-1 transition only from a current effect reconciliation and verified host authority	Compute the next actual state
Enforcement	Execution requires current authority	Enforces authority at its own interface only	Keeps request, dispatch, and enforcement evidence apart; trusts a controller's report only through the host's verifier	Ensure the governed path is the only path

The table's second column is not a feature list for either runtime column. The framework permits governance around in-flight change where suitable instrumentation and execution architecture exist; only what the third and fourth columns state may be attributed to runtime v2.1.167.

7.16 The Authorization-to-Effect Race

Every synchronous design leaves one gap. Authority is checked and consumed immediately before the function is entered. The world keeps moving. Between that moment and the moment the function's effect lands, a fact the decision relied on can change, and the decision core has no way to notice.

The runtime narrows the gap as far as a synchronous design can: authority is consumed as late as possible, every reported change can invalidate outstanding authority, and an attempt whose effect is unknown is never reported as a clean failure. It cannot close the gap. What closes the rest is host design:

- Keep each authorized action short, so the interval in which the world can change is small.
- Split actions whose steps can fail separately—authorize, capture, dispense, confirm—into separately represented interactions, as Chapter 3 recommended, so each step is authorized on current facts.
- Make actions idempotent where the world allows it, so a reconciled retry cannot double an effect.
- Keep authoritative records that let every indeterminate outcome be reconciled, and carry unreconciled outcomes into actual state until they are resolved.
- Where an effect is costly and irreversible, require confirmation from the world before recording realization.

These are integration practices, not runtime features. Chapter 8 builds them into the application; Chapter 9 tests them.

The supervision layer of build 167 narrows the gap further for actions the host carries out by its own means while an execution episode stays open: a change reported by a registered monitor reaches the execution at its next checkpoint, and a governed stop request can follow. It does not narrow the gap for a function entered through native call-through. Once authority is consumed at call entry, the supervision layer can no longer checkpoint that execution (Section 12.13). Work that may need to be stopped therefore belongs outside native call-through.

7.17 Extending Without a Second Decision System

Future observation intake or execution supervision can close more of the gap, and the architecture already says where it must connect. An observation arriving during execution is an ε observation on the existing episode: it may report that continuation is no longer sufficient, that an emergency return is required, or that a material policy-class change would be needed, and ε then ends the episode and returns upstream. A change that makes outstanding authority stale uses the existing invalidation path. A reauthorization request uses the existing handoff back to governance. Multi-step episodes already exist, so supervision would deliver evidence at step boundaries rather than invent a new loop.

What such an extension must not do is decide. A supervisor that chooses a replacement action, retries with a different policy, or relaxes a constraint because the situation is urgent has become a second decision system operating outside Σ , Constraints, and Adequacy. The rule is the one ε already follows: new evidence returns to governance; it does not substitute for it. Runtime v2.1.167 is the first such extension, and Chapter 12 shows that it keeps to this rule: its checkpoint produces a continuation posture from canonical ε or from the re-entry planner, and nothing else; it never chooses a replacement action. Further extension remains prospective work.

7.18 A Known Interface Gap: Ending a Recovery Corridor

When a recovery policy is executed, the runtime records an active recovery corridor—the remaining actions and deadline of a staged recovery—and updates it through recovery bookkeeping as the host confirms realized actions. The public runtime interface offers no operation for explicitly cancelling or expiring an active corridor, for example when an offer lapses, a counterparty withdraws, or the deadline passes without completion.

The Small Economy stress test of frozen v0.70 met this gap and handled it with an application-side shim that wrote the corridor's closed-expired status directly into the runtime's corridor record, driven by the host's authoritative period progression, and never marked it complete. Inspection of v0.141, and of build 167, shows the gap unchanged: corridor records change only through the runtime's recovery-bookkeeping paths, and no public cancellation or expiry operation exists.

Two consequences follow. Applications that need corridor expiry must record it as an explicit host-side limitation, keep their own authoritative record of the corridor's lifecycle, and make sure an expired corridor is never treated as completed. Writing directly into runtime-private state, as the shim did, bypasses the runtime's validation and should be treated as a documented workaround, not an integration pattern. A governed cancellation and expiry operation is a candidate interface for a future runtime and is carried to Chapter 11's open work.

7.19 Common Errors at the Execution Boundary

- Treating selection as authorization, or authorization as an attempt.
- Updating actual state from a function's return value instead of from authoritative evidence and the Layer-1 transition.
- Classifying a timeout or unexpected exception as a clean failure.
- Declaring a clean failure without evidence that nothing material happened.
- Reporting a realized effect that was not observed, or omitting one that was.
- Applying a Layer-1 transition only after success.
- Constructing a license or authorization by hand, reusing one, or keeping one after the facts behind it changed.

- Calling the host function directly, outside the governed path, and still describing the action as governed.
- Letting execution retry with a different action or policy class instead of returning upstream.
- Treating the recovery-unavailable fallback license as though the policy were adequate.
- Describing a pre-call control state or invalidation as mid-flight cancellation.
- Attributing framework-level in-flight governance to the runtime beyond what its supervision layer implements (Chapter 12).
- Writing into runtime-private state to work around an interface gap without recording it as a limitation.

7.20 From One Purchase to an Application

The coffee program now decides and acts: it selects a policy, holds single-use authority to attempt it, classifies the attempt honestly, and records what the world did as actual state. What it lacks is an application around it—perception formed from evidence rather than copied from state, a persistent store, re-entry driven by the runtime's invalidation machinery, reconciliation of unknown outcomes, and governed remedies for harms it has caused. Chapter 8 assembles those pieces into a complete PV-PP application.

Chapter 8 — Building a PV-PP Application

Chapters 3 through 7 produced a model and two small programs: one that evaluates a decision cycle and one that executes a purchase. Neither is an application. An application runs many episodes. It keeps actual state between them, forms each decision from current evidence, takes in reports from the outside world, withdraws authority when a relied-on fact changes, reconciles outcomes it could not observe, and remedies harms it caused. This chapter shows how to build one without letting application code take over any of the work the architecture assigns to governance.

The chapter has three parts. Sections 8.1 through 8.9 give the implementation workflow: how the definition packets from earlier chapters map onto runtime structures, where the boundary between host and runtime lies, and what the host must build around the runtime—perception, a persistent store, a Layer-1 transition, observation intake, reconciliation, and traceability. Sections 8.10 and 8.11 assemble the coffee application and run it through a day of ordinary and failing episodes. Sections 8.12 through 8.15 cover practices around the code: when a scalar shortcut may be trusted, how AI assistance fits the workflow, and how to read the runtime's own Small Economy benchmark and the earlier simulations.

**The application supplies facts and carries out effects. The runtime governs.
Nothing in between decides.**

8.1 From Definition Packets to Runtime Structures

An application should be built from the packets produced by the earlier prompts, not invented at the keyboard. Each packet item has a home in the runtime, and each home has a rule about what it may contain. Table 8.1 gives the mapping used in this chapter; exact constructors and fields belong to the PV-PP Programmer's Reference.

Table 8.1 — Where each modeled item lives in an application

Modeled item (packet)	Runtime structure	In the coffee application
Governing domains and thresholds (A.3, A.6)	Registered domains	Delivery and transaction integrity
Actual state $S_i(t)$: PP, SPV, AVS, X (A.3)	Host store, exposed as the actual-state envelope	Store, with open charges and refunds owed in AVS
Perceived state $P_i(t)$ and evidence (A.4)	Host perception hook returning a typed perceived state	Network status judged current only while its report is fresh
Objectives $O_i(t)$ (A.3)	Optional objective adapters	None; see Section 8.14
Transformations, families, classes (A.5)	Registered Graph instance and transformations; required families and classes in the request	Sell, decline, and refund
Binding rules (A.6)	Registered constraint rules; host constraint profile	Card network, reconciliation hold, refund basis

Modeled item (packet)	Runtime structure	In the coffee application
Recovery and consequences (A.6)	Host projection records with recovery-corridor facts	Horizons and corridors per policy
Evidence dependencies (A.4, A.6)	Registered artifact dependencies	Constraints depends on network and reconciliation facts
Actions and functions (A.7)	Execution bindings	Machine functions for sell, decline, refund
Realized-state update (A.7)	Host Layer-1 transition service	Store transition from realized-effect evidence

8.2 The Application–Runtime Boundary

The runtime governs the represented decision and execution process through the interfaces it implements. The host owns everything the runtime does not interpret: authoritative actual state and world mechanics, domain measurements and their meaning, the semantic completeness of transition-relevant state, objective content, persistence, sensors and instrumentation, empirical and capability evidence, external enforcement, environmental effects, and the Layer-1 transition. Section 7.15 states the runtime side of this boundary in full.

The boundary has one rule that matters more than the others: host code supplies facts and must not smuggle governance into them. A projection must not quietly shorten a horizon to make a policy lose. A constraint profile must not report a violation because a policy is merely unattractive. A perception hook must not drop a candidate-relevant fact because it complicates the decision. Policy selection, adequacy judgments, Σ ordering, re-entry decisions, and execution authority belong to the runtime; an adapter that encodes any of them has built a second decision system inside the first.

The practical test is to ask of every line of host code: does this state something true about the world, or does it decide something? Statements belong in the host. Decisions belong in the runtime, reached through the stages.

8.3 Registration and Runtime Prerequisites

A runtime instance is built from a registry and a host adapter. The registry declares what exists for governance: domains with failure thresholds, actions, the governing tolerance and regime thresholds, constraint rules with their classes, the Graph's instances and transformations, and a Σ order used only to break exact ties.

One prerequisite is easy to miss. The runtime requires a registered action named "steady." It is the baseline continuation the collapse-horizon stage uses to ask how long each domain lasts if nothing new is done; the runtime refuses to start without it. Its meaning is domain-owned—in the coffee model it is an idle machine with a waiting buyer—but it must exist and must describe what actually happens under continuation, not what the modeler hopes would happen.

Registration is representation. Registering an action, a transformation, or an execution binding grants no authority to use it, and registering a transformation as reachable does not make it permitted. New structure discovered after start-up enters through controlled admission (Chapter 5), not by editing the registry in place.

8.4 The Adapter: Facts, Not Decisions

The adapter answers the questions each stage asks of the domain. For Φ and H it supplies margins, drift, and how pressure is measured. For Constraints it reports, per candidate, whether each registered rule is violated. For Adequacy and Σ the projection service supplies one shared projection record per candidate: projected horizons for each domain and the recovery-corridor facts. The runtime consumes the same records in both stages and recomputes none of them.

Deciding which fact belongs in which surface is the central modeling judgment, and the coffee application shows two contrasting cases. While a card capture's outcome is unknown, the owner's rule is that no new card sale may begin. That is a prohibition, so it is a hard constraint and Constraints removes the sale. While a refund is owed, however, selling is not prohibited—it simply leaves a charged buyer unremedied. That is a consequence, so it appears in the projection: every policy except the refund leaves transaction integrity failing, and Adequacy rejects them. Encoding the refund as a prohibition would hide the reason; encoding the capture hold as a bad horizon would turn a rule into a score. Each belongs where its meaning puts it.

- Report violations, not preferences. A constraint observation says whether a rule is broken, nothing more.
- Project consequences honestly and identically for every stage that consumes them. Do not tune a projection to steer the outcome.
- Mark projections past the licensed horizon as censored rather than inventing values.
- Never rank, filter, or choose in the adapter. If a candidate should not exist, fix the Graph; if it should not run, fix the constraints; if it cannot recover, let Adequacy say so.

8.5 Perception from Evidence

Chapter 6 treated the represented state as already perceived. An application must form $P_i(t)$ from actual state on every cycle, and the runtime provides a typed hook for it: the host returns a perceived decision state attributed to the actual state it came from, carrying perceived productive capability separately from other perceived components, confidence, and uncertainty. The runtime validates only attribution and routing—same actor, same state identity, same time, and a perceived capability component present. It does not derive any perceived content itself.

The coffee application's perception applies one information-governance rule from Chapter 4: evidence has an age, and stale evidence is not current evidence. A network health report counts as current for two periods. After that, perception reports the network as unknown, and because a card sale requires a network known to be reachable, Constraints removes card actions until fresh evidence arrives. Nothing in the decision cycle changed; the perceived state did, and the stages did the rest.

8.6 The Store and the Layer-1 Transition

The host store is the authoritative record of actual state. Every recorded change produces a new state with a new identity, and every change is logged with its reason: a transition from an execution episode, a report taken in, or the passage of time. The runtime never writes to it.

The Layer-1 transition updates the store from the terminal ϵ handoff. It reads two things: the realized-effect bundles the host assembled from its own sensors and records—cups dispensed, revenue moved, charges left unknown, refunds owed—and the execution information gathered during the episode, which in this application carries the network heartbeat observed at the time of the call. The transition returns the next state with declared invariants, and the runtime validates attribution, identity, time, and those invariants.

After a validated transition, the runtime can build an execution-transition provenance record. The application passes it into the next cycle only when the store has not changed in between; the runtime then checks that the cycle starts from exactly the state the previous transition produced. If a report was taken in first, the state has legitimately moved on and the provenance record no longer applies.

8.7 Observation Intake and Enforcement Interfaces

Outside systems report facts: a network monitor, a payment processor's settlement file, a clock. Intake records each report as evidence in the store and names the facts it changed. It decides nothing. Whether a change matters to a decision is determined by dependencies the application registered when it made the decision—in the coffee application, that the Constraints result relied on the network status and on the reconciliation status.

When a report arrives while authority is outstanding, the application gives the runtime the registered dependencies and the changes. The runtime matches them by explicit identity only—it never guesses a dependency from names or timing—and returns the earliest invalidated stage. If there is one, the application withdraws the outstanding authority and returns to governance. The call never happens.

Returning to governance means a new cycle from the updated actual state. The runtime also provides stage-specific re-entry executors that reuse still-valid upstream results; this application does not retain those result bundles, so it takes the conservative path and recomputes the cycle. Recomputing a stage that could have been reused costs time. Reusing one that could not be is an error.

Enforcement interfaces are kept separate from observation. A payment network that refuses an unauthorized charge, an operating-system permission, or a physical interlock enforces effects in the world; it is not a second PV-PP authority, and its refusal enters the application as a fact—a clean failure or a report—not as a decision. Neither an observation source nor an enforcement mechanism may select, relax, or authorize anything.

The coffee application builds its own intake, because build 141 had none. Runtime v2.1.167 supplies a reference intake in its supervision layer—source registration, admission, governed fact views, explicit dependencies, and fan-out—that follows exactly the rules of this section, and adds the in-flight checkpoint that a host-built intake cannot provide. Chapter 12 shows it on the operations agent. Either kind of intake is legitimate; neither may decide.

8.8 Reconciliation and Remedy

Chapter 7 left two kinds of outcome in actual state: effects that are unknown and harms that are known. An application must carry both until they are resolved, and it must resolve them through the architecture.

An unknown outcome is resolved by evidence. In the coffee application, a capture that timed out is held as unreconciled in actual viability state, and the capture-hold rule keeps new card sales from starting while it is open. When the processor's settlement file arrives, intake records what actually happened, the hold lifts, and the next cycle sees the corrected state. No decision was made about the old sale; a fact was learned.

A known harm is resolved by action, and the action is governed like any other. A buyer charged and not served is recorded as a refund owed. On the next cycle, Adequacy rejects every policy that leaves that buyer unremedied, and the refund—a corrective-repair transformation registered

in the Graph—is selected, authorized, executed, and recorded. The host never issues the refund on its own initiative; it supplies the facts that make the refund the only adequate choice.

8.9 Traceability

An application should be able to explain every change in its actual state. Three records make that possible: the store's change log, which says what changed and why; the runtime's cycle results, which say what each stage decided and which candidates each stage set aside; and the execution lineage—license, authorization, handoff, and provenance—which ties each realized change to the decision that authorized it. The runtime also supports a replay package for reconstructing a past decision from its recorded inputs, following the replay-sufficiency requirements of Chapter 4. Keep all three from the first version. Traceability added later is usually reconstruction, and reconstruction is where unrecorded assumptions return.

8.10 Program: The Coffee Application

coffee_app.py assembles the pieces. It extends the coffee model with a refund transformation and two further hard rules, and adds perception, a persistent JSON store, a Layer-1 transition, observation intake, and an application loop that runs one governed episode at a time. It runs against the unmodified frozen runtime; its test file checks each behavior described in Section 8.11. The listings show the parts that are new since Chapter 7.

Perception forms $P_i(t)$ from the actual-state envelope, including the age of the network evidence:

Listing 8.1 — Perception from recorded evidence

```
def perceived_decision_state_from_actual(self, actual):
    net = actual.context["network"]
    age = actual.time - net["as_of"]
    network = net["status"] if age <= MAX_EVIDENCE_AGE else "unknown"
    owed = actual.avs["refunds_owed"]
    facts = {"state_id": actual.state_id, "cups": actual.context["cups"],
            "network": network, "refunds_owed": owed,
            "unreconciled": bool(actual.avs["unreconciled"])}
    margins = {"D": 10.0, "T": 10.0 - 3.0 * owed}
    return PerceivedDecisionState(
        actual.actor_id, actual.state_id, actual.time,
        WorldState(actual.time, margins, facts),
        ppp={"dispense": "operational", "card_payments": network},
        x_hat={"cups": actual.context["cups"]},
        uncertainty={"network_evidence_age": age})
```

The constraint profile and the projection service are where the modeling judgment of Section 8.4 becomes code. The capture hold is a rule; the unremedied buyer is a consequence.

Listing 8.2 — Constraint facts and the projection service

```
def constraint_profile(self, state, policy_id, action_ids, regime, governing):
    m, acts = state.metadata, set(action_ids)
    card = bool({"sell", "refund"} & acts)
    return PolicyConstraintProfile(policy_id, (
        ConstraintObservation("card_network", card and m["network"] != "up"),
        ConstraintObservation("reconciliation_hold",
            "sell" in acts and m["unreconciled"]),
        ConstraintObservation("refund_basis",
            "refund" in acts and m["refunds_owed"] == 0)))

class CoffeeProjection:
    """PolicyProjectionService: projected consequences Q_t(pi). Never selects."""
```

```

def project(self, req):
    m, acts = req.represented_state.metadata, req.action_ids
    owed = m["refunds_owed"] > 0
    if "refund" in acts:
        d, t = 12.0, 30.0 # the unserved buyer is made whole
    elif "sell" in acts:
        d = t = 30.0 if m["cups"] > 0 else 0.5
        if owed: t = 0.5 # an unremedied buyer stays unremedied
    else:
        d, t = 12.0, (0.5 if owed else 30.0)
    corridors = tuple(
        RecoveryCorridorProjection(k, f, h > 1, h > 1, h > 1,
                                   1.0 if h > 1 else None, None if h > 1 else h)
        for k, f, h in (("D", "delivery", d), ("T", "transaction", t)))
    horizons = {"D": d, "T": t}
    return PolicyProjectionRecord(
        req.policy_id, True, req.represented_state, horizons, corridors,
        projection_horizon=req.projection_horizon,
        right_censored_domain_ids=tuple(k for k, h in horizons.items()
                                         if h >= req.projection_horizon),
        state_id=req.state_id, state_time=req.represented_state.time,
        candidate_mode=req.candidate_mode, model_version="coffee-q1",
        projection_input_trace=dict(req.projection_input_trace))

```

The Layer-1 transition updates the store from the realized-effect bundles and the execution information carried by the handoff:

Listing 8.3 — The host-owned store transition

```

class StoreLayer1:
    """Layer-1 transition T: the next actual state from realized-effect evidence."""
    def __init__(self, store): self.store = store

    def transition(self, current, handoff):
        s = dict(self.store.s)
        for e in handoff.execution_information: # info_exec from the episode
            if "network_heartbeat" in e:
                s["network"] = {"status": e["network_heartbeat"],
                                "as_of": s["time"] + 1.0}
        for b in handoff.realized_pv_bundles: # B_real from host evidence
            s["cups"] -= b.get("cups_dispensed", 0)
            s["revenue"] += b.get("revenue_delta", 0.0)
            s["refunds_owed"] += b.get("refunds_owed_delta", 0)
            s["unreconciled"] = s["unreconciled"] + b.get("unknown_captures", [])
        keys = ("cups", "revenue", "refunds_owed", "unreconciled", "network")
        self.store.advance(f"{handoff.selected_policy_id}: {handoff.execution_status}",
                           dt=1.0, **{k: s[k] for k in keys})
        nxt = self.store.envelope()
        return Layer1TransitionResult(
            handoff.episode_id, handoff.selected_policy_id, current.state_id, nxt, True,
            {"cups_nonnegative": nxt.context["cups"] >= 0,
             "refunds_nonnegative": nxt.avs["refunds_owed"] >= 0})

```

Intake records outside reports and names the facts they changed:

Listing 8.4 — Observation intake

```

def intake(store, report):
    """Record an outside report as evidence; return the facts it changed.

    Intake decides nothing. It updates host-owned actual state and names the
    changed facts so that registered dependencies can be checked.
    """
    kind, s = report["kind"], store.s
    if kind == "network":
        changed = report["status"] != s["network"]["status"]
        store.advance(f"network report: {report['status']}",
                     network={"status": report["status"], "as_of": s["time"]})
        return (GovernanceDependencyChange(
            f"chg-{s['version']}", "payment_network", ("network",),
            f"network reported {report['status']}"),) if changed else ()
    if kind == "settlement":

```

```

done = [e for e in s["unreconciled"] if e in report["results"]]
captured = sum(report["results"][e] == "captured" for e in done)
store.advance(f"settlement: {len(done)} reconciled",
              unreconciled=[e for e in s["unreconciled"] if e not in done],
              revenue=s["revenue"] + captured * PRICE)
return (GovernanceDependencyChange(
        f"chg-{s['version']}", "reconciliation", ("captures",),
        "captures reconciled"),)
if kind == "clock":
    store.advance(f"{report['periods']} idle periods", dt=float(report["periods"]))
    return ()
raise ValueError(f"unknown report kind: {kind}")

```

The episode is the Chapter 7 lineage with three additions: the cycle is evaluated from actual state through the runtime's integrated entry point, the decision's dependencies are registered, and intake is checked immediately before the call.

Listing 8.5 — One governed episode

```

def episode(self, arriving=()):
    """One governed episode: perceive, decide, authorize, act, record.

    `arriving` simulates reports that reach the intake queue while
    authority is outstanding, between authorization and the call.
    """
    self.drain()
    self.n += 1; cycle_id = f"cycle-{self.n}"
    actual = self.store.envelope()
    prior = self.provenance
    if prior is not None and prior.next_state_id != actual.state_id:
        prior = None
    req = request()
    cycle = self.rt.evaluate_integrated_canonical_cycle(
        actual, req, prior.transition_provenance=prior).decision
    license = self.rt.build_execution_license_from_cycle(cycle, req.domain_frame)
    episode = self.rt.instantiate_execution(
        f"ep-{self.n}", license, entry_sufficient=True, max_steps=1).episode
    auth = self.rt.issue_native_execution_authorization(
        episode, license.action_ids[0], self.bindings, decision_cycle_id=cycle_id)

    # What this decision relied on, so a later report can invalidate it.
    deps = tuple(GovernanceArtifactDependency(
        f"{cycle_id}-{kind}", f"{cycle_id}:constraints",
        "Constraints", kind, facts)
        for kind, facts in (("payment_network", ("network",)),
                           ("reconciliation", ("captures",))))
    for r in arriving: self.report(r)
    changes = self.drain() # intake immediately before the call
    if changes:
        inv = derive_dependency_invalidations(deps, changes)
        if inv.reentry.return_to_governance:
            self.rt.invalidate_native_execution_authorizations(
                decision_cycle_id=cycle_id,
                reason="; ".join(c.reason for c in changes))
            return {"selected": license.selected_policy_id, "why": why(cycle),
                    "withdrawn": inv.reentry.recompute_from_stage}

    before = self.machine.sensors()
    native = self.rt.invoke_authorized_native(auth, self.bindings)
    after = self.machine.sensors() # host evidence, not the return value
    bundle = {"cups_dispensed": before["cups"] - after["cups"],
              "revenue_delta": after["revenue"] - before["revenue"],
              "refunds_owed_delta": after["unserved"] - before["unserved"],
              "unknown_captures": [e for e, s in after["captures"].items()
                                   if s == "unknown" and e not in before["captures"]]}
    realized = bundle if any(bundle.values()) else {}
    heartbeat = self.machine.monitor()
    info = {"network_heartbeat": heartbeat} if heartbeat else {}
    ctx = ExecutionContext(native.execution_id, auth.action_id, auth.binding_id,
                          auth.decision_cycle_id, auth.attempt)
    step = self.rt.advance_execution(episode, self.bindings.epsilon_observation(
        ctx, native, realized_pv_bundle=realized, information=info))

```

```

handoff = self.rt.build_layer1_transition_handoff(step)
transition = self.rt.apply_layer1_transition(actual, handoff)
validation = self.rt.validate_layer1_transition_result(
    actual, handoff, transition)
self.provenance = self.rt.build_execution_transition_provenance(
    actual, handoff, transition, validation)
return {"selected": license.selected_policy_id, "why": why(cycle),
        "mode": license.selection_mode, "native": native.status,
        "epsilon": step.status, "provenance_checked": prior is not None}

```

Running the file plays one day at the machine. For each episode the output shows the policy selected, how the attempt ended, what set aside each other candidate, and the actual state afterward:

Listing 8.6 — Program output

1 buyer	sell succeeded -> completed set aside: decline: Sigma refund: refund_basis cups=11 revenue=2.50 unreconciled=0 refunds_owed=0
2 buyer, capture lost	sell indeterminate -> partial_realization set aside: decline: Sigma refund: refund_basis cups=10 revenue=2.50 unreconciled=1 refunds_owed=0
3 buyer, capture open	decline succeeded -> completed set aside: sell: reconciliation_hold refund: refund_basis cups=10 revenue=2.50 unreconciled=1 refunds_owed=0
4 buyer, after settle	sell succeeded -> completed set aside: decline: Sigma refund: refund_basis cups=9 revenue=7.50 unreconciled=0 refunds_owed=0
5 buyer, no cup	sell completed_invalid -> partial_realization set aside: decline: Sigma refund: refund_basis cups=8 revenue=10.00 unreconciled=0 refunds_owed=1
6 next episode	refund succeeded -> completed set aside: sell: inadequate decline: inadequate cups=8 revenue=7.50 unreconciled=0 refunds_owed=0
7 buyer, network fails	sell authority withdrawn (re-enter at Constraints) set aside: decline: Sigma refund: refund_basis cups=8 revenue=7.50 unreconciled=0 refunds_owed=0
7' buyer, re-decided	decline succeeded -> completed set aside: sell: card_network refund: card_network, refund_basis cups=8 revenue=7.50 unreconciled=0 refunds_owed=0
8 buyer, network back	sell succeeded -> completed set aside: decline: Sigma refund: refund_basis cups=7 revenue=10.00 unreconciled=0 refunds_owed=0
9 buyer, monitor silent	decline succeeded -> completed set aside: sell: card_network refund: card_network, refund_basis cups=7 revenue=10.00 unreconciled=0 refunds_owed=0

8.11 Reading the Day

Table 8.2 — What each episode shows

Episode	Decided by	What it shows
1 Buyer	Σ Stage 1	Ordinary sale. The refund is removed by Constraints because nothing is owed.
2 Capture lost	Σ , then ϵ	The attempt is indeterminate. The unknown capture is carried into actual state.
3 Capture open	Constraints	The capture hold removes the sale. The buyer is declined, not risked.
4 After settlement	Σ Stage 1	Intake recorded the settlement; the hold lifted; revenue was corrected. No decision was changed retroactively.

Episode	Decided by	What it shows
5 No cup	Σ , then ϵ	Completed but invalid: a charged, unserved buyer is recorded as a refund owed.
6 Next episode	Adequacy	Selling and declining both leave the buyer unremedied. The refund is the only adequate policy and is executed through the same lineage as a sale.
7 Network fails	Dependency match	The outage report arrives after authorization. The runtime matches it to the Constraints dependency; authority is withdrawn before the call.
7' Re-decided	Constraints	A new cycle from the updated state removes card actions.
8 Network back	Σ Stage 1	A fresh report restores current evidence.
9 Monitor silent	Constraints	Three periods pass without a report. The evidence is stale, perception reports the network as unknown, and card actions are removed.

Two episodes deserve a second look. In episode 6, no code in the application says “issue a refund when one is owed.” The host recorded a fact, projected consequences honestly, and Adequacy did the rest. In episode 9, the network may well be working. The application declines anyway because it cannot currently know, and a card action requires a network known to be reachable. That is the Chapter 4 rule in operation: confidence in a past report does not make it current evidence.

8.12 Scalar Shortcuts and the Certificate

Applications are full of numbers that look like decision rules: a revenue-per-sale figure, a priority score, a cost ranking. Chapter 1 described the proved boundary. On a narrow structural class, a plain average recovers exactly the same maximal set as the Productive Value–Productive Power (PV-PP) framework's staged selection; outside it, a single score can agree with the output while misrepresenting the process, or can disagree outright. An application may use a score as a shortcut only when it can show, for the specific episode, that the episode belongs to that class.

The shortcut is a compression of selection among already-governed candidates. It never replaces the Graph, Π completeness, Constraints, Framing, or Adequacy, and a certified score confers no execution authority; authority still comes from the execution lineage of Chapter 7.

The published containment theorem defines the class by conditions that are all checkable from the PV-PP framework's own objects. The central ones, in plain terms: the candidates' represented consequences lie on a single line (collinearity); moving along that line never worsens any governing domain and improves at least one (governing-domain monotonicity); some candidate reaches the best point on the line (attainment); and, for the unweighted average, the line's direction averaged across all represented coordinates—governing and non-governing alike—is positive (full-vector alignment). The remaining conditions—comparison-basis integrity, no side-specific enrichment, an earned order, and compression rather than origination—ensure that the score and Σ compare the same things and that the score compresses the order rather than creating it. Chapter 11 states the result exactly.

Table 8.3 — What a scalar-shortcut certificate must record

Field	Why it matters
Candidate set identity	A score evaluated on a different pool than Σ certifies nothing.
Governing set and state time	The class is defined relative to the current governing domains.
Consequence representation	The score and Σ must compare the same objects, horizons, and coordinates.
Comparator and parameters	The certificate covers one comparator, not scores in general.
Class conditions checked	Collinearity, governing monotonicity, attainment, and alignment, with results.
Source of the frontier	Σ computes the frontier; the score may only verify or compress it.
Tolerance rule	Equality and strict-inequality tests must be stated, not implied.
Revocation conditions	Any change to the governing set, candidate set, consequence representation, extraction pipeline, tolerance rule, or admission-relevant state.
Decision mode	Shortcut, verification, diagnostic only, gating required, or block and escalate.

A certificate is episode-local. It expires when anything it relied on changes, and a shortcut reused after such a change must be re-certified. A score that cannot be certified is still useful: for monitoring, triage, display, and explanation. It must not route, release, or authorize action. The coffee application uses no scalar shortcut. Its selections turn on constraints and adequacy, which no score on the adequate set could replace.

8.13 The AI-Assisted Development Workflow

Section 1.12 recommended building PV-PP applications with AI assistance and set out the division of labor. The workflow in practice runs in nine steps, each with a human checkpoint, and the Appendix A prompts are its working tools:

- Domain description. The modeler describes the system in ordinary language. Prompt A.1 checks whether the PV-PP framework is the right tool.
- Assisted scenario discovery. The assistant interviews the modeler to find the scenarios that matter (A.3).
- Proposed construct classification. The assistant proposes how facts map to PV-PP constructs, with reasons and confidence (A.2 sketches the map; A.3 details it).
- Human and domain review. The modeler or domain expert accepts, corrects, or rejects each classification. This step is not delegable.
- Definition packets. The reviewed model is written down: scenario and model (A.3), information and evidence (A.4), change and reachability (A.5), decision cycle (A.6), execution and realization (A.7).
- Implementation mapping. The packets are mapped onto runtime structures as in Table 8.1 (A.8).

- Generated tests. The assistant writes tests for each expected decision and each outcome class, as in the accompanying test files.
- Invariant and hostile-case review. Humans and assistants look for places where host code decides, evidence is laundered, or a failure is repaired downstream. Chapter 9 develops this step.
- Runtime integration. Only then is the application run against the frozen runtime and its results trusted.

AI is fast at steps 2, 3, 6, and 7 and useful at step 8. It is also prone to the category errors this Guide catalogs, and it will produce confident, well-formatted code that encodes them. Step 4 is where meaning is decided, and it stays with people.

8.14 A Second Reference: The Runtime's Small Economy Benchmark

The coffee application exercises perception, execution, reconciliation, and dependency invalidation. It does not exercise objectives. For those, the frozen runtime ships a compact reference: the Small Economy V2.1 objective-perturbation benchmark. It holds a small household-workshop economy fixed and varies only the objectives—an attainable objective, conflicting objectives, an objective at odds with viability, an impossible objective, a capability-development objective, an objective whose lifecycle changes, prohibited evidence, and no objective at all.

Its assertions are the objective rules of Chapter 6 in executable form. Objectives never change which transformations the Graph admits or which candidates are constructed; conflicting objectives coexist without weighting; supporting an objective is not adequacy; a desired capability represented as unreachable stays excluded; a currently reachable training path supports capability development without making the future skill current; a lifecycle change re-enters only at the stage that registered a dependency on the objective; and relevant evidence prohibited for use does not become current positive evidence. Read it alongside the coffee application when a model needs `O_i(t)`.

8.15 Development History Is Not a Pattern

Much of the PV-PP framework's early simulation work implemented governance logic directly inside application code: a simulation computed its own horizons, filtered its own candidates, and chose by its own rules. That work was essential to developing the theory, and its results remain part of the record. It is not a pattern to copy. An application that claims PV-PP runtime behavior must obtain that behavior by calling the runtime. Logic reimplemented in host code is untested against the runtime's invariants, drifts from the specification as either changes, and cannot be distinguished by a reader from governance the runtime actually performed.

The same applies to workarounds that reach into the runtime. The corridor-expiry shim of Section 7.18 solved a real problem, but by writing into runtime-private state it stepped outside the interface the runtime validates. Where the interface has a gap, record the gap as a limitation and keep the workaround visible.

8.16 The Same Pattern in Other Domains

The structure of the coffee application carries over. A household application keeps its accounts and the car's condition in the store, forms perception from statements and a mechanic's report whose age matters, takes in bank notices through intake, and remedies a missed payment through a governed action rather than an automatic transfer. An organization's application holds

commitments and cash in actual state, registers that a contract decision depended on a credit line, and withdraws authority when the line is cut. An engineered system perceives from telemetry with explicit freshness and holds a commanded-but-unconfirmed actuator state as unknown until confirmed. An autonomous agent registers each tool as an execution binding, records every tool result as evidence rather than truth, and withdraws authority when a permission it relied on changes.

8.17 Common Errors in Building an Application

- Writing selection, adequacy, feasibility, or ordering logic in host code instead of calling the runtime.
- Tuning a projection or a constraint observation to steer the outcome.
- Encoding a prohibition as a bad horizon, or a consequence as a prohibition.
- Copying actual state into the decision instead of forming $P_i(t)$, or treating stale evidence as current.
- Updating the store anywhere other than the Layer-1 transition and recorded intake.
- Letting intake or an enforcement system decide, relax, or authorize anything.
- Guessing which decisions a change invalidates instead of registering dependencies when the decision is made.
- Reusing a stage result after a change that invalidated it.
- Issuing remedies such as refunds from host code instead of through a governed decision.
- Dropping unknown outcomes from actual state before they are reconciled.
- Using an uncertified score to route, release, or authorize action.
- Adding traceability after the fact instead of recording it from the first version.
- Accepting AI-generated classifications or code without human and domain review.

8.18 From an Application to Evidence

The coffee application now runs a day of ordinary and failing episodes and ends each one in a state it can explain. Its tests show that it behaves as described. They do not show that the architecture is right, that the model is faithful, or that the PV-PP framework outperforms alternatives. Those are questions about evidence, and they have their own discipline: what a test or benchmark can establish, how hostile cases are built, and how framework claims are kept apart from runtime claims. Chapter 9 takes them up.

Chapter 9 — Testing and Benchmarking

The coffee application passes its tests. That fact is worth something, and it is easy to claim too much for it. A passing test shows that an implementation behaves as expected under the conditions the test sets. It does not show that the model is faithful, that the architecture is right, or that the Productive Value–Productive Power (PV-PP) framework outperforms other approaches. This chapter is about the gap between those claims and the discipline that keeps them apart.

It combines two bodies of guidance that were written separately—benchmark design and benchmark programming—into one testing method. The first half covers the kinds of test and what each can establish: tests of category boundaries, invariants, hostile cases, traces, false success, scenario benchmarks, perturbation, reproducibility, regression, and replay. Section 9.12 applies them to the coffee application. The second half reads the completed benchmark programs as case material: what they tested, the practices that made their results credible, what they show, and what they do not.

A test establishes what an implementation does under stated conditions. It never establishes more.

9.1 Kinds of Test and What Each Establishes

Different tests answer different questions. Mixing them up is the most common way evidence gets overstated: a regression suite cited as validation, a scenario benchmark cited as proof of superiority, a unit test cited as evidence that the model is right.

Table 9.1 — Kinds of test

Kind	Question it answers	What it does not establish
Unit	Does this adapter, transition, or domain function do what it should?	That the pieces govern correctly together.
Runtime contract	Does the runtime surface the application depends on behave as documented, including on malformed and hostile input?	That the application uses it correctly.
Invariant	Does a sovereignty boundary hold across every case tested?	That it holds in cases not tested.
Hostile case	Is a deliberate violation impossible, or at least visible?	That every violation has been imagined.
Scenario benchmark	Under a declared model and perturbation, does the predeclared behavior occur?	General validity outside the model.
Regression	Has behavior that was banked changed?	That the banked behavior was right.
Replay	Can a past decision be reconstructed from what was recorded at the time?	That the decision was good.

Kind	Question it answers	What it does not establish
Cross-domain study	Does the architecture preserve its distinctions across problems designed by others?	Superiority over alternative architectures.

9.2 Testing Category Boundaries

The PV-PP framework's value lies in distinctions a simpler implementation would collapse. Each distinction is a boundary, and each boundary can be tested by constructing a case in which the two sides come apart and checking that the implementation keeps them apart.

Table 9.2 — Boundaries and how to test them

Boundary	Collapse it prevents	Test by constructing
PP vs resources	Counting stock as capability	A resource present while the capability it enables is not
Reachability vs authorization	Treating a reachable path as permitted	A reachable transformation whose use Constraints prohibits
Confidence vs authority	Letting a confident source decide	High-confidence evidence prohibited for use
Fresh vs stale evidence	Treating a past report as current	The same report at increasing ages
Objective vs reachability	Letting a goal create options	An impossible objective
Selection vs execution	Treating a choice as an act	A selection whose authority is withdrawn before the call
Returned vs realized	Treating a return value as an effect	A function that returns normally and does nothing
Clean vs unknown failure	Treating any error as "nothing happened"	A function that acts and then raises
Tie order vs preference	Hiding a ranking in the Σ order	The same scenario with the order reversed

9.3 Invariant Tests

An invariant is a statement that must hold in every case: objectives never create reachability; confidence never creates authority; history never substitutes for current state; a selection never implies a realized effect; a consumed authorization is never accepted again. Invariant tests state the rule once and check it across many cases, rather than checking one expected output at a time.

The frozen runtime's own suite is built this way. Its 1,093 tests assert, among many others, that a license cannot be minted by hand, that two concurrent calls with one authorization produce one invocation, that an unclassified exception is treated as an unknown effect, and that objective changes invalidate only the artifacts that registered a dependency on them. An application should add invariants for its own model—in the coffee application, for example, that a refund owed is never left unremedied while a refund is possible.

9.4 Hostile Cases

A hostile case is an implementation built to break a rule. It asks not whether the system behaves well when everything is honest, but what happens when something is not: an adapter that hides a harm, a function that claims success without acting, an authorization copied field by field, a stale report presented as current. The test passes when the violation is impossible—the runtime refuses it—or visible—an invariant flags it.

Hostile cases should target the failures the architecture exists to prevent:

- Semantic laundering: a fact moved into the wrong surface so that a stage cannot see it—a prohibition encoded as a poor horizon, a harm dropped from a projection, a resource relabeled as capability.
- Stale authority: acting on a license, authorization, or evidence after the facts behind it changed.
- Forged authority: authority reconstructed rather than issued.
- Incomplete state: a material consequence left out of actual state so that the next decision cannot see it.
- Missing recovery: a model whose Graph lacks the exit or repair path the regime requires.
- Hollow success: a visible success with no realized effect.
- Downstream repair: a later stage adjusted to compensate for an earlier stage's defect.

A hostile implementation is not a weak comparator. Its purpose is to test whether a violation would be caught, not to make the PV-PP framework look better than an alternative.

9.5 Traces That Reveal the Earliest Failure Surface

When something goes wrong, the trace should say where. A single success flag cannot: a decision can fail because a candidate never existed, was removed, was inadequate, lost at Σ , was refused authority, failed technically, or failed in the world. Each of those points to a different fix, and Chapter 6's rule—repair at the stage that owns the defect—can be followed only if the trace names the stage.

Table 9.3 — Failure classes a trace must keep apart

Class	Meaning
Model or setup error	The specification is invalid or required state is missing.
Runtime validation refusal	The runtime rejected malformed or semantically invalid input.
Governance refusal	A candidate, piece of evidence, or action was not admissible or authorized.
Selection absence	No authorized selection exists on the represented decision surface.
Execution authorization failure	The selected policy could not proceed through ε .
Callable or technical failure	The registered function raised or returned an error.
Environmental realization failure	The call occurred, but the intended effect did not.
State-transition failure	The result could not be validly reconciled into the next actual state.

Class	Meaning
Replay insufficiency	The record cannot reconstruct the decision as claimed.
Benchmark assertion failure	Observed behavior violated a predeclared invariant.

The coffee application's output line "set aside" is a small instance of this: for every candidate not selected, it names the constraint, the adequacy failure, or Σ .

9.6 False Success

False success is the benchmark classification for a visible or local success that coexists with a failure that matters: a transaction recorded while delivery did not occur, high reward obtained while recovery access was destroyed, a function returning success while the intended effect did not happen, an objective met while a governing threshold was crossed.

Detecting it requires declaring both conditions before the experiment—the ordinary success criterion and the governing failure criterion—so that their coexistence can be flagged rather than explained away afterward. A benchmark that declares only the success criterion cannot detect false success at all.

9.7 Scenario Benchmarks

A scenario benchmark is a controlled model built to test a bounded claim. It begins with a valid scenario model (Chapter 3), not with a reward function carrying PV-PP labels, and it states its claim in a sentence that the experiment could support, contradict, or leave unresolved. "The PV-PP framework works" is not such a claim; "an impossible objective does not create a reachable transformation" is.

The design separates four kinds of variable: the fixed model structure, including the framework and runtime version; the controlled perturbations; the policy or baseline variation; and the observed outcomes, with ordinary task metrics kept apart from PV-PP observables such as threshold violations, reachability changes, adequacy, and false success. Acceptance criteria are written before the run and state an invariant or conditional behavior, not merely which policy should win.

A benchmark need not use the runtime. A narrow teaching benchmark may isolate one distinction, provided it says which parts of the PV-PP framework it implements and which it omits. A benchmark that claims canonical runtime behavior must obtain that behavior from the frozen runtime. The preferred pattern is often hybrid: environment dynamics in benchmark code, the governance surfaces under test delegated to the runtime.

Comparators should be strong. A benchmark may include an ordinary task baseline, a hard-constraint or safety-filter baseline, a scalar baseline where scalarization is a legitimate contender, and ablations that remove one PV-PP distinction at a time. If a strong conventional method reproduces the same decision or failure detection, that is an informative result, not an embarrassment.

9.8 Perturbation and Metamorphic Tests

A perturbation test changes one declared variable and checks that only the authorized surfaces respond. The runtime's Small Economy V2.1 benchmark (Section 8.14) is the reference pattern: it varies only the objectives and asserts that the Graph and the candidate set do not move. Changing several variables at once cannot isolate the mechanism the test claims to examine.

A metamorphic test checks a relation between runs rather than an expected output. If the Σ order is only a tie-breaker, reversing it must not change any selection in a scenario with no exact ties. If a variable is irrelevant to governance, changing it must not change the decision. Such tests are useful precisely when the “right answer” is hard to state but the invariance is not.

9.9 Reproducibility

A result that cannot be reproduced cannot be checked. Every run should bind its benchmark and code version, the framework and runtime versions and configuration, the runtime's identity by file hash, random seeds and trial counts where anything is stochastic, the initial actual state and any perceived or objective state, the perturbation, the policy or baseline, the expected invariants, and the destination of the trace. A publishable benchmark packages these together with the scenario and design specifications, the test suite and its expected result, the commands that reproduce the run, the raw or reconstructable outputs, and a statement of limitations.

9.10 Regression Discipline

Once behavior is stabilized, bank it: record the expected outputs, the test count, and the versions, and treat any change as a finding to be explained. The frozen runtime shows the discipline. Its Version 2.1 builds grew from 967 tests at v0.132 to 1,093 at v0.140, and the frozen v0.141 records 1,093 of 1,093 passing. Build 167 kept those tests and added its own: it records 1,320 of 1,320 passing, with warnings treated as errors, after twelve rounds of hostile review. The Version 2.1 line began by confirming that every inherited runtime and test file was byte-identical to its predecessor, the frozen tree carries a file-hash manifest, and future development is to branch from v0.141 rather than modify it in place. Build 167 did exactly that: its only change to a build-141 file is the package metadata.

Two cautions apply. A regression suite shows that behavior has not changed; it says nothing about whether the banked behavior was correct, and it is not empirical validation. And a copied test log is not a rerun: an identical result file carried forward through a series of milestones records one run, not many.

9.11 Replay Tests

A replay test asks whether a past decision can be reconstructed from what was recorded at the time: the state basis and perceived state, the objective set and version, the admitted evidence and its governance, the configuration and runtime identity, and the decision and execution artifacts. Two rules apply (Chapter 4). Later knowledge must not be inserted into the historical package because it explains the outcome. And execution completion is not replay sufficiency; a decision can have run to completion and still be unreconstructable.

9.12 Program: Testing the Coffee Application

`test_coffee_hostile.py` adds sixteen tests to the Chapter 6–8 suites, applying the methods above to the coffee application. It runs against the unmodified frozen runtime. Four groups are shown here.

The first tests the boundary between a returned call and a realized effect. A hollow machine returns a receipt and does nothing; the store records no sale, and a false-success detector declared in advance flags the episode.

Listing 9.1 — Returned is not realized

```
def false_success(out, cups_before, cups_after):
    """Visible success with no realized effect: a sale that 'succeeded' but served nobody."""
    return (out.get("native") == "succeeded" and out["selected"] == "graph:sell"
            and cups_after == cups_before)

class HollowMachine(Machine):
    def sell(self, context):
        # returns normally and does nothing
        return {"receipt": context.execution_id}

def test_returned_is_not_realized(tmp_path):
    _, s, app = fresh(tmp_path, HollowMachine())
    out = app.episode()
    assert out["native"] == "succeeded"
    assert (s.s["cups"], s.s["revenue"]) == (12, 0.0)    # state follows evidence
    assert false_success(out, 12, s.s["cups"])          # and the trace exposes it
```

The second is a hostile projection service. It launders a known harm by projecting as if no refund were owed. The invariant—a refund owed is never left unremedied—passes with the honest service and flags the laundering one. The runtime cannot catch this itself: the laundered record is well formed, so only a domain invariant reveals it.

Listing 9.2 — A hostile projection service caught by an invariant

```
class LaunderingProjection(CoffeeProjection):
    """Hides a known harm: projects as if no refund were owed."""
    def project(self, req):
        state = req.represented_state
        clean = dataclasses.replace(state, metadata=dict(state.metadata, refunds_owed=0))
        return super().project(dataclasses.replace(req, represented_state=clean))

def harm_left_unremedied(owed_before, out):
    return owed_before > 0 and out["selected"] != "graph:refund"

def owed_then_next(tmp_path, monkeypatch, projection):
    monkeypatch.setattr(app_mod, "CoffeeProjection", projection)
    m, s, app = fresh(tmp_path)
    m.faults.append("no_delivery"); app.episode()
    owed = s.s["refunds_owed"]
    return owed, app.episode()

def test_honest_projection_remedies_the_harm(tmp_path, monkeypatch):
    owed, out = owed_then_next(tmp_path, monkeypatch, CoffeeProjection)
    assert owed == 1 and not harm_left_unremedied(owed, out)

def test_laundering_projection_is_caught_by_the_invariant(tmp_path, monkeypatch):
    owed, out = owed_then_next(tmp_path, monkeypatch, LaunderingProjection)
    assert owed == 1 and harm_left_unremedied(owed, out)    # the violation is visible
```

The third applies one information-governance rule across a range of cases: network evidence counts only while it is fresh, checked at every age from zero to five periods.

Listing 9.3 — Evidence freshness across ages

```
@pytest.mark.parametrize("age", range(6))
def test_network_evidence_counts_only_while_fresh(tmp_path, age):
    _, s, app = fresh(tmp_path)
    s.advance("aged report", dt=float(age)) # report as_of stays at 0
    _, cycle, _ = selected_episode(app)
    sell_feasible = "graph:sell" in cycle.constraints.feasible_policy_ids
    assert sell_feasible == (age <= app_mod.MAX_EVIDENCE_AGE)
```

The fourth group is metamorphic, regression, and identity testing: reversing the Σ order must not change any selection in the day; the day's output must match the banked file; and the runtime's decision core must be build 141's, checked file by file against the build-141 hash manifest. On

v2.1.167 that last test still passes, because the core is unchanged. It does not check the supervision layer, which the v2.1.167 manifest covers (Appendix G).

Listing 9.4 — Metamorphic, regression, and identity tests

```
def picks(tmp_path, capsys):
    out, _ = run_day(str(tmp_path / "day.json")); capsys.readouterr()
    return [r["selected"] for _, r in out]

def test_tie_order_is_not_a_hidden_preference(tmp_path, capsys, monkeypatch):
    baseline = picks(tmp_path, capsys)
    real = app_mod.SigmaOrderDefinition
    monkeypatch.setattr(app_mod, "SigmaOrderDefinition",
                        lambda pid, order: real(pid, 100 - order)) # reverse the order
    assert picks(tmp_path, capsys) == baseline

def test_day_matches_banked_output(tmp_path, capsys):
    run_day(str(tmp_path / "day.json"))
    banked = (HERE / "coffee_day_expected.txt").read_text()
    assert capsys.readouterr().out == banked

def test_runtime_is_the_frozen_v0141_build():
    root = pathlib.Path(pvpp_runtime.__file__).resolve().parents[1]
    manifest = root / "V0141_FILE_HASHES.sha256"
    if not manifest.exists():
        pytest.skip("runtime manifest not found beside pvpp_runtime")
    for line in manifest.read_text().splitlines():
        digest, name = line.split(maxsplit=1)
        if name.startswith("pvpp_runtime/") and name.endswith(".py"):
            assert hashlib.sha256((root / name).read_bytes()).hexdigest() == digest, name
```

The results:

Listing 9.5 — Test results

```
test_returned_is_not_realized PASSED
test_ordinary_sale_is_not_flagged PASSED
test_unanticipated_error_is_indeterminate_not_clean PASSED
test_license_cannot_be_instantiated_twice PASSED
test_reconstructed_authorization_is_refused PASSED
test_network_evidence_counts_only_while_fresh[0] PASSED
test_network_evidence_counts_only_while_fresh[1] PASSED
test_network_evidence_counts_only_while_fresh[2] PASSED
test_network_evidence_counts_only_while_fresh[3] PASSED
test_network_evidence_counts_only_while_fresh[4] PASSED
test_network_evidence_counts_only_while_fresh[5] PASSED
test_honest_projection_remedies_the_harm PASSED
test_laundering_projection_is_caught_by_the_invariant PASSED
test_tie_order_is_not_a_hidden_preference PASSED
test_day_matches_banked_output PASSED
test_runtime_is_the_frozen_v0141_build PASSED
===== 16 passed in 0.32s =====
```

These tests make the coffee application harder to break silently. They do not show that the coffee model is a faithful account of a vending business, and the chapter's remaining sections apply the same caution to far larger programs.

9.13 The Evidence Record: Three Levels

The PV-PP framework's benchmark work falls into three levels, each exposing the architecture to problems less of its own making than the last.

Table 9.4 — The benchmark programs

Level and program	What was tested	Result, with its limits
Internal: Small Economy stress test (frozen v0.70)	A multi-milestone household economy run against the frozen v0.70 runtime, including five-controller comparisons over fifty periods	No theory, Σ , or runtime change required; a fixed lexicographic rule matched the PV-PP framework's mortality in the comparison; three matched seeds expose mechanisms but do not establish superiority
Internal: Small Economy V2.1 objective benchmark (v0.140; rerun at v0.141)	Eight objective perturbations against a fixed substrate	Objectives confined to authorized surfaces; a runtime conformance benchmark, not economic validation
Internal: Runtime v2.1.167 hardening (build 167)	Twelve hostile-review rounds against the supervision layer: crash, race, authority, and identity attacks	1,320 of 1,320 tests with warnings as errors; a runtime conformance record, not evidence about any application
Internal: teaching and comparison simulations	Scalar-comparison simulations, a three-path trade-off gridworld, and the Safe Margin versus Fast Gain specification	Design and development material; the archive banks no results for them, so they are not cited as evidence
Public: AI Safety Gridworlds	All 10 environments across 8 safety-problem classes	Zero canonical patches; two closed with fidelity caveats; one closed with a boundary exposed (Section 9.15)
Public: MO-Gymnasium v1.3.2	All 31 registrations, in 16 packages	No registration forced a new operator or state primitive; not a trained-agent performance comparison
Real-world: MOEA RealWorldBenchmarks	Three source-locked models: water supply, lake pollution control, aircraft family design	All passed with no canonical change; exploratory, not suite coverage

9.14 Practices That Made the Results Credible

The value of the public and real-world programs lies less in their pass counts than in how they were run. Five practices recur, and each is worth adopting.

- Freeze the architecture before execution. No operator, definition, or selection responsibility was changed during a program to rescue a failing case; a failure was to be recorded before any architecture change was considered.
- Pre-register selection and forbid substitution. MO-Gymnasium began with three environments frozen before testing, then expanded to the complete suite—decided after those results but before the rest ran—to remove the criticism that only favorable environments had been chosen. No environment could be dropped for looking unfavorable,

and none could be swapped for an easier one. The real-world cases were selected before analysis and could not be replaced because a result was unfavorable.

- Record a disposition for every case, using a fixed vocabulary. The programs classified results as scalar containment, multidimensional preservation, adequacy or constraint distinction, fallback or infeasibility, formalization gap, or architecture failure. The categories are not mutually exclusive; a case can show several.
- Keep benchmark defects apart from architecture failures. The Whisky & Gold fixture let random actions make progress but not regress; the fixture was corrected, not the PV-PP framework. The water-supply source ran its native model in one mode while declaring objectives computed in another, changing an objective for 175 of 250 candidates; the defect was recorded rather than silently patched. The lake model's time-based seeding compromised nominal Monte Carlo independence and was documented. Two gridworld environments were implemented at the mechanism level and closed with fidelity caveats. A representation that works only by changing what the original problem asks counts against the framework.
- Stop, assess, and state what is not established. Each program ended with a closeout that listed its limits before anyone generalized from it; further work is to be framed as a new, versioned study rather than a continuation.

9.15 What the Record Shows, and What It Does Not

Across the public and real-world programs, no case forced a new canonical operator or state primitive, and the architecture stayed frozen throughout. Every disposition class except architecture failure appears. Some cases are scalar-contained—Fruit Tree's outcomes were all supportable by some nonnegative linear weighting, and five of the MuJoCo tasks recover their original rewards as documented linear recombinations of their components. Others are not: in Minecart, four of ten Pareto-efficient outcomes are supported by no nonnegative weighting, and on Deep Sea Treasure's concave front only the two endpoints are. In Water Reservoir, an equal-weight scalar returns a maximizing release at every tested state even though no release satisfies every governing domain at once; the PV-PP framework reports the empty adequate set and routes to fallback. The aircraft model's total constraint violation is, for its own feasibility question, an exactly lossless scalar—a reminder that some numbers are enough.

The record also exposed one open boundary. In the hardened Friend-or-Foe case, an adversary that knows the agent's deterministic rule defeats every deterministic policy, while a fifty-fifty mixture achieves about one-half success. Whether the existing definition of a policy already permits a probability-bearing rule—selected deterministically by Σ and realized as one permitted branch by ϵ —or whether that would add something the canon does not specify, is recorded as an open formalization gap. No random-choice operator has been adopted.

What the record does not show matters as much. It does not show that the PV-PP framework is optimal, generally safe, or superior to multi-objective reinforcement learning, evolutionary optimization, operations research, or control. The MO-Gymnasium program was not a trained-agent comparison, and its linear non-recovery results concern the tested weighted-sum forms, not every nonlinear scalar. The Small Economy comparison found that a well-chosen lexicographic rule matched the PV-PP framework on mortality, and its authors drew the right conclusion: the framework should not claim that a multidimensional architecture must outperform a good fixed rule. Chapter 11 places the scalar results beside the proofs.

9.16 Framework Claims and Runtime Claims

Much of the historical evidence predates Runtime V2.1. The Small Economy stress test ran against frozen v0.70; the public and real-world programs are documented as tests of a frozen canonical architecture, not as runs against v0.141; and earlier simulations implemented PV-PP logic in their own code. That evidence bears on the framework—whether its distinctions are expressible and preserved across problems—but it is not evidence about v0.141's behavior. Claims about v0.141 rest on its own test suite and on benchmarks that call it: the Small Economy V2.1 objective benchmark and the Guide's coffee programs. Claims about v2.1.167's supervision layer rest on its own test suite and hardening record and on the Chapter 12 program, not on anything earlier.

Keep three columns in any report: what the framework claims, what a specific runtime version was shown to do, and what an application built on it was shown to do. A result in one column does not transfer to another without its own evidence.

9.17 Designing Your Own Test and Benchmark Plan

A plan for a new application can follow the order of this chapter: state the bounded claims; list the category boundaries the model depends on and a test for each; write the invariants; design hostile implementations for the failures that matter most; declare false-success conditions; define the trace fields; design any scenario benchmarks with their controls, perturbations, comparators, and acceptance criteria; fix the reproducibility package; decide what will be banked for regression; and write the limitations statement before the first result arrives. Prompt A.9 walks through the same sequence.

9.18 Common Errors in Testing and Benchmarking

- Citing a passing regression suite as validation.
- Reporting a benchmark win as proof of general superiority.
- Declaring the success criterion but not the governing failure criterion, so false success cannot be seen.
- Using a single success flag instead of distinguishing where a failure occurred.
- Designing a weak comparator, or omitting a strong one because it might match the PV-PP framework.
- Selecting environments after seeing results, or dropping unfavorable ones.
- Changing framework rules between experimental arms, or during a program to rescue a failing case.
- Classifying a fixture error or source defect as an architecture failure, or the reverse.
- Reimplementing runtime behavior in benchmark code while claiming the result tests the runtime.
- Treating a copied test log as an independent rerun.
- Inserting later knowledge into a historical replay.
- Attributing framework-level evidence to a runtime version, or v0.70 evidence to v0.141.

9.19 From Evidence to Worked Examples

The coffee model has now been modeled, decided, executed, assembled, and tested. One running example cannot show that the architecture carries across domains. Chapter 10 builds complete examples in a household, an organization, an engineered system, and an autonomous agent, with the testing discipline of this chapter applied to each.

Chapter 10 — Worked Examples

The coffee machine has carried the Guide from modeling to testing. A single running example cannot show that the architecture travels. This chapter works four more, in domains chosen to be unlike one another: a household deciding under changing information, a small organization whose capability depends on how its people and equipment are configured, an engineered cooling system, and an autonomous software agent. Each continues a case introduced earlier in the Guide, so the reader meets familiar facts carried through the whole architecture.

The examples are built at the same depth and in the same order, so they can be read side by side as a pattern library. The autonomous-agent example is also fully coded against the frozen runtime, and it exercises the runtime surfaces the coffee programs did not: reachability refresh, regime-shaped candidate families, and controlled admission of a new tool. The other three are specified at the level of the definition packets and walked through stage by stage; their numbers are illustrative, and they can be coded by following Chapters 6 through 9 with the Appendix A prompts. Section 10.6 adds a labeled teaching benchmark, and Section 10.7 sets out what carries across domains and what does not.

The questions are the same in every domain. The answers—measurements, mechanisms, and authority—belong to the domain.

10.1 How Each Example Is Built

Every example follows the same sequence, which is the sequence of the Guide itself. Table 10.1 lists the steps and where each was developed.

Table 10.1 — The sequence each example follows

Step	Question	Developed in
Boundary and entities	What system, episode, and actors are modeled?	Chapter 3
State and capability	What are $S_i(t)$, PP, SPV, and broader state?	Chapters 2–3
Perception and evidence	How is $P_i(t)$ formed, and what evidence may it use?	Chapter 4
Objectives	What does the actor want, kept adjacent to perception?	Chapters 3, 6
Change and reachability	What can change, and what makes each path reachable?	Chapter 5
Decision	What does each stage decide?	Chapter 6
Execution and realization	What is attempted, and what actually happens?	Chapter 7
Change, re-entry, and history	What invalidates what, and what is kept for replay?	Chapters 4–6, 8

10.2 Example 1: A Household Deciding Under Changing Information

Boundary. The household from Chapters 3 through 5 must preserve dependable transportation to work after its car fails. The episode runs from the first diagnosis until a transportation plan is adopted and its first commitment is realized. The rest of the household's financial life is outside the boundary except where reserves, obligations, income, and credit bear on this decision.

State and capability. Transportation PP is the actual capability to get the working member to work on time. The car is an enabling resource, not the capability; savings are SPV; the monthly obligations, the relative who can lend a car, and the transit schedule are broader state. Transportation and solvency are the governing domains. Transportation's horizon is short—days, not months—because the job depends on it.

Perception and evidence. The first mechanic reports a failed transmission and recommends replacement. The household's perceived state follows the report: the car is treated as effectively lost. In actual state the car is repairable—the fault is a failed speed sensor—but nothing the household holds yet says so. The mechanic is an authoritative source for this car within the scope of the inspection; a forum thread about the model year is background, not evidence about this car.

Objectives. Keep the job, keep an emergency reserve, avoid a payment the household cannot sustain. A wish for a newer vehicle is recorded as an objective too. It may direct attention toward financing routes; it makes none of them reachable and none of them adequate.

Change and reachability. Repair is represented but, on the first diagnosis, unreachable at a price the reserve can bear. Replacement is reachable only if financing is approved and a suitable car is available. The bridge—transit plus the relative's car on two days a week—is reachable only while the relative agrees. The staged path that matters is bridge, then replace.

Decision. With transportation's horizon short, the regime moves out of Mission, and substitution paths become active candidates. A lender approves a loan, but approval is a fact about the lender, not about the household: the payment would leave solvency failing within the horizon, so Adequacy rejects the loan-financed replacement even though Constraints admits it. Selling the car for cash and buying an older one outright fails Adequacy for transportation, because nothing bridges the gap between sale and purchase unless the bridge is part of the policy. The selected policy is the staged one: bridge now, buy a modest replacement with cash in two weeks.

Execution and realization. Arranging the bridge is an interaction with its own outcome: the relative agrees for two weeks, not indefinitely. That realized fact, not the plan, enters actual state.

Change, re-entry, and history. Four days later a second mechanic finds the speed sensor. The corrected diagnosis supersedes the first for the current decision, perception changes, and governance re-enters at the start of the cycle. Repair is now reachable and cheap, adequate on both domains, and it beats the cash replacement on solvency, so it is selected; the remaining bridge days lapse unused. The earlier decision is not rewritten. Its replay package keeps the first diagnosis as the evidence admitted at the time, because that is what the household knew. A faithful record shows a reasonable decision made on evidence later corrected, not a mistake.

Table 10.2 — What decided the household's choices

Stage	Before the correction	After the correction
Perception	Transmission failed; car effectively lost	Sensor failed; car repairable

Stage	Before the correction	After the correction
Regime	Elevated: substitution families active	Elevated until repair completes
Constraints	Admits loan-financed replacement (approval obtained)	Admits repair
Adequacy	Rejects the loan (solvency fails) and an unbridged sale (transportation fails)	Repair adequate on both domains
Σ	Selects the staged bridge-then-buy policy	Selects repair over cash replacement
Replay	Keeps the first diagnosis as admitted evidence	Does not reach back to change the first decision

What this example exercises: actual-versus-perceived divergence, source authority, supersession without rewriting history, an objective that directs attention without creating options, and the difference between a counterparty's approval and the actor's own adequacy.

10.3 Example 2: An Organization Whose Capability Depends on Configuration

Boundary. A small commercial bakery supplies twelve cafés every morning. The modeled episode is one week in which its main oven fails and a large new customer offers a contract. The bakery qualifies as a collective productive entity in the sense of Chapter 3: it holds obligations, equipment, accounts, and decisions of its own. Its capability is not the sum of its members' capabilities.

State and capability. Production PP exists only as a configuration: a working oven, a certified baker on shift, flour on hand, and the van. Remove any one and the capability is gone, however capable the rest remain. The governing domains are liquidity and staffed capacity—the hours of certified labor available each night. Customer goodwill is broader state that matters but does not govern this week's decision.

Perception and evidence. The oven service company reports the part on back-order for ten days; its report is authoritative about the part, not about whether the bakery can meet its orders. The rota shows which bakers can take extra shifts; a baker's verbal "probably" is recorded with its uncertainty.

Change and reachability. Paths include renting night oven time at a restaurant across town, which is reachable only if a certified baker can staff it there; cutting the smallest cafés' orders for the week; buying finished goods from a wholesaler; and accepting or declining the new contract. The rented-oven path is a configuration change: the capability is rebuilt from a different oven with the same people.

Decision. One recovery obligation is already active: last Monday's missed delivery must be made good by Friday, and it needs two certified night shifts. The runtime first checks that active obligations can still be carried out against the shared shift calendar (Section 6.10); they can, because two certified shifts remain. Whether a particular candidate keeps them feasible is a projection fact, judged by Adequacy. Accepting the new contract now would consume three more shifts; its projection leaves the Friday obligation unmet and staffed capacity failing, so Adequacy rejects it. The contract is profitable; profit does not appear in the question. The

selected policy rents oven time for the obligated shifts, trims two small cafés' orders with notice, and defers the contract by a week—an option the customer's own terms leave reachable.

Execution and realization. The rented oven is booked; the first night's bake is realized and delivered. The second night the restaurant's oven is unavailable—an interaction outcome that re-enters governance with one obligated shift still due.

Change, re-entry, and history. With one night lost, the recovery corridor for Friday is still open but narrower. The next cycle finds the wholesaler path adequate for the remaining obligation, and the bakery takes it at a loss. The history records the missed delivery, the recovery, and its cost; none of it becomes current capability, and none of it silently changes the bakery's standing with its customers except through recorded facts.

Table 10.3 — What decided the bakery's week

Stage	What it decided
Entity qualification	The bakery, not its members, is the deciding entity; member capabilities are not summed
Graph	The rented-oven path exists only as a configuration change with a certified baker
Joint recovery feasibility	Confirms the active Friday obligation can still be met from the shared shift calendar
Adequacy	Rejects accepting the contract now, whose projection leaves the obligation unmet; accepts renting plus trimmed orders; later accepts the wholesaler path for the lost night
ε and Layer 1	Records the lost second night as a realized fact that reopens governance

What this example exercises: configuration-dependent PP, collective-entity qualification, an active recovery obligation competing for shared capacity, and a profitable option removed for a reason no revenue figure can express.

10.4 Example 3: An Engineered Cooling System

Boundary. A plant keeps a server room below its temperature limit with two chillers, one normally running and one standby. The modeled episode begins when the running chiller trips and ends when cooling is stable or the room has been protected by shedding load. The physics—heat load, thermal mass, refrigerant cycles—belongs to the engineering disciplines; the Productive Value–Productive Power (PV-PP) framework supplies the governance structure around it.

State and capability. The preserved function is cooling of the server room. Chiller 1 and chiller 2 are units that can provide it; so can a rented portable unit or reduced heat load. Domain Framing holds the target at the function, which is what lets a substitute count. The governing domains are room temperature margin and equipment integrity; the horizon is the time until the room reaches its limit, measured in minutes.

Perception and evidence. Perception comes from telemetry, and telemetry has age and confidence. A temperature reading older than its sampling interval is not current. A commanded valve position is not a measured one: until the flow sensor confirms it, the valve's state is unknown. The trip alarm is authoritative that chiller 1 stopped; it is not evidence of why.

Change and reachability. Restarting chiller 1 is represented and reachable. Starting chiller 2 is represented, but its reachability is uncertain: it was under maintenance last week and the clearance record has not been confirmed. Shedding non-critical servers is reachable. Emergency shutdown of the whole room is reachable and always available as an escape.

Decision. With the horizon at minutes, the regime is elevated, and emergency-escape families are active candidates—in the runtime they become eligible only in Survival and Existential regimes. Restarting chiller 1 without knowing the trip cause is executable, but its projection shows a likely repeat trip within the horizon, and Adequacy rejects it. Starting chiller 2 would be adequate, but its uncertain reachability is resolved by a maintenance supervisor's confirmation within two minutes; until then the governed choice is to shed non-critical load, which is adequate for the time it buys. When the clearance arrives, reachability is refreshed, the Graph is re-entered, and starting chiller 2 is selected.

Execution and realization. The start command is issued. Until flow and supply temperature confirm that chiller 2 is carrying the load, the outcome is indeterminate, and the room is treated as still at risk. The high-temperature interlock is an enforcement mechanism: it will trip servers off on its own if the limit is reached. It is not a second decision system, and its action would enter governance as a fact.

Change, re-entry, and history. Once measured flow confirms cooling, actual state records chiller 2 carrying the load and chiller 1 out of service pending diagnosis. The shed servers are restored by a later governed decision, not automatically.

Table 10.4 — What decided the cooling response

Stage	What it decided
Domain Framing	Target is cooling of the room, so a substitute unit or reduced load can preserve it
Regime	Minutes-long horizon: emergency-escape families become candidates
Graph	Chiller 2 uncertain until clearance is confirmed; then refreshed to reachable
Adequacy	Rejects restarting chiller 1 blind; accepts shedding load as a bridge
ε and Layer 1	Treats the start as indeterminate until measured flow confirms it

What this example exercises: function-level framing, evidence freshness, commanded-versus-measured state, uncertain reachability resolved by evidence, regime-driven family activation, and an interlock kept on the enforcement side of the boundary.

10.5 Example 4: An Autonomous Operations Agent

Boundary. A software agent keeps a report service available. It can restart the service, patch it in place, fail over to a replica, or escalate to the on-call human. The episode is one shift. The agent's tools act on real infrastructure, so every action runs through the governed path of Chapter 7; keeping that path the only path is the job of the surrounding platform's permissions.

State and capability. The governing domains are availability—users can reach the service—and control—a human override and a rollback point remain available. Control is what makes this an agent-governance problem: an action that restores availability by removing the rollback point trades a recoverable outage for an unrecoverable configuration.

Perception and evidence. The monitor reports service health; a diagnosis—memory leak or bad configuration—comes from logs. The write credential and the change-freeze calendar are facts the agent perceives, not permissions it holds by default.

Change and reachability. The replica route is represented from the start but unreachable: the network path has not been confirmed. Later, a scanner confirms it. Separately, the agent is given a new tool that corrects configuration faults. The two events are handled differently. The route is already represented, so the scanner's report refreshes its reachability. The tool is new, so it must be admitted—a new function together with its first transformation, with source, provenance, and rationale—before it can even be bound.

agent_ops.py codes this example against the frozen runtime. The adapter reports rule violations; a projection service reports projected consequences. Patching in place restores availability fastest but destroys the rollback point, so its projection leaves control failing; the freeze and the credential are rules.

Listing 10.1 — The agent's constraint facts and projection service

```
def constraint_profile(self, state, policy_id, action_ids, regime, governing):
    m, acts = state.metadata, set(action_ids)
    return PolicyConstraintProfile(policy_id, (
        ConstraintObservation("write_credential",
                               bool(acts & WRITE_ACTIONS) and not m["credential"]),
        ConstraintObservation("change_freeze",
                               bool(acts & CHANGE_ACTIONS) and m["freeze"])))

class AgentProjection:
    """PolicyProjectionService: projected consequences Q_t(pi). Never selects."""

    def project(self, req):
        diag = req.represented_state.metadata["diagnosis"]
        a, c = {
            "restart": (20.0 if diag == "memory leak" else 0.5, 30.0),
            "hotfix": (30.0, 0.5), # fast, but destroys the rollback point
            "failover": (25.0, 30.0),
            "escalate": (12.0, 30.0), # a human restores service, slowly
            "reconfigure": (30.0 if diag == "bad config" else 0.5, 30.0),
        }[req.action_ids[0]]
        corridors = tuple(
            RecoveryCorridorProjection(k, f, h > 1, h > 1, h > 1,
                                       1.0 if h > 1 else None, None if h > 1 else h)
            for k, f, h in (("A", "availability", a), ("C", "control", c)))
        horizons = {"A": a, "C": c}
        return PolicyProjectionRecord(
            req.policy_id, True, req.represented_state, horizons, corridors,
            projection_horizon=req.projection_horizon,
            right_censored_domain_ids=tuple(k for k, h in horizons.items()
                                             if h >= req.projection_horizon),
            state_id=req.state_id, state_time=req.represented_state.time,
            candidate_mode=req.candidate_mode, model_version="ops-q1",
            projection_input_trace=dict(req.projection_input_trace))
```

The two change surfaces use the runtime's own operations. A reachability refresh changes only an already-represented transformation and returns a plan to re-enter at the Graph stage. An admission adds the new action and its first transformation atomically and returns the same kind of plan. Neither grants any authority.

Listing 10.2 — Reachability refresh and controlled admission

```

def scanner_reports(self, transformation_id, status, source):
    """Refresh reachability of an already-represented transformation."""
    reg = self.rt.registry
    observation = GraphReachabilityObservation(
        transformation_id, status, self.time, source, f"{source}:{self.time}")
    trig = refresh_graph_reachability_and_plan_reentry(
        reg, GraphReachabilityRefreshRequest(
            reg.graph_substrate_identity(), (observation,), f"refresh-{self.version}"))
    self.record(f"reachability {transformation_id} -> {status}")
    return trig.reentry_plan.recompute_from_stage if trig.reentry_plan else None

def admit_tool(self, action_id, family, cls, text, source, rationale):
    """Admit a new function atomically with its first transformation."""
    reg = self.rt.registry
    action = ActionDefinition(action_id, text, ("A", "C"))
    transformation = GraphTransformationDefinition(
        action_id, "service", "service", family, ("A", "C"), "reachable",
        action_id, (cls,))
    trig = admit_novel_function_and_plan_reentry(reg, NovelFunctionAdmissionRequest(
        f"admit-{action_id}", reg.action_registry_identity(),
        reg.graph_substrate_identity(), action, transformation,
        "cfg-2", source, f"{source}:{action_id}", rationale))
    self.record(f"admitted tool {action_id}")
    self.rebind() # binding becomes possible only now
    return trig.reentry_plan.recompute_from_stage if trig.reentry_plan else None

```

Running the file plays one shift:

Listing 10.3 — Program output

```

1 memory leak
  selected: restart      completed, service healthy=True
  regime: Stabilization; in Graph: restart, hotfix, escalate
  set aside: hotfix: inadequate | escalate: Sigma
2 bad config
  selected: escalate      completed, service healthy=False
  regime: Stabilization; in Graph: restart, hotfix, escalate
  set aside: restart: inadequate | hotfix: inadequate
-- scanner: replica route reachable; re-enter at Graph/Seed
3 route reachable, freeze on
  selected: escalate      completed, service healthy=False
  regime: Stabilization; in Graph: restart, hotfix, failover, escalate
  set aside: restart: inadequate | hotfix: change_freeze | failover: change_freeze
4 freeze lifted, credential revoked
  selected: failover      authority withdrawn before the call
  regime: Stabilization; in Graph: restart, hotfix, failover, escalate
  set aside: restart: inadequate | hotfix: inadequate | escalate: Sigma
4' re-decided without credential
  selected: escalate      completed, service healthy=False
  regime: Stabilization; in Graph: restart, hotfix, failover, escalate
  set aside: restart: write_credential | hotfix: write_credential | failover: write_credential
-- new tool: binding refused before admission (action not registered)
-- new tool admitted with its first transformation; re-enter at Graph/Seed
5 tool admitted
  selected: reconfigure    completed, service healthy=True
  regime: Stabilization; in Graph: restart, hotfix, failover, escalate, reconfigure
  set aside: restart: inadequate | hotfix: inadequate | failover: Sigma | escalate: Sigma

```

Table 10.5 — What decided each episode of the shift

Episode	Decided by	What it shows
1 Memory leak	Adequacy, then Σ	The in-place patch is rejected for destroying control; restart beats escalation.
2 Bad config	Adequacy	A restart will not help; escalation is the only adequate policy.

Episode	Decided by	What it shows
Scanner report	Reachability refresh	The replica route becomes reachable; the Graph is re-entered.
3 Freeze on	Constraints	The route is reachable but not permitted during the freeze.
4 Credential revoked	Dependency match	Failover is selected and authorized; the revocation arrives before the call and the authority is withdrawn.
4' Re-decided	Constraints	Without a credential, every write action is removed.
New tool	Controlled admission	Binding is refused until admission; admission re-enters the Graph.
5 Tool admitted	Σ Stage 1	The new tool is governed like any other and is selected on its merits.

Two details are easy to miss. Whenever the replica route appears as a candidate, it does so only because the degraded service puts the agent in the Stabilization regime; in the runtime, Mission regime activates only continuation and local-adjustment families, so a structural path such as failover is suppressed when the service is healthy unless the request requires it. The accompanying tests check this directly. And in episode 5, the new tool wins not because it is new but because its projection dominates on both governing domains.

What this example exercises: reachability refresh versus structural admission, reachable versus permitted, authority withdrawn by a relied-on fact, regime-shaped candidate families, and a control domain that keeps the agent's fastest fix from being its chosen one.

Episode 4 is where the shift meets the decision core's boundary. The revocation arrived before the call, so withdrawing authority was enough. Had it arrived while the failover was under way, the core could not have noticed. Chapter 12 replays that case on runtime v2.1.167, with the identity system as a registered monitor and the failover supervised while it runs.

10.6 A Teaching Benchmark: Safe Margin versus Fast Gain

The Safe Margin versus Fast Gain gridworld is a teaching benchmark, specified for Version 2.1 as a narrow demonstration. Its proposition is bounded: task success or visible reward is not identical to operational viability. A Fast Gain policy chases the highest immediate reward; a Safe Margin policy first keeps energy and recovery access above declared limits and only then pursues reward. False success is declared in advance as a visible success accompanied by a governing viability or recovery failure.

The specification is explicit about what the benchmark is not. It is not an implementation or validation of the PV-PP framework or of any runtime; its Safe Margin policy is a simple heuristic and must not be labeled "the PV-PP algorithm"; and it does not show that a safe-margin policy is always preferable or that conventional methods cannot represent the same distinction. Safe Margin is not to be hard-coded to win. The benchmark succeeds if it shows clearly when reward and viability agree and when they diverge.

The archive holds the specification but no banked results, so this Guide reports none. A reader who implements it should follow Chapter 9: freeze the configuration, declare the false-success

condition and the acceptance criteria before running, record seeds and trial counts, and report both policies' scores beside their viability outcomes.

10.7 What Carries Across, and What Does Not

Set side by side, the examples show which parts of the architecture travel unchanged and which are supplied fresh in each domain.

Table 10.6 — The same constructs in five domains

Construct	Coffee	Household	Bakery	Cooling plant	Agent
Governing domains	Delivery; transaction	Transport; solvency	Liquidity; staffed capacity	Temperature margin; integrity	Availability; control
Enabling resource vs PP	Cups vs dispensing	Car vs transport	Oven vs production	Chiller vs cooling	Replica vs availability
Evidence with limits	Network report age	Diagnosis scope	Part back-order report	Telemetry age	Scanner confirmation
Reachable but not permitted	Sale while capture open	Loan approved but inadequate	Contract beyond shared capacity	Restart without cause	Failover in a freeze
Unknown outcome	Capture timeout	Relative's agreement	Rented oven night	Chiller start before flow	Tool-call timeout
Governed remedy	Refund	Bridge rides	Wholesaler for obligation	Load shedding	Escalation

What carries across is the set of questions and the discipline that keeps them apart. What does not carry across is everything that answers them: how margins are measured, what causes a failure, which source is authoritative, which rule binds, and what professional standard governs the work. Those remain with the domain—the mechanic, the engineer, the bakery's food-safety rules, the platform's security policy. An application built on the PV-PP framework should say so plainly, and in regulated domains it should carry the disclaimers the domain requires. The examples in this chapter illustrate structure; they are not financial, engineering, or security advice.

The examples also do not show that unlike domains are the same. A household, a bakery, a cooling plant, and a software agent differ in almost everything that matters to the people who run them. They share a decision structure: preserve what must be preserved, keep recovery open, act only on current authority, and record what actually happened.

10.8 Common Errors When Porting the Architecture

- Copying the coffee model's structure instead of rebuilding the model from the new domain's facts.
- Treating the enabling resource as the capability: the car, the oven, the chiller, the replica.
- Framing a unit instead of a function, so that a genuine substitute cannot count.
- Letting a counterparty's approval stand in for the actor's own adequacy.

- Treating a commanded state as a measured one, or a report's confidence as current evidence.
- Summing members' capabilities to obtain a collective entity's capability.
- Letting an interlock, a lender, or a platform permission act as a second decision system.
- Borrowing numbers from an example as if they were domain facts.
- Presenting an illustrative model as professional advice in a regulated domain.

10.9 From Examples to Positioning

The Guide has now shown what the PV-PP framework is, how to model with it, how to build and test applications on the frozen runtime, and how the architecture carries across domains. What remains is to place it: how it relates to expected utility, optimal control, game theory, and viability and safety methods; what the scalar-reduction proofs establish and where they stop; and which questions are still open. Chapter 11 takes those up.

Chapter 11 — Positioning, Reduction, and Advanced Topics

The Guide has explained what the Productive Value–Productive Power (PV-PP) framework is, how to model with it, and how to build and test applications on the frozen runtime. This last chapter places the framework among other ways of making decisions. It is the chapter where overclaiming is easiest, so it is organized around a simple discipline: every claim is labeled as proved, demonstrated, or prospective, and every proved claim carries its conditions.

The centerpiece is the scalar-reduction proof program, closed in September 2026, which answers the question most readers bring to a multidimensional framework: when is one number enough? Its answer runs in two directions, and both are bounded. The chapter then compares the PV-PP framework with expected utility, dynamic programming and optimal control, game theory, and viability and safety methods; treats security and agent governance as an advanced application; explains what portability across disciplines does and does not mean; takes up the advanced topics the earlier chapters deferred; and lists the questions that remain open.

Scalar comparison is locally recoverable within the PV-PP framework under certified conditions. The PV-PP framework is not globally equivalent to scalar comparison within the established non-reduction classes. Neither claim is universal.

11.1 What Positioning Means Here

The PV-PP framework does not reject optimization, utility representation, control theory, equilibrium analysis, reachability, constraint programming, or scalar summaries. Its narrower position is that whether a single governing comparison is valid for a decision is a modeling result to be established, not an assumption to be made. The PV-PP framework is best understood as an architecture for deciding what must remain explicit—viability thresholds, recovery, the difference between actual and perceived state, the authority of evidence, the completeness of the options, the separation of selection from execution—before optimization, equilibrium, or a score is applied.

Three kinds of claim appear in this chapter and must not be mixed. Proved claims are theorems, with the conditions under which they hold. Demonstrated claims are what software and benchmarks have shown, with the limits Chapter 9 attached to them. Prospective claims describe work the architecture allows but that has not been done. Section 11.12 sets the three side by side.

11.2 Direction 1: Containment

The containment theorem answers the first half of the scalar question: are there decisions on which a plain score gives exactly the PV-PP framework's answer? It proves that there are, on a precisely defined subclass. On that subclass, the set of policies that the PV-PP framework's Σ Stage 1 leaves on the governing frontier is exactly the set that an unweighted average of the represented consequence coordinates ranks highest. The claim is set-valued: the two procedures identify the same maximal set; choosing one policy from it remains the architecture's job.

Table 11.1 — The containment subclass in plain terms

Condition	Plain meaning
Collinearity (SC-Geo)	Every candidate's represented consequences lie on a single line.
Governing-domain monotonicity (SC-GDM)	Moving along that line never worsens a governing domain and improves at least one; non-governing coordinates are unconstrained.
Attainment (SC-X)	Some admissible candidate reaches the best point on the line.
Alignment ($K_2 > 0$)	For the unweighted average, the line's direction averaged over all represented coordinates—governing and non-governing—is positive. This does not follow from monotonicity.
Shared basis (SC-1 to SC-5)	Both routes compare the same candidates, exclusions, and consequences; the score adds nothing of its own; an earned order holds; and the score compresses the order rather than originating it.

Three features make the result useful rather than merely true. Every condition is checkable from the PV-PP framework's own objects, so the framework can certify when the score is entitled to agree—this is the basis of the scalar-shortcut certificate in Section 8.12. Membership is assessed per decision: a situation may qualify at one time and fail at the next if the governing set changes. And the result is local: it establishes containment on this subclass only, not for scalar methods generally. The program classifies Direction 1 as proved locally only.

The result is published as *When Is One Number Enough? A Containment Theorem for Scalar Comparison in Staged Decision Architectures* (Amundsen, version 2.0, September 2026; DOI 10.5281/zenodo.22715202).

11.3 Direction 2: Non-Reduction

The second half asks whether scalar comparison can stand in for the PV-PP framework's staged selection in general. On a stated class it cannot, and the program establishes this at three levels. The class is the governing-conflict environment: at least two governing domains are active, the Stage 1 frontier contains policies that do not dominate one another on the governing domains, and—the condition called SC-S2—at least two frontier policies tie on every governing domain but differ on some non-governing one, so that Stage 2 has real work to do.

- Level 1, fidelity. A faithful score would need two properties: its top set equals the Stage 1 frontier exactly, and it separates any two frontier policies that Stage 2 separates. On any governing-conflict environment with SC-S2, no scalar function has both. The result holds for any scalar and survives small relaxations of the first property.
- Level 2, stage conflation. On the same class, every scalar that makes Stage 2's distinctions must give different values to policies that Stage 1 cannot tell apart. Such a score may still return the right set; the failure is one of process, not necessarily of output. It merges two questions the architecture keeps separate.
- Level 3, dynamics. Consider scores that depend only on consequence vectors and never consult the governing set. Under stated conditions—the governing set changes while the

admissible set and consequences stay fixed, together with specified shift conditions—no such score identifies the PV-PP framework's Stage 2 set correctly at both states. A score that does track the PV-PP framework across such changes must consult which domains are governing at each state; it is then PV-PP-guided compression, not an independent scalar method.

The program classifies Direction 2 as strong but conditional. Its static result is a failure of process fidelity, not an automatic error in output; its dynamic results hold under the stated shift conditions; and none of it is a universal theorem against scalar models.

11.4 The Boundaries of the Proofs

The program's own summary is the claim this Guide adopts: scalar comparison is locally recoverable within the PV-PP framework under certified structural conditions, while scalar comparison is not globally equivalent to the PV-PP framework within the established non-reduction classes and stated dynamic boundary conditions. Its closeout records two claims as neither made nor required: broad scalar containment and universal scalar impossibility.

Several questions are explicitly open. Vector-valued utilities may represent what a single scalar cannot. An infinite sequence of scalars might approximate the PV-PP framework arbitrarily closely. Nonlinear governing scalars lie outside the linear verification method the program developed. Whether the SC-S2 condition is generic, whether the containment comparator can be widened when the alignment condition fails, and whether collinearity can be relaxed are also open, as is dynamic tracking by scores that consult the PV-PP framework's state.

Two further boundaries matter for anyone applying the results. First, the linear verification method checks whether a nonnegative governing-linear score can recover a frontier that Σ has already computed; it does not compute the frontier and does not extend containment. Second, the program found that the historical theorem packet left under-specified how per-domain capability relates to the consequence description used to exclude candidates. It repaired this with narrowly scoped companion authority for applying the theorem to current decisions. The repair improves lawful use of the theorem; it does not retroactively strengthen what the historical theorem claimed.

The program was closed on September 11, 2026.

11.5 What the Benchmarks Add

The benchmark programs of Chapter 9 point the same way as the proofs, without being proofs. Some public problems are scalar-contained: every Fruit Tree outcome is supportable by some nonnegative linear weighting, and five MuJoCo tasks recover their original rewards as documented linear combinations of their components. Others are not: in Minecart, four of ten Pareto-efficient outcomes are supported by no nonnegative weighting, and on Deep Sea Treasure's concave front only the two endpoints are. Water Reservoir shows a different gap: an equal-weight scalar returns a best release at every tested state even though no release satisfies every governing domain, where the PV-PP framework reports an empty adequate set and routes to fallback. And the aircraft model's total constraint violation is an exactly lossless scalar for its own feasibility question—some numbers are enough.

These findings concern the weighted-sum forms that were tested. They do not show that every nonlinear scalar fails, and they are not trained-agent comparisons. The Small Economy comparison adds a useful counterweight: a well-chosen lexicographic rule matched the PV-PP

framework on mortality there, and the framework should not claim that a multidimensional architecture must outperform a good fixed rule.

11.6 Structural Comparison with Established Methods

Each established approach already represents more than a caricature suggests. The comparison below follows the framework's structural-positioning material: what each method already handles, what the PV-PP framework makes separately explicit, and when the established method may be enough.

Table 11.2 — The PV-PP framework and established methods

Approach	Already represents	The PV-PP framework makes separately explicit	May be sufficient when
Expected utility	Disciplined preference over uncertain, possibly rich consequences	Actual versus perceived state, objective provenance, viability thresholds, candidate completeness, evidence authority, execution realization	The structures can be represented faithfully in a utility model
Dynamic programming and optimal control	Multidimensional state, stochastic dynamics, partial observability, hard constraints, safety sets, reachability	Whether the governing comparison is validly scalar for this decision, asked before the value criterion is set	Scalar validity is established; then these methods can run inside or alongside the PV-PP framework
Game theory	Multiple actors without a group utility	A role-disciplined productive architecture for each qualified actor	Strategic interaction is the question; the two are complementary
Viability, reachability, safety, constraint methods	Sets, invariance, safety, reachable state—the PV-PP framework's nearest relatives	A productive ontology, perception, objectives, evidence authority, completeness, and the separation of selection, execution, and state change	The added structure changes no decision; whether it does is an empirical question

The PV-PP framework adds most where the distinctions it keeps are active: viability thresholds or recovery corridors matter; consequences are heterogeneous and non-compensable; the regime changes; perception diverges materially from actual state; objectives change or conflict; evidence can be stale or unauthorized; commitments require exact authority; the set of options must be constructed rather than given; history must not collapse into a score; or selection can succeed while execution or realization fails.

Established methods may be sufficient or preferable when the objective is stable and validated, the admissible set and transition model are well specified, safety and authority can be represented without loss, the options are known to be complete, a valid scalar recovery is

known, and solver efficiency matters more than distinctions that are inactive. In such cases the PV-PP framework may add documentation and governance overhead without changing the decision. That is an engineering consideration to state, not a defect to hide.

11.7 Security and Autonomous-Agent Governance

Agent governance is the application in which the PV-PP framework's separations do the most visible work, and the framework describes it as a dual corridor. The upper side governs consequential capability and authority: what an agent, tool, or subsystem can reach, control, combine, and execute. Individually ordinary capabilities—communication, persistence, credentials, external access, code execution, privilege—can become dangerous in combination. The lower side preserves threshold capacity: the reserves, recovery routes, and governing conditions that must remain available for the system to stay viable. A decision can be insecure without anyone gaining a dramatic new privilege, simply by leaving too little viable capacity underneath.

The rule that follows is compact: govern the capability being created, preserve the capacity that cannot be lost, and authorize a consequential effect only while both remain inside the viable corridor. Chapter 10's operations agent is a small instance: its control domain kept the fastest fix from being the chosen one, and its tools acted only through current, single-use authority.

Runtime v2.1.167 adds the piece agent governance most often asks for: a monitoring or identity system that withdraws a privilege while an agent is acting can, through the supervision layer, have that withdrawal governed against the running action; the host can then issue the stop the decision permits, which the runtime validates and records and the controller that runs the action carries out (Chapter 12). The division of labor is unchanged. The monitor reports; the runtime decides whether the action may continue; the controller stops it; and the host reconciles what actually happened.

The limits are as important as the idea. The PV-PP framework is a governance layer above operating-system security, identity and access management, network controls, and sandboxes; it replaces none of them. The agent-governance runtime work is architecture, not certification: production use requires security review, threat modeling, test harnesses, and domain validation. The runtime cannot make an untrusted tool trustworthy by reasoning about it after the fact. And the hardest open problems in this application are measuring domain effects and recovery sufficiency, and knowing that the candidate set is complete.

11.8 Portability Across Disciplines

The PV-PP framework supplies a common ontology, a decision architecture, and a set of owned distinctions. Each application supplies its domain mechanics: physical equations, biological processes, engineering failure models, market mechanisms, clinical knowledge. Those mechanics do not become PV-PP primitives or operators by being used in a PV-PP model. Chapter 10's examples showed the division in practice.

Portability carries three obligations. Do not relabel ordinary domain variables as PV, PP, or SPV because the label is convenient. When a domain fact cannot be expressed without changing a framework meaning, treat the mismatch as a framework limitation or research question, not something an adapter should hide. And state the framework's normative scope: it is descriptive at the framework level; it does not by itself prescribe a moral hierarchy, and applications in regulated domains carry the disclaimers those domains require.

11.9 Advanced Topics

Nested and collective entities. A group is treated as a productive entity only when it qualifies on its own terms—a boundary, persistent state, transition structure, viability conditions, retention across turnover, governance rules, and a real reason to treat it as a unit. A collective's perceived state is not the average of its members' perceptions, and individual and collective viability can conflict without either automatically dominating. Composite human-AI entities may qualify as nested individuals, but that treatment remains exploratory.

Non-scalar governing domains. The framework forbids by default the aggregation of heterogeneous domain magnitudes into one score, the pooling of utility across agents, and any assumption of commensurability. Section 11.2 describes the one proved exception and the conditions that license it.

Corridor representations. Reserve, recovery capability, and recovery reachability are distinct, and unsupported hope is not support. "Corridor" is a useful description of governed paths, including in security; it is not a framework primitive.

What makes a domain governing. In the runtime, a domain governs when its baseline horizon is close enough to the shortest one. A domain that does not deteriorate under baseline continuation—however important—can therefore fall outside the governing set; this happened while building Chapter 10's agent example, whose control domain dropped out until its baseline decline was modeled. Whether that behavior needs a framework-level rule for non-decaying but essential domains, such as safety conditions in some benchmark mappings, is recorded here as an open modeling question.

Framework and runtime evolution. The runtime instantiates what the framework licenses; it does not define the theory, and runtime convenience must not redefine framework meaning. Frozen runtimes are preserved rather than modified. Build 167 branched from v0.141 and left its decision core untouched; later builds on Framework 2.1 are numbered from v2.1.168. A framework change is decided in the owner specifications first and implemented afterward.

11.10 Open Questions

The Guide closes its account of the framework with the questions that remain open. None of them is resolved by anything in the preceding chapters.

- **Mixed policies.** In the hardened Friend-or-Foe benchmark, only a randomized policy succeeds. Whether the existing definition of a policy already permits a probability-bearing rule—selected deterministically by Σ and realized as one branch by ϵ —or whether that would add something the canon does not specify, is an open formalization gap. No random-choice operator has been adopted.
- **Scalar classes the proofs leave open:** vector-valued utilities, approximation sequences, nonlinear governing scalars, and dynamic tracking by scores that consult the PV-PP framework's state (Section 11.4).
- **Execution supervision beyond build 167:** governing a call already entered through native call-through, autonomous sensing, and distributed supervision with coordinated fencing. Build 167 supplies monitor intake, in-flight checkpoints, and governed stop requests around a synchronous core (Sections 7.15–7.17; Chapter 12).
- **Recovery-corridor cancellation and expiry:** a governed interface operation for ending an active corridor (Section 7.18).
- **Essential domains that do not decay under baseline continuation,** and how they should enter the governing set (Section 11.9).

- Layer 2 as a dynamical system. A candidate question is whether the decision layer should be formalized as an explicit dynamical system over perceived productive power and regime states.
- Empirical comparison. Broader tests against strong conventional baselines, and in particular which apparent advantages are specific to the PV-PP framework and which a strong safety, control, decision, or governance architecture can reproduce.

11.11 Common Errors in Positioning Claims

- Claiming that the PV-PP framework is superior to expected utility, optimal control, game theory, or safety methods in general.
- Claiming that scalar methods are wrong or obsolete, or that every scalar model fails.
- Stating containment without its conditions, or treating it as containment of scalar methods generally.
- Stating non-reduction as a universal impossibility theorem.
- Citing benchmark findings about tested weighted sums as proofs about all scalars.
- Treating a verification that a score recovers Σ 's frontier as a derivation of that frontier.
- Treating a score that consults the PV-PP framework's governing state as an independent scalar method.
- Confusing the proof program's "scalar containment" with a sidecar's "score only admissible actions," which is ranking after gating.
- Presenting the PV-PP framework as a replacement for domain science, security controls, or professional standards.
- Attributing prospective capabilities to the frozen runtime, or supervision-layer capabilities to its decision core.

11.12 What V2.1 Establishes, What Has Been Demonstrated, and What Remains

Table 11.3 — The state of the claims

Established (framework and proofs)	Demonstrated (software and benchmarks)	Prospective or open
A productive-system ontology: PV, SPV, PP, actual and perceived state, objectives	Frozen runtime v2.1.167 implements the decision cycle, the execution lineage, and its validations, with 1,320 tests passing; its decision core is build 141's	Supervision of a native call already entered; autonomous sensing; distributed fencing
Execution under change: ϵ may return upstream when continuation is no longer licensed	Build 167's supervision layer governs an action while it runs and requests its stop; the Chapter 12 program and its tests	Supervision at scale and in distributed deployments
A staged decision architecture that keeps each question separate	The Guide's coffee and agent programs run against the unmodified runtime with their tests	A governed corridor cancellation and expiry operation

Established (framework and proofs)	Demonstrated (software and benchmarks)	Prospective or open
Information governance, reachability, and execution distinctions	All 10 AI Safety Gridworlds and all 31 MO-Gymnasium registrations without canonical change; three real-world models	Mixed or randomized policies
Containment: proved locally on a certified subclass	Scalar-contained and non-recovered cases in public benchmarks, for the tested forms	Vector utilities, approximation sequences, nonlinear governing scalars
Non-reduction: strong but conditional on stated classes	A lexicographic rule matching the PV-PP framework in one internal comparison	Layer 2 as a dynamical system; empirical advantage over strong baselines

11.13 Closing

The PV-PP framework began with two questions: what passes between us, and what remains possible afterward. The Guide has tried to turn those questions into something a reader can use—a way to model a productive system, govern its decisions stage by stage, act only on current authority, record what actually happened, and test the result honestly. The framework's claim is not that it answers every decision better than the alternatives. It is that some distinctions matter too much to be lost inside a single number, and that when one number is enough, the architecture can say so and prove why.

Chapter 12 then carries the runtime one step past the boundary Chapter 7 drew: it governs an action while it runs, when the facts it depends on change, without letting anything but the decision architecture decide.

Chapter 12 — Supervising Execution While It Runs

Chapter 7 drew a line. The runtime's decision core governs a decision and the authority to act on it, but once an action is under way it notices nothing until the host reports the outcome. A credential can be revoked, a permit withdrawn, or a limit crossed while the action runs, and the core cannot know. Section 7.17 said where an extension that closes more of that gap must connect, and what it must never do. Runtime v2.1.167 is that extension. This chapter explains its supervision layer, the package `pvpp_runtime.supervision`, and replays Chapter 10's fourth episode with one change: the operations agent's write credential is revoked while the failover is running.

Nothing in Chapters 1 through 11 changes. The supervision layer works through the same license, episode, authorization, ϵ , and re-entry machinery, and it is subject to the same rule: new evidence returns to governance; it never substitutes for it. Exact constructors, fields, and errors are in the Programmer's Reference, Chapter 16.

12.1 What Supervision Must Not Become

A supervisor sits close to the action, sees the world change, and is tempted to act on what it sees. That is exactly the second decision system Section 7.17 forbids. The supervision layer is built around three refusals:

- A monitor never decides. Its report is evidence about a fact. Whether the fact matters is settled by the dependencies the application declared, and what follows is settled by governance.
- A checkpoint never chooses an action. It answers one question—may this execution continue?—through canonical ϵ or the re-entry planner, and records the answer as a continuation posture. Choosing what to do instead is a new decision cycle.
- A stop request is not a stop, and a stop is not an effect. The runtime records the request; the controller carries it out and reports; the world's effect is reconciled separately; and only then may the host record a Layer-1 transition.

Monitors report. Governance decides. Controllers act. The host reconciles what happened.

12.2 The Layer at a Glance

The decision core is unchanged and still synchronous. The supervision layer wraps it with durable records kept in a supervisory store—SQLite in the reference implementation—and a set of services, each owning one link of the chain from a monitor's report to a recorded Layer-1 transition.

Table 12.1 — The supervision chain

Link	Service and records	What it settles	What it does not settle
Register	<code>CanonicalRuntimeBridge</code> ; <code>ActiveExecutionRecord</code>	Which canonical execution is being supervised, bound to its exact authorization	Any authority beyond the canonical one

Link	Service and records	What it settles	What it does not settle
Report and admit	ObservationService; SourceRegistration, ExternalObservation, GovernedFactView, AdmittedFactChange	Whether a report from a registered, current, in-scope source becomes a governed fact change	Whether the fact matters to any decision
Fan out	DependencyService; ExecutionDependencyBinding, GovernanceSnapshotToken	Which running executions declared a dependency on the changed fact	Dependencies nobody declared
Checkpoint	ContinuationService; ContinuationAssessment	Whether the execution may continue, and whether its unused authority must be fenced	What to do instead
Request a stop	ControlService; SupervisoryOwnershipRecord, ExecutionControlRequest	Which stop the current owner may request under the checkpoint's decision	Whether the controller can or will comply
Dispatch and enforce	ControlService; AdapterRegistration, ControlAttemptRecord, EnforcementRecord	The controller's own command for the request, and the controller's attested report	What the world now looks like
Reconcile and record	EffectService; EffectEvidence, WorldEffectReconciliationState, Layer1TransitionRecord	What effect authoritative evidence shows, and the host-authoritative transition	What PP, SPV, AVS, or X now mean
Recover	RecoveryService; RecoveryRequirementProfile	What must be reconciled before supervision resumes after a crash	The world's state; replay is evidence, not authority

Every service call is made by host code. The layer has no threads and no event loop, and nothing in it calls the host back except the trust verifiers the host itself supplies. What it provides is host-driven in-flight supervision: the host delivers external observations, calls checkpoints, and issues the controls a checkpoint permits, while the decision core remains synchronous. Every consequential operation also writes an append-only audit event, so the chain can be reconstructed; an audit event is evidence, never authority.

12.3 Registering an Execution

Supervision starts from the ordinary lineage of Chapter 7: a license from a completed cycle, an episode instantiated from it, and a native execution authorization for the action. The difference is that the episode is opened through a CanonicalRuntimeBridge, which records the exact objects the runtime issued, and the execution is then registered as active in the store. The record binds

the episode, the license, the decision cycle, the action, the binding version, the configuration, the attempt, and the authorization. It confers nothing: authority stays in the runtime's ledger.

The authorization matters here for a reason it did not in Chapter 7. It is the runtime's record of the execution's authority, and every later supervision step resolves the execution through it. While it remains issued, the execution can be checkpointed and its authority fenced. Once it is consumed—which native call-through does at the moment it enters the function—the bridge can no longer resolve the execution, and supervision of it fails closed. Section 12.13 draws the consequence.

12.4 Monitors, Admission, and Governed Facts

A monitor becomes a source by registration under a host trust root. The registration names the source's evidence roles, the fact kinds and subjects it may report on, the configurations it serves, its permitted uses, its validity window, and its status: current, limited, superseded, or revoked. Chapter 4's distinctions are exactly the admission checks. A report is admitted only if its source is current, applicable to the fact and subject, permitted for governance use, and temporally valid. Anything else is retained as evidence and changes nothing.

An admitted report becomes a governed fact change only if it changes the current view. Arrival order is not truth order: a report whose effective time is older than the current view is retained but does not overwrite it, and two reports that disagree about the same effective time leave the fact unresolved rather than letting the later arrival win. The monitor gains no decision authority by reporting.

12.5 Dependencies, Fan-Out, and Snapshots

An execution is affected by a fact only if a dependency on that exact fact—configuration, subject, fact kind, and fact identity—was registered for it, under a trust root. Dependencies are never inferred from names, timing, or proximity; Section 8.7's rule now has a runtime home. When a change is committed, fan-out returns every running execution that declared a dependency on it. One change can affect many executions, and each is then governed on its own.

Each checkpoint works from a governance snapshot: a token recording the execution's current dependency set and the exact versions of the facts in it. A material change between capturing the snapshot and using it makes the token stale, and the checkpoint is refused as a concurrency conflict rather than decided on facts that are no longer current. It does not rewrite any history that was valid when it was made.

12.6 Checkpoints and Continuation Postures

A checkpoint takes one of two kinds of input, and each reaches governance through machinery the earlier chapters already described:

- An execution observation—progress, completion, failure, an emergency—advances the canonical ϵ episode exactly as Chapter 7 described. The observation is frozen into plain data, and that one snapshot is used for its identity, the durable intent, the bridge's checks, and the canonical call.
- A governance invalidation signal names the earliest stage whose result a change has invalidated. The checkpoint passes it to the re-entry planner of Chapter 6; it does not recompute anything. The signal comes from host integration code: an admitted fact does not classify itself, and the checkpoint neither infers what a fact means nor discovers dependencies nobody declared.

The result is a continuation assessment, bound to the exact canonical step at which it was made. Its posture says whether the execution may continue and, if not, what kind of return is required; it also says whether the execution's unused authority must be fenced.

Table 12.2 — Continuation postures and the stops they permit

Posture	Arises when	Fence required	Stops the owner may request
continue	ϵ remains active or completes normally; or the planner finds no stage invalidated	No	None
reauthorization_required	An invalidation signal names a stage to re-enter	Yes	hold, pause, cancel, reauthorization_required
abort_return	ϵ returns upstream: continuation or completion is no longer sufficient	Yes	Any, including terminate
emergency_return	ϵ reports an emergency	Yes	Any, including terminate
indeterminate	ϵ cannot classify the step	No	hold, pause, reauthorization_required

Two properties keep checkpoints honest under retries and concurrency. A checkpoint is idempotent by event identity: repeating the same observation returns the same assessment without advancing ϵ again, and reusing an event identity with different content is refused. And a checkpoint can never be requested with a posture of the caller's choosing; the posture comes only from canonical ϵ or the planner.

12.7 Ownership, Fencing, and Stop Requests

Supervisory ownership is separate from PV-PP execution authority. A supervisor registered under a trust root acquires ownership of an execution, and each acquisition raises a fence number. Only the current owner, presenting the current fence, may fence authority or request a stop, so a supervisor that lost ownership—after a failover, a partition, or a restart—is refused as stale. An ownership may carry a lease, judged against the host's trusted clock rather than against timestamps the caller supplies.

Fencing invalidates the execution's authorizations that are still unused. It cannot reach authority already consumed. A stop request names one of a closed set of semantic controls—hold, pause, cancel, reauthorization_required, terminate—and is accepted only if the checkpoint's posture permits it (Table 12.2), the snapshot is still valid, and the canonical step has not moved since the assessment. A posture of continue permits no stop at all: governance cannot halt a healthy action.

Dispatch goes through an adapter registered for the controller. The adapter declares which semantic controls the controller supports and the controller's own command for each, and a mapping that would widen a request—cancel carried out as terminate—is refused at registration. Dispatch also checks that the target belongs to the adapter's namespace and is the target the request named.

12.8 Enforcement, Effect, and Layer 1

Chapter 7's four events become longer at the end. The controller's report is enforcement evidence: accepted, completed, rejected, too_late, timed_out, or unknown, each kept distinct, and attested only if the host's verifier accepts the controller's evidence. A completed enforcement says the controller did what it was asked. It does not say what the world now looks like.

That is the effect, and it is reconciled from evidence submitted by a source registered for that role. Its states distinguish no material effect, partial, completed, invalid, and unknown effects; partial and unknown effects stay unresolved until later evidence resolves them, and a timeout never becomes no effect. The canonical record of how ε ended is kept separately and never rewritten by later evidence. A Layer-1 transition can be recorded only from a current effect reconciliation and only with authority the host's verifier accepts; the runtime records it without interpreting it. In the Chapter 12 program these records stay separate: the checkpoint's assessment, the enforcement record, the effect reconciliation, and—when the host records one—the Layer-1 transition. A retry is treated as safe only if an idempotency contract was registered before execution, or if the evidence shows no material effect.

12.9 Crashes, Divergence, and Recovery

The canonical runtime and the supervisory store are not one transaction. The layer therefore records a durable intent before every canonical advance and classifies what it finds when something fails between the two.

Table 12.3 — Failures between the core and the store

Condition	State	What follows
Failure before the canonical call is entered	No canonical movement	The intent is closed; a fresh checkpoint can proceed
The canonical call was entered and failed; its state cannot be proved	canonical_state_indeterminate	Further advance or invocation is refused until the actual state is established
The canonical step advanced, but the assessment was not saved	reconciliation_required	Blind retry is refused; an authorized reconciliation compares the canonical state with the durable records
The divergence cannot be reconstructed	terminally_fenced	No revival of this attempt; the host reconciles the world and starts a new governed execution

After a restart, recovery is governed by a requirement profile registered before execution, which the recovering owner cannot weaken. Recovery reconciles control, effect, and world state against that profile and requires a fresh governance snapshot before supervision resumes. Replay of history is evidence, never live authority, and the live link between a stored execution and the runtime object is not durable: after a restart, operations that need an old execution's live canonical state fail closed. Durable records and audit are not a restored episode. A terminally fenced execution cannot be revived; after a restart, resumption requires the profile's recovery checks, and where a new attempt is warranted, a new decision and new governed authority. An old execution cannot be rebound to a new runtime object.

12.10 Trust and Deployment

The layer does not supply identity, cryptography, or credentials. The host does, through a `HostTrustProvider`: the registration roots that authorize sources, owners, adapters, dependencies, and recovery profiles; the verifier that judges a controller's attestation; the verifier that judges Layer-1 authority; and the trusted clock. Labels and booleans supplied by a caller are not authority, and a weak verifier yields weak trust; the layer cannot strengthen bad evidence by declaring it good. Code that can modify private objects in the same Python process is inside the trusted computing base.

The reference deployment is a durable store with one logical supervisor service. An in-process deployment suits embedded use and tests; a distributed deployment needs a coordination and fencing authority visible at every consequential boundary, which the host must provide. Build 167 imposes no size limit on observations, and an observation is stored in more than one record: in hostile testing a single 20 MB observation materially enlarged the store. Cap serialized observation size before admission, pass bulky evidence as references to immutable governed evidence, and monitor the store's size, its write-ahead log, and checkpoint latency.

12.11 Program: Supervising the Failover

`supervised_failover.py` replays episode 4 of Section 10.5 on v2.1.167. `ops_model.py` reduces the Chapter 10 agent to the decision that matters here: with the service degraded by a bad configuration, the replica route reachable, and the change freeze off, the agent may fail over only while it holds the write credential, and otherwise escalates. The deployment orchestrator and the trust configuration are small stand-ins for systems that, in a deployment, belong to the host.

The first function decides, registers the execution, registers two sources—the identity monitor, which reports facts, and the orchestrator, which reports effects—declares the execution's dependency on the credential, takes ownership, registers the orchestrator's adapter, and starts the rollout by the host's own means, reporting progress at a first checkpoint.

Table 12.4 — Who does what in the program

Party	What it does in the program
Host	Supplies the decision model, the sources and dependency declarations, the trust roots, the mapping of a revoked credential to Constraints, every checkpoint call, and the stop request
Runtime	Admits reports, finds affected executions, decides whether the execution may continue and which stops are permitted, and records every step
Orchestrator	Carries out the command and attests what it did; reports the world's effect through a separate, registered effect role

Listing 12.1 — Registering the supervised failover

```
def start_supervised_failover():
    """1-4. Decide, register the execution, its sources, owner, and adapter; start the rollout."""
    s = SimpleNamespace(orch=Orchestrator(), actual=actual_state(credential=True))
    s.rt, out = decide(s.actual)
    s.decision = out.decision.selection.selected_policy_id
    license = s.rt.build_execution_license_from_cycle(out.decision, request().domain_frame)

    # 1. Open the episode through the bridge; issue and register the execution's authority.
    s.bridge = CanonicalRuntimeBridge(s.rt)
    s.episode = s.bridge.instantiate_execution("failover-1", license, entry_sufficient=True,
```

```

                                max_steps=10).episode
s.bindings = ExecutionBindingRegistry(tuple(s.rt.registry.actions))
s.bindings.register(ExecutionBindingIdentity("failover-bind", "failover", "1"),
                    lambda ctx: s.orch.start("replica"))
s.auth = s.bridge.issue_native_execution_authorization(
    s.episode, "failover", s.bindings, decision_cycle_id="cycle-1", configuration_id=CFG)
s.store = SQLiteSupervisoryStore(":memory:", trust_provider=trust())
handle = s.bridge.resolve_canonical_execution(s.auth, configuration_id=CFG)
s.active = s.bridge.register_active_execution(
    s.store, handle, registered_at=at(0),
    execution_id=s.rt.native_execution_authorization_execution_id(s.auth.authorization_id))
s.aid = s.active.active_execution_id

# 2. Sources: the identity monitor reports facts; the orchestrator reports effects.
s.obs, s.deps = ObservationService(s.store), DependencyService(s.store)
s.cont = ContinuationService(s.store, s.deps, s.bridge)
s.effects = EffectService(s.store, s.obs)
for reg_id, source, roles, kinds, uses in (
    ("reg-iam-1", "iam-monitor", ("observation",), ("write_credential",), ("governance",)),
    ("reg-orch-1", "orchestrator", ("effect",), ("rollout",), ("effect_reconciliation",))):
    s.obs.register_source(SourceRegistration(
        reg_id, source, roles, kinds, (AGENT,), (CFG,), uses, "signed-feed", None, None,
        "current", ADMIN, 1), authority_proof=PROOF)
report_credential(s, "cred-1", "granted", at(0))
s.deps.register_dependency_binding(
    active_execution_id=s.aid, configuration_id=CFG, subject_id=AGENT,
    fact_kind="write_credential", fact_id="deploy", registration_authority=ADMIN,
    authority_proof=PROOF, registered_at=at(0))

# 3. Two supervisors may own executions; one does now. The orchestrator's adapter.
s.ctl = ControlService(s.store, s.deps, s.cont, s.bridge)
for owner in ("supervisor-1", "supervisor-2"):
    s.ctl.register_supervisory_owner(owner_id=owner, configuration_id=CFG,
                                    registration_authority=ADMIN, authority_proof=PROOF,
                                    registered_at=at(0))
s.owner = s.ctl.acquire_ownership(active_execution_id=s.aid, owner_id="supervisor-1",
                                acquired_at=at(0))
s.adapter = s.ctl.register_adapter(
    adapter_id="orchestrator", configuration_id=CFG, target_handle_namespace="orchestrator:",
    supported_semantic_controls=("hold", "cancel", "terminate"),
    command_mapping={"hold": "hold", "cancel": "cancel", "terminate": "terminate"},
    vendor_command_mapping={"hold": "hold_rollout", "cancel": "abort_rollout",
                            "terminate": "kill_rollout"},
    controller_attestation_method="signed", control_region_evidence_method="rollout-state",
    registration_authority=ADMIN, authority_proof=PROOF, registered_at=at(0))

# 4. The host starts the rollout by its own means; the episode stays open while it runs.
s.rollout = s.orch.start("replica")
s.tick = s.cont.checkpoint(active_execution_id=s.aid, checkpoint_at=at(2),
                          observation=ExecutionObservation("rollout-25pct"))

return s

```

The revocation is reported, admitted, and fanned out. Every affected execution is checkpointed with a signal naming Constraints, where the credential rule is enforced; that mapping from a revoked credential to Constraints is the host's, written into `revoke_credential`, and the runtime validates and governs the signal it is given. Then the current supervisory owner fences the authorization not yet used—which withdraws authority, not the running operation—issues the control request the decision permits, dispatches it through the registered adapter, and records the outcome the orchestrator reports.

Listing 12.2 — The revocation, the checkpoint, and the governed stop

```

def revoke_credential(s):
    """5. The credential is revoked mid-rollout; the host checkpoints every affected execution."""
    change = report_credential(s, "cred-2", "revoked", at(5))
    s.affected = s.deps.resolve_affected_executions(change, configuration_id=CFG)
    s.assessments = {}
    for aid in s.affected:
        signal = GovernanceInvalidationSignal(
            "credential-revoked", "Constraints", "write credential revoked",
            "monitor:iam-monitor", change.observation_ids, (), s.active.execution_id, CFG)

```

```

        s.assessments[aid] = s.cont.checkpoint(active_execution_id=aid, checkpoint_at=at(5),
                                              invalidation_signals=(signal,),
                                              trigger_ids=(change.change_id,))

    return s.assessments[s.aid]

def stop(s, assessment, control, owner_id="supervisor-1", fence=None):
    """6. Fence unused authority, request the stop, and record what the orchestrator reports."""
    fence = s.owner.owner_fence if fence is None else fence
    s.fenced = ()
    if assessment.authority_fence_required:
        s.fenced = s.ctl.fence_execution_authority(
            active_execution_id=s.aid, owner_id=owner_id, owner_fence=fence,
            reason="authority withdrawn")
    req = s.ctl.issue_control_request(
        assessment_id=assessment.assessment_id, active_execution_id=s.aid, owner_id=owner_id,
        owner_fence=fence, requested_control=control, target_handle=s.rollout,
        created_at=at(5))
    s.attempt = s.ctl.dispatch_control_request(
        control_request_id=req.control_request_id, owner_id=owner_id, owner_fence=fence,
        adapter_registration_id=s.adapter.adapter_registration_id, target_handle=s.rollout,
        control_region_at_request="interruptible", capability_evidence_ref="orch-caps-1",
        mapping_identity="orch-map-1", dispatched_at=at(5))
    outcome, evidence = s.orch.send(s.attempt.mapped_command, s.rollout) # the orchestrator acts
    return s.ctl.record_enforcement_outcome(
        control_attempt_id=s.attempt.control_attempt_id, outcome=outcome,
        controller_evidence_ref=f"{s.rollout}:ack",
        observed_control_region=s.orch.rollouts[s.rollout], recorded_at=at(6),
        attestation_evidence=evidence)

```

Finally the orchestrator's evidence of what the world shows is reconciled, and the agent is governed again from facts in which it no longer holds the credential.

Listing 12.3 — Reconciling the effect and governing again

```

def reconcile(s, state="no_material_effect_evidenced"):
    """7. Enforcement is not effect: reconcile what the orchestrator says the world shows."""
    e = s.effects.submit_effect_evidence(
        active_execution_id=s.aid, source_id="orchestrator", configuration_id=CFG, state=state,
        realized_bundle_ref=None, evidence_ref=f"{s.rollout}:traffic-report",
        provenance="orchestrator traffic report", observed_at=at(7), effective_at=at(6),
        received_at=at(7))
    return s.effects.reconcile_effect(active_execution_id=s.aid,
                                     effect_evidence_id=e.effect_evidence_id, updated_at=at(7))

def govern_again():
    """8. Govern again from the new facts: the agent no longer holds the credential."""
    _, out = decide(actual_state(credential=False, time=1.0), make_runtime())
    return out.decision.selection.selected_policy_id

```

In sequence: report, admission, fan-out, checkpoint (reauthorization_required, with a fence), fence of the unused authorization, control request (cancel), dispatch (abort_rollout), enforcement record (completed, attested), effect evidence and reconciliation (no material effect), and a new decision cycle (escalate).

Listing 12.4 — Program output

```

decision:      graph:failover | rollout orchestrator:rollout-1 in_progress
checkpoint 09:02: continue (epsilon active)
credential revoked at 09:05; affected executions: 1
checkpoint 09:05: reauthorization_required | fence required: True
unused authority: 1 fenced; invalidated
stop:          cancel -> abort_rollout | completed | attestation verified | aborted
world effect:  no_material_effect_evidenced | unresolved: False
governed again: graph:escalate

```

12.12 Reading the Run

At 09:02 the rollout is a quarter done and the checkpoint's posture is continue: nothing it depends on has changed. At 09:05 the identity monitor reports the revocation. Admission makes it a governed fact change, fan-out finds the one execution that declared a dependency on the

credential, and the checkpoint returns `reauthorization_required` with a fence: the Constraints result the failover was licensed under is no longer valid. The owner fences the one authorization still unused, requests a cancel, and the adapter translates it into the orchestrator's `abort_rollout`. The orchestrator reports that it completed the cancellation, and its attestation verifies. That is enforcement evidence: it says what the orchestrator did.

Two steps separate that report from any conclusion about the world. The orchestrator's traffic report, submitted through its separately registered effect role, is admitted as effect evidence—no material effect, because traffic never switched—and only then is the effect reconciled. And the decision of what to do next is not the supervisor's. A new cycle from the updated facts removes every write action at Constraints and selects escalation, as episode 4' did in Chapter 10.

The tests check the boundaries the chapter has described: the governed stop; enforcement kept apart from effect, with unknown effects left unresolved; terminate refused under `reauthorization_required` and permitted after an emergency; no stop of a healthy rollout; a stale supervisor refused; an unregistered source ignored; idempotent checkpoint replay and refused reuse of an event identity; a stale snapshot refused; and the native call-through boundary of the next section.

Listing 12.5 — Test results

```
test_revocation_mid_rollout_is_governed_and_the_rollout_stopped PASSED
test_enforcement_is_not_effect_and_the_agent_is_governed_again PASSED
test_terminate_needs_an_abort_or_emergency_decision PASSED
test_a_healthy_rollout_cannot_be_stopped_by_governance PASSED
test_a_stale_supervisor_cannot_issue_a_stop PASSED
test_a_report_from_an_unregistered_source_changes_nothing PASSED
test_checkpoint_replay_is_idempotent_and_reuse_with_new_content_is_refused PASSED
test_a_stale_snapshot_is_refused PASSED
test_after_native_call_entry_the_runtime_can_no_longer_checkpoint PASSED
9 passed
```

12.13 What Supervision Still Cannot Do

- Govern a native call already entered. `invoke_registered_native` consumes the authorization at call entry, and from then on every checkpoint of that execution is refused. If the revocation arrives before entry, the fence makes the call refuse; after entry, the runtime cannot issue a governed stop. Work that may need to be stopped should be carried out by the host's own means with the episode open, as in the program.
- Act on its own. Nothing happens until the host delivers a report and calls a checkpoint for each affected execution.
- Stop anything physically, or undo an effect that cannot be interrupted. A payment sent or a message delivered stays sent or delivered, and an effect that may have happened is reconciled, not assumed away.
- Guarantee exactly-once effects. That needs idempotency or enforcement the controller provides.
- Discover dependencies, or know the world. An execution is affected only by the facts it declared, and the store holds supervisory records, not actual state.
- Remove the decision core's defects. Build 167 leaves the core as it was, including the Reference's defect K2, so a re-entry at Π completeness or earlier still needs the host's joint-recovery check (Reference 15.2).

12.14 Common Errors in Supervised Execution

- Letting a monitor, adapter, or supervisor choose, relax, or authorize an action.

- Treating a monitor's report as a fact before it is admitted, or treating arrival order as truth order.
- Guessing dependencies instead of declaring them when the execution is registered.
- Committing a change and checkpointing only some of the executions fan-out returned.
- Supplying a continuation posture instead of letting canonical ε or the planner produce it.
- Fencing or requesting a stop as a stale owner, or with a snapshot that is no longer current.
- Treating a stop request, a dispatch, or a completed enforcement as a realized effect.
- Recording a timeout or an unknown effect as no effect, or retrying without an idempotency contract or evidence of no effect.
- Running work that may need to be stopped inside native call-through, and then describing it as supervised.
- Describing the supervision layer as asynchronous in the sense of acting on its own, or as able to stop a process.
- Writing into the store's tables, or rebinding an old execution after a restart, instead of using the services and the recovery path.

Appendix A — AI Interview Prompts

This appendix collects one AI interview prompt per chapter. Each prompt asks an AI assistant to interview you about your own problem using that chapter's concepts and to produce the chapter's structured output. The prompts build on one another: each takes the previous prompt's output as its starting point, so that working through them in order carries a problem from a first fit assessment to definition packets, and eventually to implementation and tests. Section 1.12 explains the division of labor they assume: the assistant maps and drafts; the modeler remains responsible for meaning and review.

Using the prompts. Copy the text in the shaded box into a new session with a capable AI assistant, attach this Guide, and attach the outputs of earlier prompts where indicated. Prompts for Chapters 1–5 and 11 require only this Guide and earlier outputs; the Chapter 6–10 and 12 prompts can also use the frozen runtime package to produce runnable code and tests. Attach the companion PV-PP Programmer's Reference as well, for exact interface detail. Answer in your own words; the assistant is instructed to translate into Productive Value–Productive Power (PV-PP) terms and show you the translation. Save each output; it is the input to the next prompt.

Status. Prompts A.1–A.12 accompany Chapters 1–12, one per chapter. The prompts are working tools, not canonical specifications, and are expected to be revised as they are tested in use.

A.1 Chapter 1 Prompt — Is the PV-PP Framework the Right Tool?

Purpose. Understand the problem in the modeler's own terms and decide whether the PV-PP framework adds structure that a simpler scalar method would lose. Concluding that one number is enough is a legitimate result.

Attach. This Guide (Chapter 1 at minimum; the whole Guide is better).

Produces. A PV-PP Fit Assessment: a plain statement of the problem, a first separation of value from capability, a check of the structures the PV-PP framework keeps explicit, and a recommendation.

Prompt

You are helping me decide whether the PV-PP framework, described in the attached PV-PP Programmer's Guide, is the right tool for a problem I want to model or govern. Use Chapter 1 of the Guide as your main reference.

GROUND RULES

- Treat the attached PV-PP Programmer's Guide as your authority for PV-PP terms. If the Guide does not settle a question, say so; do not invent PV-PP semantics, operators, or state objects.
- Interview me one topic at a time. Ask no more than three questions per turn, in plain language. Do not require me to use PV-PP vocabulary; translate my answers yourself and show me the translation.
- Never invent facts about my domain. If you need a fact, ask for it. If I do not know, record it as an open assumption.
- When you propose a classification, give the reason and a confidence label (high, medium, or low).
- Keep three things visibly separate: what I told you, what you inferred, and what is still unknown.
- Before giving your final output, check it against the Guide's common-error list for this chapter and tell me what you corrected.

INTERVIEW TOPICS (in this order)

1. The problem. What system, decision, or process do I want to model or govern? Who or what acts? What is at stake? Over what time scale? What would I use the result for: understanding the system, supporting decisions, governing an automated agent, or running simulations or benchmarks?
2. Value and capability. In the interactions that matter, what actually passes between the parties (goods, money, information, care, protection, harm)? What capability does each party need, and how can an interaction strengthen, preserve, or damage that capability afterward?
3. Imperfect information. What do the decision-makers not know, misperceive, remember wrongly, or learn late? Where could a record, report, or belief be stale, incomplete, or wrong?
4. Bounds and limits. Are there minimums that must be preserved (reserves, safety margins, staffing, health) or ceilings that must not be exceeded (exposure, authority, access)? Are any conditions non-negotiable, so that no gain elsewhere could make up for violating them?
5. Change and recovery. Can the available options appear or disappear over time? If something goes wrong, is there a way back, and what does it depend on?
6. Acting on a decision. Is choosing an action different from being permitted to carry it out? Can an action be attempted without the intended effect actually happening? Does anyone need to reconstruct later why a decision was made?

ANALYSIS

Using the table in Section 1.8 of the Guide, classify each of these structures as clearly present, possibly present, absent, or unknown in my problem, citing what I said: state, capability, hard constraints, reachability, information and evidence, history and memory, recovery, selection versus execution, and realization. Then apply Section 1.7: if the decision genuinely reduces to one stable number, say so plainly.

OUTPUT: PV-PP FIT ASSESSMENT

1. Problem statement in my own words, followed by your one-paragraph restatement.
2. First-pass separation: likely Productive Value flows, likely Productive Power, and things that are neither (resources, conditions, obligations, information), each with a confidence label.
3. Structure table: each Section 1.8 structure, its status, and the evidence from my answers.
4. Recommendation: (a) a scalar or conventional method is probably sufficient, and why; (b) the PV-PP framework is likely to add structure, naming which structures and why they cannot safely be folded into one score; or (c) unclear, naming what we would need to learn.
5. Open questions and assumptions.
6. Next step: if the recommendation is (b) or (c), say what to bring to the Chapter 2 prompt (Appendix A.2).

Begin by asking me about topic 1.

Reviewing the output. Before relying on the result, check:

- The problem statement matches what you actually meant, in your own words, before any PV-PP translation.
- Anything classified as Productive Power is a real capability, not merely a useful resource, permission, or favorable condition.
- Every structure marked present is backed by something you said, not by the assistant's assumption.
- The recommendation seriously considered that a scalar method might be enough.

A.2 Chapter 2 Prompt — Sketch the Construct Map

Purpose. Translate the problem into a first-pass PV-PP construct map: who acts, what their actual, perceived, and objective states contain, what value and capability are involved, how change is represented, and which decision stages will carry the most weight.

Attach. This Guide (Chapters 1–2 at minimum) and the Fit Assessment from Appendix A.1.

Produces. A Construct Map Sketch: a first-pass table of the major constructs for this problem, a narrative of one governed decision in the domain's own terms, and a list of questions for Chapter 3.

Prompt

You are helping me sketch a first-pass PV-PP construct map for my problem, using Chapter 2 of the attached PV-PP Programmer's Guide. My Fit Assessment from the Chapter 1 prompt is attached; start from it rather than re-asking what it already answers.

GROUND RULES

- Treat the attached PV-PP Programmer's Guide as your authority for PV-PP terms. If the Guide does not settle a question, say so; do not invent PV-PP semantics, operators, or state objects.
- Interview me one topic at a time. Ask no more than three questions per turn, in plain language. Do not require me to use PV-PP vocabulary; translate my answers yourself and show me the translation.
- Never invent facts about my domain. If you need a fact, ask for it. If I do not know, record it as an open assumption.
- When you propose a classification, give the reason and a confidence label (high, medium, or low).
- Keep three things visibly separate: what I told you, what you inferred, and what is still unknown.
- Before giving your final output, check it against the Guide's common-error list for this chapter and tell me what you corrected.

This is a sketch, not a finished model. Chapter 3 will refine it. Prefer a clearly labeled gap to a confident guess.

INTERVIEW TOPICS (in this order)

1. Actors and time. Which actor or entity is the main decision-maker (actor i)? Are other actors or collective entities involved? What is a sensible time step t for this problem?
2. Actual state $S_i(t)$. What actually exists for actor i at a given time: capabilities (PP), retained value (SPV), viability condition (AVS), and other context (X)? Keep these separate, following Section 2.4.
3. Perceived state $P_i(t)$. What does actor i believe about that state, including its own capability (PPP)? From what information does that belief come, and where could it diverge from reality (Section 2.5)?
4. Objectives $O_i(t)$. What is actor i trying to achieve, preserve, prevent, recover, or develop? Do any objectives conflict? Remember that objectives do not create capability, reachability, or authority (Section 2.6).
5. Value and viability. What PV flows occur, what becomes SPV, and what conditions define viable operation, including lower and upper bounds (Section 2.7)?
6. Possible change. What actions, routes, or transformations are available, and what makes them reachable or unreachable (Section 2.9)?
7. Execution and realization. When an action is chosen, what actually carries it out, who or what determines whether it worked, and what updates the actual state afterward (Section 2.11)?

ANALYSIS

Using Table 2.1, identify which canonical decision stages are likely to carry the most weight for this problem (for example: governing-domain identification, Constraints, Adequacy and recovery, or execution) and explain why. Note any place where what the framework can represent may exceed

what a synchronous runtime can detect on its own (Section 2.12).

OUTPUT: CONSTRUCT MAP SKETCH

1. Construct table with columns: construct ($S_i(t)$ and its components, $P_i(t)$ and PPP, $O_i(t)$, PV, SPV, PP, viability conditions, possible changes, execution and realization, information sources), first-pass content, source (told, inferred, or unknown), and confidence.
2. One-loop narrative: walk one decision in this domain through the loop in Section 2.3, in plain language, as Section 2.13 does.
3. Heavily loaded stages from Table 2.1, with reasons.
4. Open questions for Chapter 3, grouped by construct.

Begin by summarizing what my Fit Assessment already establishes, then ask me about topic 1.

Reviewing the output. Before relying on the result, check:

- Actual state, perceived state, and objectives are kept apart; no objective has been counted as capability or authority.
- PP is one component of actual state, not the whole state.
- Each divergence between actual and perceived state has a named information source.
- Items marked “inferred” are ones you agree with, or are carried forward as open questions.

A.3 Chapter 3 Prompt — Build the Scenario and Model Definition Packet

Purpose. Build the Scenario and Model Definition Packet of Section 3.15 for the modeler's problem by working through Chapter 3 section by section.

Attach. This Guide (Chapters 1–3 at minimum), the Fit Assessment (A.1), and the Construct Map Sketch (A.2).

Produces. A complete Scenario and Model Definition Packet, a walk-through narrative, and a self-check against Section 3.18.

Prompt

You are helping me build a Scenario and Model Definition Packet, as defined in Section 3.15 of the attached PV-PP Programmer's Guide, for my problem. My Fit Assessment and Construct Map Sketch are attached; use them as the starting point and confirm or correct them as we go.

GROUND RULES

- Treat the attached PV-PP Programmer's Guide as your authority for PV-PP terms. If the Guide does not settle a question, say so; do not invent PV-PP semantics, operators, or state objects.
- Interview me one topic at a time. Ask no more than three questions per turn, in plain language. Do not require me to use PV-PP vocabulary; translate my answers yourself and show me the translation.
- Never invent facts about my domain. If you need a fact, ask for it. If I do not know, record it as an open assumption.
- When you propose a classification, give the reason and a confidence label (high, medium, or low).
- Keep three things visibly separate: what I told you, what you inferred, and what is still unknown.
- Before giving your final output, check it against the Guide's common-error list for this chapter and tell me what you corrected.

Additional rules for this chapter:

- Classify every fact by the role it plays in the modeled interaction, not by what it is called in ordinary language (Section 3.4). The same object can occupy different roles at different times.
- Do not solve an ontology problem by putting every useful thing into PP.
- Do not propose weights, scores, or a reward function. If a comparison seems to need one, record it

as an open question.

INTERVIEW SEQUENCE (follow Chapter 3 in order; skip a step only when I confirm it does not apply)

1. Scenario boundary: what starts and ends the episode, what is excluded, the time horizon, and whether capability or structure can change inside it (3.1).
2. Interactions: for each interaction that matters, its trigger, participants, PV flow, required capability, authority and evidence, bounds, execution, and result; add nested interactions only when they change governance (3.2).
3. Participants and roles (3.3).
4. Classification of PV, SPV, PP, enabling resources and mechanisms, and broader state (3.4).
5. Actual state, then how perceived state is formed from it, including where they can diverge (3.5).
6. For each important resource: possession, access, permission, control, and operability, separately (3.6).
7. Capabilities that exist only when several components are jointly present (3.7).
8. Objectives, kept separate from capability and authority (3.8).
9. Constraints, thresholds, governing domains, and corridors, including what counts as false success (3.9).
10. Recovery, decomposed into reserve, capability, mechanism, prerequisites, reachability, and obligation (3.10).
11. Whether any group should be modeled as a productive entity in its own right, applying the qualification conditions (3.11).
12. The environment and execution boundary: for each consequential action, the callable or mechanism, the required authority, what can fail or become indeterminate, and what evidence proves completion (3.12).
13. What can change during the episode and how the model will know (3.13).
14. History, current consequences, and any replay obligation (3.14).

OUTPUT: SCENARIO AND MODEL DEFINITION PACKET

1. The packet table from Section 3.15, one row per element, each entry marked told, inferred, or unknown.
2. A walk-through narrative of the complete model in the style of Section 3.16.
3. Open assumptions and deliberate exclusions.
4. Self-check: each common error in Section 3.18, whether the packet avoids it, and anything you corrected.

Begin with step 1.

Reviewing the output. Before relying on the result, check:

- Every item classified as PP is an actual capability under stated conditions; resources that merely enable it are listed separately.
- Possession, access, permission, control, and operability are recorded separately wherever they can diverge.
- Recovery is decomposed, not asserted; its current reachability is stated.
- The execution boundary names what proves an effect actually happened, not merely that a call returned.
- No weights or scores appear anywhere in the packet.

A.4 Chapter 4 Prompt — Build the Information and Evidence Packet

Purpose. Extend the Chapter 3 packet with the Information and Evidence Definition Packet of Section 4.15: how each decision-relevant fact is known, how far it can be trusted and used, and what happens when it changes.

Attach. This Guide (Chapters 1–4 at minimum) and the Scenario and Model Definition Packet from Appendix A.3.

Produces. An Information and Evidence Definition Packet, a dependency and invalidation map, a list of information-governance risks, and a self-check against Section 4.18.

Prompt

You are helping me build the Information and Evidence Definition Packet described in Section 4.15 of the attached PV-PP Programmer's Guide. My Scenario and Model Definition Packet from the Chapter 3 prompt is attached.

GROUND RULES

- Treat the attached PV-PP Programmer's Guide as your authority for PV-PP terms. If the Guide does not settle a question, say so; do not invent PV-PP semantics, operators, or state objects.
- Interview me one topic at a time. Ask no more than three questions per turn, in plain language. Do not require me to use PV-PP vocabulary; translate my answers yourself and show me the translation.
- Never invent facts about my domain. If you need a fact, ask for it. If I do not know, record it as an open assumption.
- When you propose a classification, give the reason and a confidence label (high, medium, or low).
- Keep three things visibly separate: what I told you, what you inferred, and what is still unknown.
- Before giving your final output, check it against the Guide's common-error list for this chapter and tell me what you corrected.

Additional rules for this chapter:

- Keep confidence, source authority, applicability, and permitted use separate. High confidence never creates authority; relevance never creates permission (4.4–4.5).
- Never overwrite a superseded item; record its disposition instead (4.6).
- Never place later knowledge into an earlier decision basis (4.7, 4.11).

INTERVIEW SEQUENCE

1. From my Chapter 3 packet, list every fact, signal, record, report, memory, or inference that a decision depends on. Confirm the list with me before going further.
2. For each item, establish in turn: its source and provenance; its quality or confidence; what the source is actually authorized to establish; the actor, state, interval, configuration, or decision it applies to; which decisions or governance steps may use it; its disposition (current, superseded, historical, disputed, and so on); and, where timing matters, when it was true versus when it was recorded or learned (4.2–4.7).
3. Where actual state and perceived state can diverge, identify the information pathway that produced the belief and what could make it wrong (4.3).
4. Separate event history from memory and expectation: which events need a Transfer History record, and which of their consequences remain presently material and must live in current state (4.8–4.10).
5. Replay: if anyone may need to reconstruct a past decision, what decision-time material must be retained (4.11–4.12)?
6. Change: for each item, what makes it stale, superseded, or contradicted; which artifacts depend on it; and where governance should resume when it changes (4.13–4.14).

OUTPUT: INFORMATION AND EVIDENCE DEFINITION PACKET

1. One row per information item, with the columns in Section 4.15, each entry marked told, inferred, or unknown.
2. Dependency and invalidation map: for each item, the state, reachability, policy, authority, adequacy, or execution artifacts that depend on it, and the earliest stage to revisit when it changes.
3. Information-governance risks specific to this problem: where confidence could be mistaken for authority, stale records for current state, or history for present truth.
4. A short worked example in the style of Section 4.16 showing one item going stale and the resulting update, without rewriting the historical record.
5. Self-check: each common error in Section 4.18, whether the packet avoids it, and anything you corrected.

Begin with step 1.

Reviewing the output. Before relying on the result, check:

- Confidence and authority are recorded separately for every item.
- Superseded or historical items are kept with their disposition, not deleted or overwritten.
- Each event that creates a lasting consequence has that consequence placed in current state, not only in history.
- Invalidation follows specific dependencies rather than “recompute everything.”
- The worked example never uses later knowledge to rewrite the earlier decision basis.

A.5 Chapter 5 Prompt — Build the Change and Reachability Packet

Purpose. Represent what can change in the modeler's system: the transformations that exist, what makes each reachable, the staged paths that recovery or exit requires, and how new routes, closures, and capability changes will enter the model.

Attach. This Guide (Chapters 1–5 at minimum), the Scenario and Model Definition Packet (A.3), and the Information and Evidence Definition Packet (A.4).

Produces. A Change and Reachability Definition Packet with its family-scan record, compositions, seeds, graph-validity result, a change walk-through, and a self-check against Section 5.18.

Prompt

You are helping me build the Change and Reachability Definition Packet described in Section 5.14 of the attached PV-PP Programmer's Guide. My Scenario and Model Definition Packet (Chapter 3 prompt) and Information and Evidence Definition Packet (Chapter 4 prompt) are attached; start from them.

GROUND RULES

- Treat the attached PV-PP Programmer's Guide as your authority for PV-PP terms. If the Guide does not settle a question, say so; do not invent PV-PP semantics, operators, or state objects.
- Interview me one topic at a time. Ask no more than three questions per turn, in plain language. Do not require me to use PV-PP vocabulary; translate my answers yourself and show me the translation.
- Never invent facts about my domain. If you need a fact, ask for it. If I do not know, record it as an open assumption.
- When you propose a classification, give the reason and a confidence label (high, medium, or low).
- Keep three things visibly separate: what I told you, what you inferred, and what is still unknown.
- Before giving your final output, check it against the Guide's common-error list for this chapter and tell me what you corrected.

Additional rules for this chapter:

- If one actor in my model can create, copy, or replace another, record what the new actor inherits. Configuration and information may pass; permissions only if I declare the transfer; current PP never—the new actor's PP must be established from its own evidence (Section 5.12).
- Do not propose a transformation I have not confirmed as possible in my domain. You may ask whether one exists; you may not assume it does.
- Record every transformation's reachability as reachable, unreachable, or uncertain—the three values the runtime accepts—with the evidence for it; an uncertain status needs a reason. Do not exclude something because it is difficult.
- Keep reachability separate from capability, desirability, and permission (Section 5.5). Record permissions as conditions; do not decide feasibility here.
- Configuration facts may say only reachable, unreachable, or uncertain, available or unavailable, and timing notes, with their evidence. Record permission facts separately, as conditions for Constraints. Never encode preferences or which loss matters more (Section 5.6).
- Objectives may guide where we look, but they never make anything reachable. Planned or desired capability is not current capability (Section 5.12).

CONSTRUCTION SEQUENCE (follow this order; it is required, per Sections 5.3–5.8)

1. Preservation object: state the function or capability to preserve or recover, independent of any particular mechanism unless the mechanism itself must be preserved (5.3).
2. Domains: for each, the threshold, maintenance need, drift, dependencies, substitutability, recovery structure, and irreversibility (5.2).
3. Instances: every support and burden instance, with condition, ownership and control, and what it supports (5.2).
4. Family scan: for each relevant instance, go through all eight families in Table 5.2 and ask me whether each applies. Record why any family does not apply (5.4).
5. Reachability: for each transformation, its enabling and blocking conditions, including access, control, operability, compatibility with other parties, timing, and irreversibility (5.5–5.6).
6. Composition: staged paths wherever recovery, substitution, or exit takes more than one step; do not drop a path just because it is long (5.7).
7. Family coverage, checked after composition (5.7).
8. Seeds: reduce to structurally distinct seeds, never merging retain/sell, repair/substitute, direct/staged, continue/release, or mechanism/function (5.8).
9. Graph validity: PASS, FAIL, or CONDITIONAL PASS, naming any failure classes from Table 5.4.

THEN, FOR CHANGE OVER TIME

10. Recovery routes, decomposed as in Section 5.9.
11. Closure watch list: windows, deadlines, and degradations that can close routes, and what could reopen them (5.10).
12. Dynamic structure: which reachability can be refreshed by evidence, where coverage may be incomplete, and how a new route or function would be admitted (5.11).
13. Capability change: which capabilities can change, what evidence would show it, and which transformations' reachability depends on them (5.12).
14. Invalidation map: for each kind of change, what becomes stale and where governance resumes (5.13).

OUTPUT: CHANGE AND REACHABILITY DEFINITION PACKET

1. The packet table from Section 5.14, each entry marked told, inferred, or unknown.
2. The family-scan record and the list of compositions and seeds, with the reduction basis.
3. The graph-validity result with reasons.

4. A short walk-through in the style of Section 5.15: one realistic change arriving, what it refreshes or invalidates, and where governance resumes.
 5. Self-check: each common error in Section 5.18, whether the packet avoids it, and anything you corrected.
- Begin with step 1.

Reviewing the output. Before relying on the result, check:

- Every one of the eight families was scanned for each relevant instance, and each exclusion has a stated reason.
- The preservation object is a function, not a specific mechanism, unless the mechanism genuinely must be preserved.
- No transformation was excluded merely for being difficult, and none was admitted whose conditions cannot be met.
- Staged recovery paths are present wherever recovery takes more than one step.
- Nothing in the configuration facts expresses a preference or a judgment of adequacy.
- No planned or desired capability is treated as present, and objectives never make anything reachable.

A.6 Chapter 6 Prompt — Configure the Decision Cycle

Purpose. Configure the canonical decision cycle for the modeler's system: what each stage receives, what decides each scenario, where each kind of failure belongs, and—when the runtime is available—a first runnable decision-cycle program.

Attach. This Guide (Chapters 1–6 at minimum) and the packets from prompts A.3, A.4, and A.5. Optionally, the frozen runtime package (runtime-v2.1.167, v2.1.167), which enables the code step.

Produces. A Decision Cycle Configuration Packet with its stage table, scenarios, failure-attribution table, and a self-check against Section 6.18; with the runtime attached, also a runnable program and its tests.

Prompt

You are helping me configure the canonical decision cycle for my model, using Chapter 6 of the attached PV-PP Programmer's Guide. My packets from the Chapter 3, 4, and 5 prompts are attached. If the frozen PV-PP runtime package (runtime-v2.1.167, v2.1.167) is also attached, you will finish by writing a first decision-cycle program modeled on `coffee_decision.py` in Section 6.15.

GROUND RULES

- Treat the attached PV-PP Programmer's Guide as your authority for PV-PP terms. If the Guide does not settle a question, say so; do not invent PV-PP semantics, operators, or state objects.
- Interview me one topic at a time. Ask no more than three questions per turn, in plain language. Do not require me to use PV-PP vocabulary; translate my answers yourself and show me the translation.
- Never invent facts about my domain. If you need a fact, ask for it. If I do not know, record it as an open assumption.
- When you propose a classification, give the reason and a confidence label (high, medium, or low).
- Keep three things visibly separate: what I told you, what you inferred, and what is still unknown.
- Before giving your final output, check it against the Guide's common-error list for this chapter and tell me what you corrected.

Additional rules for this chapter:

- Treat each stage as a separate question (Table 6.1). Never let one stage fix a problem that belongs to

another (Table 6.2).

- Do not design weights, scores, or priorities. The Σ order is a tie-breaker only and must not express a preference.
- Constraints must be classified as hard, conditional, or soft; do not turn a prohibition into a penalty.
- Adequacy is conjunctive across governing domains; no domain may compensate for another.
- If you write code: all Φ , H , G , $\mathcal{R}$, Π , completeness, Constraints, Framing, Adequacy, and Σ logic stays in the runtime. The host code may only supply facts through the registry, the adapter, and the request. Do not change the runtime.

INTERVIEW SEQUENCE

1. Domains and baselines: for each domain in my packets, its threshold, how its margin is measured, and how it drifts if nothing new is done (6.3–6.4). Confirm what the required “steady” baseline means in my system.
2. Governing tolerance: how close to the shortest horizon a domain must be to count as governing, and which domains are recovery-necessary for others (6.5).
3. Regime: what horizon and pressure levels would mean Stabilization, Survival, or Existential posture in my system, and whether any contextual interruption should force escalation (6.6).
4. Required families and classes: for each regime, which policy families and classes must be present, based on my Chapter 5 packet (6.7).
5. Constraints: each binding rule, its class (hard, conditional, soft), and, for conditional rules, when necessity-driven relaxation could apply (6.8).
6. Domain frame: the preservation target for each governing domain, stated as a function, with the substitutes that genuinely preserve it (6.9).
7. Recovery obligations: any recovery commitments that could already be active from earlier decisions, and what shared capacity or deadlines they compete for (6.10).
8. Projections: for each candidate policy, what it is projected to do to each governing domain, whether it preserves viability until recovery, enters a recovery-capable region, and supports joint sustainment (6.11).
9. Tie order: a fixed, state-independent order for breaking exact ties, with a note that it is not a preference (6.12).

OUTPUT: DECISION CYCLE CONFIGURATION PACKET

1. A table with one row per stage from Table 6.1, stating what my model supplies to that stage and what the stage is expected to decide.
2. Three to five scenarios, each with the stage I expect to decide it and why, in the style of Table 6.3. Include at least one case decided by Constraints, one by Adequacy, and one missing-family case that should stop before selection.
3. A failure-attribution table for my system in the style of Table 6.2.
4. If the runtime package is attached: a runnable program with a registry, a host adapter, and a request, plus a test file asserting each scenario's expected result and that the cycle does not change host state. Run nothing you cannot run; tell me exactly how to run it.
5. Self-check: each common error in Section 6.18, whether the packet (and code) avoids it, and anything you corrected.

Begin with step 1.

Reviewing the output. Before relying on the result, check:

- Every constraint is classified, and no prohibition appears as a penalty or weight.
- The Σ order is described and used only as a tie-breaker.

- Each scenario names the stage that should decide it, and the missing-family scenario stops before selection.
- Projection facts say what each policy does to each governing domain; nothing in them ranks or selects.
- If code was produced: host code supplies facts only, runs against the unmodified frozen runtime, and its tests pass.

A.7 Chapter 7 Prompt — Plan Execution and Realization

Purpose. Plan how the modeler's selected policies become realized state: what carries out each action, how each outcome is classified and evidenced, how actual state is updated after every outcome, which changes invalidate authority, and what the deployment's implementation boundary is—and, when the runtime is available, extend the Chapter 6 program to execute.

Attach. This Guide (Chapters 1–7 at minimum) and the packets from prompts A.3–A.6. Optionally, the frozen runtime package (runtime-v2.1.167, v2.1.167) and the Chapter 6 program, which enable the code step.

Produces. An Execution and Realization Packet with its action-binding, outcome, Layer-1, invalidation, and race tables, a deployment boundary statement, and a self-check against Section 7.19; with the runtime attached, also the extended program and its tests.

Prompt

You are helping me plan how my model moves from a selected policy to realized state, using Chapter 7 of the attached PV-PP Programmer's Guide. My packets from the Chapter 3–6 prompts are attached. If the frozen PV-PP runtime package (runtime-v2.1.167, v2.1.167) and my Chapter 6 program are also attached, you will finish by extending that program to execute, modeled on `coffee_execute.py` in Section 7.12.

GROUND RULES

- Treat the attached PV-PP Programmer's Guide as your authority for PV-PP terms. If the Guide does not settle a question, say so; do not invent PV-PP semantics, operators, or state objects.
- Interview me one topic at a time. Ask no more than three questions per turn, in plain language. Do not require me to use PV-PP vocabulary; translate my answers yourself and show me the translation.
- Never invent facts about my domain. If you need a fact, ask for it. If I do not know, record it as an open assumption.
- When you propose a classification, give the reason and a confidence label (high, medium, or low).
- Keep three things visibly separate: what I told you, what you inferred, and what is still unknown.
- Before giving your final output, check it against the Guide's common-error list for this chapter and tell me what you corrected.

Additional rules for this chapter:

- Keep the four events separate for every action: selection, current authority, attempt, and realization (Table 7.1).
- Never treat a function's return value as evidence of what changed in the world. Ask me where authoritative evidence comes from.
- A failure may be classified as clean only if I can say what evidence proves nothing material happened. Everything else is indeterminate or completed-but-invalid.
- Never propose that execution retry with a different action or policy. Anything that needs a new choice returns upstream.
- Keep framework capability, the runtime's decision-core behavior, its supervision layer, my host responsibilities, and prospective work in separate columns (Section 7.15). Do not attribute in-flight

sensing or cancellation to the decision core, or physical cancellation to either.

- If you write code: use the runtime's execution lineage, never construct licenses or authorizations by hand, and never write to runtime-private state. Do not change the runtime.

INTERVIEW SEQUENCE

1. Actions and functions: for each action in my Chapter 6 packet, the function or real-world procedure that carries it out, and whether it is one step or several steps that can fail separately (7.5, 7.16).
2. Outcome evidence: for each action, how I would know it succeeded, what a clean failure looks like and what proves it, what could leave the effect unknown, and what a completed-but-wrong result looks like (7.7).
3. Authoritative records: which sensors, ledgers, confirmations, or counterparties I trust for what actually happened, and how quickly they report (7.8).
4. Layer-1 transition: for each outcome class, what changes in PP, SPV, AVS, and X, including how unresolved outcomes appear in actual state, and which invariants must hold (7.9–7.10).
5. Stale authority: which facts, if they changed between decision and call, should invalidate authority, and how I would learn of each change (7.6, 7.11).
6. The race: for each action, how long the call takes, whether its effect is reversible, whether it can be made idempotent, and how an unknown outcome would be reconciled (7.16).
7. Recovery corridors: whether any recovery process can lapse or be withdrawn, and how I will record expiry given the known interface gap (7.18).

OUTPUT: EXECUTION AND REALIZATION PACKET

1. An action-binding table: action, function or procedure, number of separately failing steps, and the evidence source for each.
2. An outcome table per action in the style of Table 7.3: what counts as succeeded, failed cleanly, indeterminate, and completed-invalid, and the ϵ status each should produce.
3. A Layer-1 transition specification: for each outcome class, the changes to actual state and the invariants checked.
4. An invalidation register: each fact whose change invalidates authority, and how the change is detected.
5. A race register: each action's exposure between authorization and effect, and the host design measures that reduce it.
6. A boundary statement for my deployment in the style of Table 7.5.
7. If the runtime package and my Chapter 6 program are attached: the program extended to execute, a Layer-1 transition service, and a test for each outcome class, for stale authority, and for single use of an authorization. Run nothing you cannot run; tell me exactly how to run it.
8. Self-check: each common error in Section 7.19, whether the packet (and code) avoids it, and anything you corrected.

Begin with step 1.

Reviewing the output. Before relying on the result, check:

- Every clean-failure classification names the evidence that nothing material happened.
- No state update is derived from a return value; each outcome class has a Layer-1 transition, including failures.
- Unknown outcomes are carried into actual state until reconciled.
- The boundary statement keeps framework capability, decision-core behavior, the supervision layer, host duties, and prospective work separate.
- If code was produced: it uses the runtime lineage end to end, writes nothing into runtime internals, runs against the unmodified frozen runtime, and its tests pass.

A.8 Chapter 8 Prompt — Build the Application

Purpose. Turn the modeler's packets into an application plan: where each modeled item lives in the runtime, what every piece of host code supplies, how perception, the store, intake, reconciliation, and remedies work, and which scores may be trusted—and, when the runtime is available, build and test the application.

Attach. This Guide (Chapters 1–8 at minimum) and the packets from prompts A.3–A.7. Optionally, the frozen runtime package (runtime-v2.1.167, v2.1.167) and the programs from the Chapter 6 and 7 prompts, which enable the build step.

Produces. An Application Implementation Packet with its implementation map, boundary register, rule-or-consequence table, perception, intake, reconciliation, traceability, and scalar registers, a day-in-the-life scenario, and a self-check against Section 8.17; with the runtime attached, also the application and its tests.

Prompt

You are helping me turn my PV-PP model into an application, using Chapter 8 of the attached PV-PP Programmer's Guide. My packets from the Chapter 3–7 prompts are attached. If the frozen PV-PP runtime package (runtime-v2.1.167, v2.1.167) and my earlier programs are also attached, you will finish by building the application, modeled on `coffee_app.py` in Section 8.10.

GROUND RULES

- Treat the attached PV-PP Programmer's Guide as your authority for PV-PP terms. If the Guide does not settle a question, say so; do not invent PV-PP semantics, operators, or state objects.
- Interview me one topic at a time. Ask no more than three questions per turn, in plain language. Do not require me to use PV-PP vocabulary; translate my answers yourself and show me the translation.
- Never invent facts about my domain. If you need a fact, ask for it. If I do not know, record it as an open assumption.
- When you propose a classification, give the reason and a confidence label (high, medium, or low).
- Keep three things visibly separate: what I told you, what you inferred, and what is still unknown.
- Before giving your final output, check it against the Guide's common-error list for this chapter and tell me what you corrected.

Additional rules for this chapter:

- For every piece of host code you propose, say whether it states a fact or decides something. Anything that decides belongs to the runtime (8.2).
- For every binding rule and every harm, decide with me whether it is a prohibition (a constraint) or a consequence (a projection), and record the reason (8.4).
- Evidence has an age. Ask me how long each kind of report stays current (8.5).
- Outside systems report facts or enforce effects. They never decide, relax, or authorize (8.7).
- Do not propose a scalar score as a decision shortcut unless every field of Table 8.3 can be filled for the episode. Otherwise label it diagnostic.
- If you write code: call the runtime for every stage and every authority; never write to runtime-private state; do not change the runtime.

INTERVIEW SEQUENCE

1. Mapping: walk through my packets and place each item in the runtime structure of Table 8.1. Flag anything with no home (8.1).
2. Registration: confirm the domains, thresholds, the required "steady" baseline and what it means in my system, the constraint rules and classes, the Graph, and the tie order (8.3).
3. Rule or consequence: for each binding rule and each harm my system can cause, classify it and give

the reason (8.4).

4. Perception: which evidence forms each perceived fact, how old each may be before it stops counting as current, and what the perceived value is when it is stale (8.5).
5. Store and transition: where actual state is kept, what every recorded change looks like, and how each realized effect updates it (8.6).
6. Intake and dependencies: which outside systems report which facts, and which decisions depend on which facts (8.7).
7. Reconciliation and remedy: which outcomes can be unknown and what evidence resolves them; which harms need a governed remedy, and which transformation carries it (8.8).
8. Traceability: what I will record from the first version (8.9).
9. Scores: any numbers I use to rank or prioritize, and whether each can be certified or stays diagnostic (8.12).

OUTPUT: APPLICATION IMPLEMENTATION PACKET

1. An implementation map in the style of Table 8.1, with every packet item placed.
2. A boundary register: each host function or adapter method, what fact it supplies, and a confirmation that it decides nothing.
3. A rule-or-consequence table with reasons.
4. A perception and evidence-freshness specification.
5. An intake and dependency register: report sources, facts changed, and the decisions that depend on them.
6. A reconciliation and remedy plan.
7. A traceability plan.
8. A scalar register: each score, its mode (shortcut, verification, diagnostic, gating required, block), and for any shortcut a completed certificate.
9. A day-in-the-life scenario in the style of Table 8.2, with the stage expected to decide each episode.
10. If the runtime package and my earlier programs are attached: the application and a test file covering every episode of the scenario. Run nothing you cannot run; tell me exactly how to run it.
11. Self-check: each common error in Section 8.17, whether the packet (and code) avoids it, and anything you corrected.

Begin with step 1.

Reviewing the output. Before relying on the result, check:

- Every host function is listed with the fact it supplies, and none of them selects, filters, ranks, or authorizes.
- Each rule and harm is classified as a prohibition or a consequence, with a reason that would convince a domain expert.
- Every perceived fact has a freshness limit and a defined stale value.
- Remedies are governed actions in the Graph, not host routines.
- Any score used as a shortcut has a complete, episode-local certificate; all others are labeled diagnostic.
- If code was produced: it runs against the unmodified frozen runtime, writes nothing into runtime internals, and its tests pass.

A.9 Chapter 9 Prompt — Plan Testing and Evidence

Purpose. Plan how the modeler's application will be tested and what may be claimed from the results: bounded claims, boundary and invariant tests, hostile cases, false-success conditions,

scenario benchmarks with strong comparators, reproducibility and regression, and a limitations statement—and, when the runtime is available, write the tests.

Attach. This Guide (Chapters 1–9 at minimum) and the packets from prompts A.3–A.8. Optionally, the frozen runtime package (runtime-v2.1.167, v2.1.167) and the application from the Chapter 8 prompt, which enable the test-writing step.

Produces. A Test and Evidence Plan with its claim register, boundary tests, invariants, hostile-case register, trace and false-success specification, benchmark designs, reproducibility plan, limitations statement, and a self-check against Section 9.18; with the runtime attached, also the test files.

Prompt

You are helping me plan how to test my PV-PP application and what I may claim from the results, using Chapter 9 of the attached PV-PP Programmer's Guide. My packets from the Chapter 3–8 prompts are attached. If the frozen PV-PP runtime package (runtime-v2.1.167, v2.1.167) and my application are also attached, you will finish by writing the tests, modeled on test_coffee_hostile.py in Section 9.12.

GROUND RULES

- Treat the attached PV-PP Programmer's Guide as your authority for PV-PP terms. If the Guide does not settle a question, say so; do not invent PV-PP semantics, operators, or state objects.
- Interview me one topic at a time. Ask no more than three questions per turn, in plain language. Do not require me to use PV-PP vocabulary; translate my answers yourself and show me the translation.
- Never invent facts about my domain. If you need a fact, ask for it. If I do not know, record it as an open assumption.
- When you propose a classification, give the reason and a confidence label (high, medium, or low).
- Keep three things visibly separate: what I told you, what you inferred, and what is still unknown.
- Before giving your final output, check it against the Guide's common-error list for this chapter and tell me what you corrected.

Additional rules for this chapter:

- Every claim must name its kind of evidence (Table 9.1) and what that evidence does not establish.
- Write acceptance criteria and false-success conditions before any result is discussed.
- Propose strong comparators, including ones that might match my application. Never design a comparator to lose.
- Keep three columns separate: framework claims, runtime-version claims, and claims about my application (9.16).
- A hostile case must target a specific failure (9.4). Say which one.
- If you write tests: run them against the unmodified frozen runtime, and include a runtime identity check.

INTERVIEW SEQUENCE

1. Claims: the few bounded statements I want the tests to support, each in one sentence that a result could contradict (9.7).
2. Boundaries: which category boundaries of Table 9.2 my model depends on, and a case that separates each (9.2).
3. Invariants: the rules that must hold in every case, stated once (9.3).
4. Hostile cases: for semantic laundering, stale or forged authority, incomplete state, missing recovery, hollow success, and downstream repair, whether each applies to my system and what hostile implementation would test it (9.4).
5. Failure surfaces and false success: what my trace records, and the success and governing-failure

conditions for each scenario (9.5, 9.6).

6. Scenario benchmarks: controls, perturbations, observables, comparators, and acceptance criteria (9.7, 9.8).
7. Reproducibility and regression: what I will bind to every run, what I will bank, and how I will detect a changed runtime (9.9, 9.10).
8. Replay: whether I need to reconstruct past decisions, and what must be recorded at decision time (9.11).

OUTPUT: TEST AND EVIDENCE PLAN

1. A claim register: each claim, its kind of evidence, and what that evidence will not establish.
2. A boundary test table in the style of Table 9.2.
3. An invariant list, each with the cases that will check it.
4. A hostile-case register: failure targeted, hostile implementation, and expected result (refused or flagged).
5. A trace specification and false-success definitions.
6. Scenario benchmark designs with predeclared acceptance criteria and comparators.
7. A reproducibility and regression plan, including a runtime identity check.
8. A limitations statement written before results, with framework, runtime, and application claims in separate columns.
9. If the runtime package and my application are attached: the test files, run against the unmodified frozen runtime. Run nothing you cannot run; tell me exactly how to run it.
10. Self-check: each common error in Section 9.18, whether the plan (and tests) avoid it, and anything you corrected.

Begin with step 1.

Reviewing the output. Before relying on the result, check:

- Every claim names its evidence and states what that evidence does not establish.
- False-success conditions and acceptance criteria were written before any results.
- Comparators are strong enough that the PV-PP framework could fail to beat them.
- Each hostile case targets a named failure and is expected to be refused or flagged.
- The limitations statement keeps framework, runtime, and application claims apart.
- If tests were produced: they include a runtime identity check, run against the unmodified frozen runtime, and pass.

A.10 Chapter 10 Prompt — Build a Worked Example in Your Domain

Purpose. Produce a complete worked example of the modeler's own system at the depth of the Chapter 10 examples: every step from boundary to re-entry, a sequence of episodes that exercises the system's hardest distinctions, and a clear statement of what stays with the domain—and, when the runtime is available, code it.

Attach. This Guide (Chapters 1–10 at minimum) and, if available, the packets from prompts A.3–A.9. Optionally, the frozen runtime package (runtime-v2.1.167, v2.1.167), which enables the coding step.

Produces. A Worked Example Packet with its eight-step walk-through, stage table, cross-domain row, domain-ownership statement, and a self-check against Section 10.8; with the runtime attached, also runnable code and tests.

Prompt

You are helping me build a complete worked example of my own system, using Chapter 10 of the attached PV-PP Programmer's Guide as the pattern. My packets from the Chapter 3–9 prompts are attached if I have them; if not, we will build the essentials as we go. If the frozen PV-PP runtime package (runtime-v2.1.167, v2.1.167) is also attached, you will finish by coding the example, modeled on `agent_ops.py` in Section 10.5.

GROUND RULES

- Treat the attached PV-PP Programmer's Guide as your authority for PV-PP terms. If the Guide does not settle a question, say so; do not invent PV-PP semantics, operators, or state objects.
- Interview me one topic at a time. Ask no more than three questions per turn, in plain language. Do not require me to use PV-PP vocabulary; translate my answers yourself and show me the translation.
- Never invent facts about my domain. If you need a fact, ask for it. If I do not know, record it as an open assumption.
- When you propose a classification, give the reason and a confidence label (high, medium, or low).
- Keep three things visibly separate: what I told you, what you inferred, and what is still unknown.
- Before giving your final output, check it against the Guide's common-error list for this chapter and tell me what you corrected.

Additional rules for this chapter:

- Follow the sequence of Table 10.1 exactly, and give each step the depth the Chapter 10 examples give it.
- Tell me which Chapter 10 example my system most resembles, and why. Use it as a pattern, never as a source of facts or numbers.
- Keep the enabling resource separate from the capability, and frame functions, not units (10.8).
- Name everything that stays with my domain—measurements, mechanisms, authoritative sources, binding rules, professional standards—and any disclaimer my domain requires (10.7).
- If you write code: call the runtime for every stage, use its refresh and admission operations for changes to the represented world, and do not change the runtime.

INTERVIEW SEQUENCE

1. Boundary and entities: the system, the episode, and whether any group involved qualifies as a collective entity (10.3).
2. State and capability: the governing domains, their horizons, the capabilities, and the resources that enable them.
3. Perception and evidence: sources, their authority and scope, freshness limits, and any commanded-versus-measured distinctions (10.4).
4. Objectives: what I want, kept separate from what is possible.
5. Change and reachability: the paths, what makes each reachable, which could be refreshed by evidence, and which would need admission as new structure (10.5).
6. A sequence of three to six episodes that exercises my system's hardest distinctions, including at least one change that re-enters governance and one outcome that is unknown at first.
7. For each episode: the stage expected to decide it, what is attempted, and what is realized.

OUTPUT: WORKED EXAMPLE PACKET

1. A narrative walk-through in the eight steps of Table 10.1.
2. A stage table in the style of Tables 10.2–10.5.
3. A row for my system in the style of Table 10.6.
4. A domain-ownership statement: what the PV-PP framework supplies, what my domain supplies, and required disclaimers.
5. If the runtime package is attached: runnable code for the episode sequence and a test for each

episode's expected decision. Run nothing you cannot run; tell me exactly how to run it.

6. Self-check: each common error in Section 10.8, whether the example (and code) avoids it, and anything you corrected.

Begin with step 1.

Reviewing the output. Before relying on the result, check:

- Every step of Table 10.1 is present and at comparable depth.
- Resources and capabilities are distinct, and every framing target is a function.
- At least one episode re-enters governance, and at least one outcome is unknown when first attempted.
- Nothing is borrowed from a Chapter 10 example as a fact about my system.
- The domain-ownership statement names the authoritative sources and any required disclaimers.
- If code was produced: it runs against the unmodified frozen runtime and its tests pass.

A.11 Chapter 11 Prompt — Position Your Application and State Its Claims

Purpose. State accurately what the modeler's application is, how it compares with the strongest established alternative, which scores may be trusted, and which claims are proved, demonstrated, or prospective.

Attach. This Guide (all chapters) and, if available, the packets and test plan from prompts A.3–A.10.

Produces. A Positioning and Claims Statement with a positioning paragraph, a three-column claims table, a comparison with the strongest alternative, a scalar statement, a limitations paragraph, and a self-check against Section 11.11.

Prompt

You are helping me position my PV-PP application and state its claims accurately, using Chapter 11 of the attached PV-PP Programmer's Guide. My packets and test plan from the earlier prompts are attached if I have them.

GROUND RULES

- Treat the attached PV-PP Programmer's Guide as your authority for PV-PP terms. If the Guide does not settle a question, say so; do not invent PV-PP semantics, operators, or state objects.
- Interview me one topic at a time. Ask no more than three questions per turn, in plain language. Do not require me to use PV-PP vocabulary; translate my answers yourself and show me the translation.
- Never invent facts about my domain. If you need a fact, ask for it. If I do not know, record it as an open assumption.
- When you propose a classification, give the reason and a confidence label (high, medium, or low).
- Keep three things visibly separate: what I told you, what you inferred, and what is still unknown.
- Before giving your final output, check it against the Guide's common-error list for this chapter and tell me what you corrected.

Additional rules for this chapter:

- Label every claim as proved, demonstrated, or prospective (11.1), and attach its conditions or evidence.
- Use the proof program's own wording for scalar claims (11.4). Never state containment without its conditions or non-reduction as universal.

- Compare my application with the strongest established method that could address the same decision, and say plainly if that method may be sufficient (11.6).
- Keep domain science, security controls, and professional standards on the domain's side of the line (11.7, 11.8).

INTERVIEW SEQUENCE

1. The decision: what my application decides, and which established methods someone might otherwise use for it.
2. Active distinctions: which of the conditions in Section 11.6 are active in my decisions, with an example of each.
3. Sufficiency: whether any condition under which an established method would be sufficient applies (11.6).
4. Scores: any score I use or might use, whether its decisions could ever meet the containment conditions of Table 11.1, and therefore whether it can be certified or must stay diagnostic (8.12, 11.2).
5. Evidence: what my tests and benchmarks demonstrated, and what they did not (Chapter 9).
6. Open questions: which open questions of Section 11.10 touch my application.

OUTPUT: POSITIONING AND CLAIMS STATEMENT

1. A one-paragraph positioning statement in the style of Section 11.1.
2. A claims table with three columns—proved, demonstrated, prospective—in the style of Table 11.3, for my application.
3. A comparison with the strongest established alternative in the style of Table 11.2, including when it would suffice.
4. A scalar statement: each score, whether it can be certified, and the exact wording I may use about it.
5. A limitations and open-questions paragraph.
6. Self-check: each common error in Section 11.11, whether the statement avoids it, and anything you corrected.

Begin with step 1.

Reviewing the output. Before relying on the result, check:

- Every claim is labeled proved, demonstrated, or prospective, with its conditions or evidence.
- Scalar claims use the proof program's bounded wording; nothing is stated as universal.
- The comparison names a strong alternative and says when it would be sufficient.
- Nothing is claimed about the frozen runtime that it does not implement.
- Domain science, security controls, and professional standards remain the domain's responsibility.

A.12 Chapter 12 Prompt — Plan Supervised Execution

Purpose. Decide which of the modeler's actions need supervision while they run, which monitors may report what, which facts each execution depends on, how each stop is carried out and evidenced, and how effects are reconciled—and, when the runtime is available, extend the Chapter 7 or 8 program to supervise one action.

Attach. This Guide (Chapters 1–8 and 12 at minimum) and the packets from prompts A.3–A.8. Optionally, the frozen runtime package (runtime-v2.1.167, v2.1.167) and the Chapter 7 or 8 program, which enable the code step.

Produces. A Supervision Packet with its action, source, dependency, control, effect, and recovery registers, a boundary statement, and a self-check against Section 12.14; with the runtime attached, also the extended program and its tests.

Prompt

You are helping me plan how my model governs actions while they run, using Chapter 12 of the attached PV-PP Programmer's Guide. My packets from the Chapter 3–8 prompts are attached. If the frozen PV-PP runtime package (runtime-v2.1.167, v2.1.167) and my Chapter 7 or 8 program are also attached, you will finish by extending that program to supervise one action, modeled on `supervised_failover.py` in Section 12.11.

GROUND RULES

- Treat the attached PV-PP Programmer's Guide as your authority for PV-PP terms. If the Guide does not settle a question, say so; do not invent PV-PP semantics, operators, or state objects.
- Interview me one topic at a time. Ask no more than three questions per turn, in plain language. Do not require me to use PV-PP vocabulary; translate my answers yourself and show me the translation.
- Never invent facts about my domain. If you need a fact, ask for it. If I do not know, record it as an open assumption.
- When you propose a classification, give the reason and a confidence label (high, medium, or low).
- Keep three things visibly separate: what I told you, what you inferred, and what is still unknown.
- Before giving your final output, check it against the Guide's common-error list for this chapter and tell me what you corrected.

Additional rules for this chapter:

- A monitor, adapter, or supervisor never decides. Anything that would choose, relax, or authorize an action belongs to a new decision cycle (12.1).
- Never say the runtime stops a process. It records a governed stop request; my controller carries it out; the effect is reconciled separately (12.8).
- Never propose running work that may need to be stopped inside native call-through; after call entry the runtime cannot checkpoint it (12.13).
- Every dependency must name an exact fact: configuration, subject, fact kind, and fact identity. Do not infer dependencies.
- If you write code: use the supervision services, never the store's tables or private helpers; never supply a continuation posture; never write to runtime-private state. Do not change the runtime.

INTERVIEW SEQUENCE

1. Actions: which of my actions take long enough, or have effects severe enough, that a change while they run must be governed, and how each is carried out (12.3, 12.13).
2. Sources: which monitors or systems report facts those actions depend on, what each may report on, how its reports are authenticated, and which systems report effects (12.4).
3. Dependencies: for each supervised action, the exact facts whose change should be governed against it (12.5).
4. Stops: for each action, which controller stops it, the controller's own commands for hold, pause, cancel, and terminate, and how the controller attests what it did (12.7).
5. Effects: what evidence shows what the world did after a stop, how unknown effects are resolved, and whether the action is idempotent (12.8).
6. Ownership and recovery: who supervises, how ownership passes, and what must be reconciled before supervision resumes after a crash (12.7, 12.9).

OUTPUT: SUPERVISION PACKET

1. An action register: each supervised action, how it is carried out, and why it needs supervision.
2. A source register: each source, its roles, fact kinds, subjects, permitted uses, and how its reports are

verified.

3. A dependency register: each supervised action and the exact facts it depends on.
4. A control register: each controller, its supported controls and commands, and its attestation method, with the postures under which each stop is permitted (Table 12.2).
5. An effect and retry register: effect evidence sources, unresolved states, and idempotency contracts.
6. A recovery profile for each supervised action.
7. A boundary statement in the style of Table 7.5.
8. If the runtime package and my program are attached: the program extended to supervise one action, and tests for a governed stop, a refused stop of a healthy action, a stale owner, an unregistered source, and enforcement kept apart from effect. Run nothing you cannot run; tell me exactly how to run it.
9. Self-check: each common error in Section 12.14, whether the packet (and code) avoids it, and anything you corrected.

Begin with step 1.

Reviewing the output. Before relying on the result, check:

- No monitor, adapter, or supervisor in the packet chooses, relaxes, or authorizes anything.
- Every dependency names an exact fact, and every supervised action is carried out outside native call-through.
- Every stop has a controller, its own command, and an attestation method, and no stop is treated as an effect.
- Unknown effects stay unresolved until evidence resolves them, and retries rest on an idempotency contract or evidence of no effect.
- If code was produced: it uses the supervision services end to end, writes nothing into runtime internals or store tables, runs against the unmodified frozen runtime, and its tests pass.

Appendix B — Notation, Symbols, and Glossary

This appendix collects the notation and fixed vocabularies used in the Guide, with the chapter where each is developed. Symbols follow the V2.1 owner specifications; where the runtime uses a different spelling for the same object, the runtime name is given. Definitions here are reminders, not replacements: the chapter cited, and the owning specification behind it, controls.

B.1 State, Perception, and Objectives

Table B.1 — State, perception, and objectives

Symbol	Name	Meaning	Ch.
i, t	Actor or entity; time	Whose state, at which decision or transition time	2
S_i(t)	Actual persistent state	What presently exists for i: {PP, SPV, AVS, X, ...}; host-owned	2, 3
PP_i(t)	Productive Power	Actual capability under stated conditions to produce or preserve PV	1, 2
SPV_i(t)	Stored Productive Value	Realized PV retained for later use, transfer, or conversion	2
AVS_i(t)	Actual viability state	Where i actually stands against its governing conditions	2
X_i(t)	Other typed state	Resources, obligations, standing, environment, and context	2, 3
PV	Productive Value	Value actually transferred, delivered, realized, or imposed through interaction	1, 2
P_i(t)	Perceived decision state	The decision-time representation of relevant state, formed through licensed informational pathways	2, 4
PPP_i(t)	Perceived Productive Power	Perceived capability; one component of P _i (t), which may diverge from PP	2, 4
O_i(t)	Objectives	What i wants, carried beside P _i (t); never a source of reachability, feasibility, or authority	2, 6
B_i(t)	Realized PV bundle	The PV consequences of an interaction that actually occurred	2, 7
T_i	Layer-1 transition	The host-owned update $S_i(t+1) = T_i(S_i(t), B_i(t), \dots, \text{ExecPath}_i(t))$	7

B.2 The Decision Stages

Table B.2 — The canonical decision stages and runtime additions

Symbol	Stage	Question it answers	Ch.
Φ	Pressure Field	How strongly is each domain demanding correction now?	6

Symbol	Stage	Question it answers	Ch.
H	Collapse Horizon	Under baseline continuation ("steady"), how long until each domain fails?	6
G	Governing Domain Identification	Which domains must be preserved for recoverable viability?	6
$\mathcal{R}$	Regime Classification	Which posture—Mission, Stabilization, Survival, Existential—applies?	6
Graph / seed	Graph and seed substrate	Which transformations and paths are represented and reachable?	5, 6
Π	Policy construction	Which bounded candidate policies can be built from the seeds?	6
Π completeness	Completeness validation	Are all materially required policy classes present?	6
—	Constraints	Which candidates are executable under hard, conditional, and soft rules?	6
—	Domain Framing	What preservation target, stated as a function, are candidates judged against?	6
—	Joint recovery feasibility (runtime)	Can active recovery obligations still be met from shared capacity? A runtime stage serving Adequacy, not an owner-stack operator	6
—	Adequacy	Does the policy open a recovery corridor for every governing domain?	6
$Q_t(\pi)$	Shared projection	What each candidate is projected to do; consumed by Adequacy and Σ , not a stage	6
Σ	Selection	Which adequate policy is selected, by governing non-domination, global dominance, and a tie order?	6
ϵ	Execution transition	How is the selected policy realized as a bounded, monitored episode?	7

B.3 Execution and Realization Objects

Table B.3 — Execution and realization objects

Object	Meaning	Ch.
$\pi_{\text{sel}}(t)$	The policy Σ selected; a decision, not an event	7
$x_{\text{exec}}(t)$	The instantiated execution episode	7
$E_{\text{exec}}(t, \tau)$	The bounded state within execution	7
$\text{ExecPath}(t)$	What actually occurred within the licensed corridor	7
$\text{status}_{\text{exec}}(t)$	The auditable terminal status of the episode (Table B.4)	7

Object	Meaning	Ch.
B_real(t)	The realized Productive-Value bundle or staged sequence	7
info_exec(t)	Execution-generated information for authorized update surfaces	7, 8
Execution license	Runtime record that ε may be entered for the selected policy; built only from a completed cycle	7
Native authorization	Single-use, ledger-backed authority for one attempt of one action through one binding version	7
Execution binding	Registration of a host function for an action; representation, not authority	7
Layer-1 handoff	The terminal ε packet given to the host transition; contains no next state	7
Transition provenance	Audit record linking prior state, episode, and next state; confers no authority	7, 8
Active execution	Supervisory record binding one canonical execution and its authorization; confers no authority	12
Continuation assessment	A checkpoint's answer: the posture, and whether unused authority must be fenced	12
Control request; enforcement record	A governed request to stop, and the controller's attested report; neither is an effect	12
World-effect reconciliation	What authoritative evidence shows the world did; unknown and partial effects stay unresolved	12

B.4 Fixed Vocabularies

Table B.4 — Values used across the Guide

Vocabulary	Values	Ch.
Regime	Mission; Stabilization; Survival; Existential	6, 10
Constraint class	Hard (removes a policy); conditional (binding, relaxable only under the stated necessity rule); soft (never removes a policy)	6
Transformation family	Continuation; maintenance / local adjustment; corrective / repair; structural reconfiguration; exit / transfer / liquidation; substitution; release / restructuring; escape / emergency	5
Reachability status	Reachable; unreachable; uncertain (with a required note)	5
Graph construction result	PASS; FAIL; CONDITIONAL PASS	5
Capability-change kind	Gained; lost; increased; decreased; reconfigured; uncertain	5

Vocabulary	Values	Ch.
Information disposition	Current; rejected; superseded; defective; prohibited for use; historically valid but noncurrent; unresolved; contradicted	4
Σ selection mode (license)	Standard; recovery-unavailable fallback; terminal infeasibility (entry blocked)	6, 7
Call-through outcome	Succeeded; failed cleanly; indeterminate; completed but invalid	7
ϵ terminal status	Completed; completed_staged; partial_realization; aborted_return; failed; emergency_return	7
Control state (pre-call)	Ready; paused; aborted; reauthorization required	7

B.5 Graph Construction Failure Classes

Table B.5 — Graph failure codes

Code	Failure	Meaning
G-001	Taxonomy violation	A transformation is unclassified or outside the eight families
G-002	Coverage failure	A materially required family is missing
G-003	False structural reachability	An unrealizable transformation was admitted
G-004	Unverified exclusion	A transformation was left out without a recorded reachability basis
G-005	Missing multi-step path	A required staged recovery was never constructed
G-006	Composition collapse	Depth truncation removed the only viable corridor
G-007	Explosion	Equivalent variants were not reduced
G-008	Coverage loss in reduction	Distinct policy classes were merged into one seed
G-009	Graph- Π boundary violation	Π invented structure, or the Graph constructed policies
G-010	Execution-order violation	Construction steps ran out of order

B.6 Scalar-Reduction Terms

Table B.6 — Terms from the scalar-reduction results

Term	Meaning	Ch.
Containment (Direction 1)	On a certified subclass, an unweighted average's top set equals Σ 's Stage 1 frontier; proved locally only	11
Non-reduction (Direction 2)	On stated classes, no scalar faithfully represents staged selection; strong but conditional	11
SC-Geo	Collinearity: represented consequences lie on one line	8, 11
SC-GDM	Governing-domain monotonicity along that line	8, 11

Term	Meaning	Ch.
SC-X	Attainment: some admissible candidate reaches the line's best point	8, 11
$K_2 > 0$	Full-vector alignment for the unweighted average	8, 11
SC-S2	Two frontier policies tie on every governing domain but differ elsewhere	11
Stage conflation	A score that separates policies Stage 1 cannot tell apart	11
Scalar-shortcut certificate	An episode-local, revocable record that a score may compress selection	8

B.7 Glossary

Table B.7 — Glossary

Term	Meaning	Ch.
Adequacy	Whether a policy opens a real recovery corridor for every governing domain; conjunctive, a filter not a ranking	6
Checkpoint	A supervised governance step for a running execution; produces a continuation posture	12
Collective entity	A group that independently qualifies as a productive entity; its PP is not the sum of members' PP	3, 10
Configuration-dependent capability	Capability that exists only when several components are jointly present and compatible	3, 10
Continuation posture	continue, reauthorization_required, abort_return, emergency_return, or indeterminate	12
Controlled admission	The governed path by which a genuinely new transformation or function enters the model	5, 10
Corridor	A region or path in which governing conditions remain jointly satisfied; descriptive, not a primitive	2, 11
Domain Framing	Fixing the preservation target as a function, so substitutes that preserve it can count	6
Enabling resource	Something that helps realize a capability without being the capability	3, 10
Evidence freshness	How long a report counts as current; stale evidence is not current evidence	4, 8
False success	A visible success coexisting with a predeclared governing failure	9
Fan-out	Finding every running execution that declared a dependency on a changed fact	12
Fence (supervisory)	The ownership generation that makes a stale supervisor's requests refusable	12

Term	Meaning	Ch.
Governing domain	A domain whose preservation is required for recoverable viability in this decision	6
Graph	The represented structure of instances, transformations, and paths from which seeds are drawn	5
Hostile case	An implementation built to violate a rule, to show the violation is refused or visible	9
Indeterminate outcome	An attempt whose external effect cannot be ruled out	7
Information governance	Keeping source authority, applicability, permitted use, disposition, confidence, and time separate	4
Inheritance	What a new actor receives from the one that created it: configuration and information, permissions only by declaration, never current PP	5
Layer 1 / Layer 2	Layer 1: actual state and its transition. Layer 2: the decision architecture	2, 7
Preservation object	What the decision must preserve or recover, stated as a function	5, 6
Re-entry	Returning to governance at the earliest stage a change invalidated	5, 6, 8
Reachability	Whether a represented transition can actually be attempted from the current state; not capability, desirability, or permission	5
Recovery corridor	A projected path by which a governing domain is preserved until recovery becomes reachable	6
Replay sufficiency	Whether a past decision can be reconstructed from what was recorded at the time	4, 9
Seed	A structurally distinct policy class handed from the Graph to Π	5
Supervision layer	Runtime v2.1.167's durable services around the unchanged decision core	7, 12
Transfer History	An event record kept separate from current state, memory, and PP	4
Transition-relevant state sufficiency	Current state contains every presently material consequence that can change transitions or governance	2, 4
Viability	Operating within the governing conditions that matter for the modeled system	2

Appendix C — Modeling Checklist

This checklist consolidates the three definition packets of Chapters 3 through 5—scenario and model, information and evidence, change and reachability—into one review. Use it before any code is written, and again when prompts A.3 through A.5 have produced their packets. Each item should be answerable with a recorded answer, not a nod; an item that cannot be answered is an open assumption and should be listed as one. Section numbers point to the text that explains each item.

C.1 Scenario and Model (Chapter 3)

- Is the scenario bounded: start, end, exclusions, horizon, and what one episode means? (3.1)
- Was the interaction discovered before any decision rule or score was written? (3.2)
- Is each participant's role stated, and is every group treated as an entity shown to qualify as one? (3.3, 3.11)
- Is each item classified by role—PV, SPV, PP, enabling resource, or broader state—rather than by name? (3.4)
- Is actual state defined before perceived state, and is each perceived fact traced to how it is formed? (3.5)
- Are possession, access, permission, control, and operability kept separate wherever they can diverge? (3.6)
- Is every configuration-dependent or jointly required capability recorded with all its components? (3.7)
- Are objectives recorded with identity, scope, lifecycle, and source, and kept out of reachability and feasibility? (3.8)
- Are constraints, thresholds, governing domains, and corridors identified, with each constraint's class? (3.9)
- Is recovery modeled as structure—reserve, capability, mechanism, prerequisites, reachability—rather than hope? (3.10)
- Is the execution boundary stated: which effects are external, what counts as completion evidence, and what Layer 1 owns? (3.12)
- Is it stated what can change and how the model will learn of each change? (3.13)
- Are history, current state, and replay obligations kept separate? (3.14)
- Are nested interactions that can fail separately represented separately? (3.2, 3.16)

C.2 Information and Evidence (Chapter 4)

- For each information item: is its class recorded—observation, memory, expectation, record, inference? (4.2)
- Are source and provenance preserved with identity and version? (4.2)
- Is confidence kept separate from authority, and is each source's authority stated with its scope? (4.4)
- Are applicability and permitted use recorded—which decisions and which stages may use the item? (4.5)
- Does every item carry a disposition, and does supersession preserve the earlier record? (4.6)
- Where timing matters, are valid time and record or knowledge time both recorded? (4.7)
- Is memory kept separate from Transfer History, and history separate from current state? (4.8, 4.9)
- Is every still-material consequence of a past event carried into current state? (4.10)

- Is it stated what must be retained for replay, and that later knowledge will not be inserted into it? (4.11, 4.12)
- Can information change state, capability, or reachability only through the governed path? (4.13)
- For each item, is it recorded which artifacts depend on it and where governance resumes if it changes? (4.14)
- Does each perceived fact have a freshness limit and a defined value when stale? (4.4, 8.5)

C.3 Change and Reachability (Chapter 5)

- Is the preservation object stated as a function, not a mechanism? (5.3)
- Are domains, instances, and transformations kept distinct, with transformation rules on transformations? (5.2)
- Was every relevant instance scanned across all eight families, with exit and substitution explicitly checked? (5.4)
- Does every transformation have a reachability status—reachable, unreachable, or uncertain—with its evidence, and a note for uncertainty? (5.5)
- Is permission recorded as a condition for Constraints rather than as a reachability value? (5.5, 5.6)
- Are difficult transformations kept, with costs recorded, and unrealizable ones left out? (5.5)
- Are configuration, access, and compatibility conditions recorded without preferences or adequacy judgments? (5.6)
- Are the staged paths that recovery or exit requires constructed, including bridges? (5.7)
- Were seeds reduced without merging retain/sell, repair/substitute, direct/staged, continue/release, or mechanism/function? (5.8)
- Did the Graph receive PASS, FAIL, or CONDITIONAL PASS, with failures named by code (Table B.5)? (5.8)
- Are recovery routes represented with their prerequisites and reachability? (5.9)
- Is there a closure watch list of windows, deadlines, and degradations, with possible reopenings? (5.10)
- Is it stated which reachability can refresh from evidence and how new structure would be admitted? (5.11)
- Does capability change follow intention → action → realized transition → evidence → current PP? (5.12)
- Where one actor can create, copy, or replace another, is it recorded what the new actor inherits, with permissions passed only by declaration and current PP never copied? (5.12)
- Is there an invalidation map: for each kind of change, what becomes stale and where governance resumes? (5.13)

C.4 Consistency Across the Packets

- Does every governing domain have a threshold, a way of measuring its margin, and an evidence source?
- Does every transformation's enabling condition refer to something recorded in state or evidence?
- Does every perceived fact used by a constraint or projection have a source, an authority, and a freshness limit?
- Does every evidence dependency name the stage at which governance resumes?
- Is every objective absent from every reachability basis and every constraint?

- Is every external effect paired with the evidence that will establish it and the Layer-1 change it produces?
- Is every open assumption listed in one place, with who can resolve it?

C.5 Errors to Rule Out

Each chapter ends with the errors most likely at its stage. Before signing off, confirm that the model avoids each one.

Table C.1 — Modeling errors to rule out

Stage	Errors to rule out
Scenario and model (3.18)	Starting from a reward or utility; treating every useful resource as PP; treating PP as the whole state; equating possession, access, permission, control, and operability; summing member PP; treating an objective as evidence of capability or reachability; treating call success as realization; treating history as current truth; inventing domain weights; letting code supply missing semantics
Information and evidence (4.18)	Treating confidence as authority or relevance as permission; assuming the newest record governs; overwriting superseded evidence; treating Transfer History as current state or PP; dropping a still-material consequence; substituting replay evidence for current state; using later knowledge in replay; letting information create capability, reachability, or authority directly; invalidating everything instead of following dependencies
Change and reachability (5.18)	Generating options from intuition; framing a mechanism; putting transformation rules on domains or instances; treating reachability as capability, desirability, or permission; excluding the difficult; admitting the imaginary; encoding preferences in configuration; truncating paths; merging distinct seeds; letting Π or the application invent transformations; treating projected closure as realized; admitting routes or tools without control; counting desired capability as current; recomputing everything after a change

C.6 Sign-Off

- The packets from prompts A.3, A.4, and A.5 are attached, or their equivalents written by hand.
- A domain expert has reviewed every classification the AI assistant proposed, not only the summary.
- Every open assumption is listed with its owner.
- The model has been walked through at least one ordinary case and one failing case at the depth of Sections 3.16, 4.16, and 5.15.

Appendix D — Implementation Checklist

This checklist covers Chapters 6 through 8 and 12: configuring the decision cycle, crossing the execution boundary, building an application around the frozen runtime, and supervising execution while it runs. Use it when prompts A.6 through A.8 and A.12 have produced their packets and again before code is trusted. As in Appendix C, each item needs a recorded answer, and the section number points to the explanation. Exact constructors and fields belong to the PV-PP Programmer's Reference.

D.1 Decision-Cycle Configuration (Chapter 6)

- Does every registered domain have a failure threshold, a margin measure, and a baseline drift? (6.3, 6.4)
- Is the required "steady" action registered, and does it describe what actually happens under continuation? (6.4, 8.3)
- Is the governing tolerance set so that every domain that must govern does, and has the governing set been checked against intent? (6.5, 11.9)
- Are regime thresholds stated, and is it understood which policy families each regime activates? (6.6, 10.5)
- Are the required families and materially required policy classes stated in the request? (6.7)
- Is every constraint classified hard, conditional, or soft, with the relaxation rule stated for conditional ones? (6.8)
- Does the domain frame cover exactly the governing set, with each target stated as a function? (6.9)
- Are active recovery obligations and the shared capacity they compete for represented where they exist? (6.10)
- Does every candidate have one shared projection record—horizons per domain and recovery-corridor facts—used identically by Adequacy and Σ , with censored values marked? (6.11)
- Is the Σ order fixed, state-independent, and used only to break exact ties? (6.12)
- Does each expected scenario name the stage that should decide it? (6.16)

D.2 Execution Lineage (Chapter 7)

- Are selection, current authority, attempt, and realization treated as four separate events? (7.1)
- Is the execution license built only from a completed cycle, and is its selection mode respected—standard, recovery-unavailable fallback, or terminal infeasibility? (7.4)
- Is entry sufficiency stated by the host on current facts, not assumed? (7.4)
- Is every consequential function registered as a binding, with an implementation version, and never called outside the governed path? (7.5, 7.6)
- Is each native authorization single-use, obtained for one attempt, and invalidated when a fact it relied on changes? (7.6, 7.11)
- For each action: what counts as succeeded, failed cleanly (and what evidence proves no effect), indeterminate, and completed but invalid? (7.7)
- Does realized-effect evidence come from authoritative host records, never from a return value? (7.8)
- Does every terminal episode reach the host's Layer-1 transition, including failures? (7.9, 7.10)

- Does the Layer-1 transition declare invariants, and are provenance and recovery bookkeeping recorded where used? (7.9)
- Does a return upstream lead to governance, never to a retry with a different action? (7.11)
- For each action, is the authorization-to-effect exposure stated, with the host measures that reduce it? (7.16)
- Is the deployment's boundary statement written with framework, decision-core, supervision-layer, host, and prospective columns kept apart? (7.15)
- If recovery corridors can lapse, is expiry recorded as a host-side limitation rather than a write into runtime state? (7.18)

D.3 Application Boundary and Adapter Discipline (Chapter 8)

- Does every packet item have a home in the runtime structures of Table 8.1? (8.1)
- For every host function and adapter method: does it state a fact, and does it decide nothing? (8.2, 8.4)
- Is each rule and each harm classified as a prohibition (constraint) or a consequence (projection), with a reason? (8.4)
- Is $P_i(t)$ formed from actual state through the perception hook, with a freshness limit for each perceived fact? (8.5)
- Is actual state held in a host store in which every recorded change is a new state with a logged reason? (8.6)
- Is transition provenance carried into the next cycle only when the state has not changed in between? (8.6)
- Does observation intake record facts and decide nothing, and are enforcement mechanisms kept on the enforcement side? (8.7)
- Are the dependencies of each decision registered when it is made, so that later reports are matched by identity? (8.7)
- Is re-entry either a fresh cycle from updated state or a stage-specific re-entry with valid reusable results—never reuse of an invalidated result? (8.7)
- Are unknown outcomes held in actual state until reconciled by evidence, and are remedies governed actions in the Graph? (8.8)
- Are the store log, cycle results, and execution lineage all recorded from the first version? (8.9)
- Is every score either certified for the episode or labeled diagnostic? (8.12)

D.4 Supervised Execution (Chapter 12)

- Is each supervised action carried out by the host's own means with its episode open, not inside native call-through? (12.3, 12.13)
- Is every source registered under a trust root, with roles, fact kinds, subjects, permitted uses, and a validity window? (12.4)
- Does every supervised execution declare its dependencies on exact facts? (12.5)
- After every committed change, is every execution that fan-out returns checkpointed? (12.5, 12.6)
- Does every posture come from canonical ε or the planner, never from host code? (12.6)
- Is ownership held with a current fence, and is a stale owner's request treated as refused, not retried blindly? (12.7)
- Does each adapter map each semantic control to the controller's own command without widening it, and is each controller's attestation verified by the host? (12.7)

- Is enforcement kept apart from effect, are unknown effects held unresolved, and is a Layer-1 transition recorded only from a current reconciliation? (12.8)
- Is a recovery profile registered before execution, and does recovery require a fresh snapshot? (12.9)
- Are observation sizes capped before admission, with bulky evidence passed by reference? (12.10)

D.5 Errors to Rule Out

Table D.1 — Implementation errors to rule out

Stage	Errors to rule out
Decision cycle (6.18)	Reading the stack as a scoring pipeline; feeding actual state or raw memory into the cycle; letting an objective remove, rank, or authorize; confusing pressure with governing force or dropping a neglected domain; swapping baseline and policy-conditioned horizons; using regime as a priority; letting Π invent structure or treating a completeness failure as a selection problem; encoding a prohibition as a penalty; framing a mechanism; treating Adequacy as ranking or letting a non-governing gain offset a governing failure; recomputing projections downstream; using the Σ order as a preference; treating an empty adequate set as an ordinary choice; implementing stage logic in host code; fixing a failure where it became visible
Execution (7.19)	Treating selection as authorization or authorization as an attempt; updating state from a return value; classifying a timeout as a clean failure; claiming a clean failure without evidence; misreporting realized effects; applying Layer 1 only after success; hand-built, reused, or stale authority; calling the function outside the governed path; retrying with another action; treating a fallback license as adequacy; calling a pre-call gate mid-flight cancellation; attributing framework capability to the decision core; writing into runtime-private state
Application (8.17)	Selection, adequacy, feasibility, or ordering logic in host code; tuned projections or constraint observations; prohibitions as horizons or consequences as prohibitions; copied state or stale evidence as $P_i(t)$; store updates outside the transition and intake; intake or enforcement that decides; guessed dependencies; reuse of invalidated results; remedies issued by host code; dropped unknown outcomes; uncertified scores that route or authorize; late traceability; unreviewed AI output
Supervision (12.14)	Monitors, adapters, or supervisors that decide; unadmitted reports or arrival order treated as truth; guessed dependencies; affected executions left unchecked; caller-supplied postures; stale owners or snapshots; requests, dispatches, or enforcement treated as effects; timeouts recorded as no effect; stoppable work inside native call-through; claims that the runtime acts on its own or stops processes; writes into store tables

D.6 Sign-Off

- The packets from prompts A.6, A.7, A.8, and, where actions are supervised, A.12 are attached, or their equivalents written by hand.
- The application runs against the unmodified frozen runtime, verified by hash (Appendix G).
- Every expected scenario has been run and decided at the stage the packet predicted, or the difference is explained.
- A reviewer other than the author has checked the boundary register of Section 8.2 line by line.

Appendix E — Testing and Benchmark Checklist

This checklist covers Chapter 9. Its first part plans tests of an application; its second specifies a benchmark; its third fixes what must be recorded and reported. Use it with prompt A.9, and complete the claims and acceptance criteria before any result is seen.

E.1 Claims and Kinds of Test

- Is every intended claim written as one sentence that a result could contradict? (9.7)
- Does each claim name the kind of test that supports it and what that test does not establish (Table 9.1)? (9.1)
- Are framework, runtime-version, and application claims kept in separate columns? (9.16)

E.2 Application Tests

- For each category boundary the model depends on, is there a test that separates the two sides (Table 9.2)? (9.2)
- Is each invariant stated once and checked across many cases? (9.3)
- For each targeted failure—semantic laundering, stale or forged authority, incomplete state, missing recovery, hollow success, downstream repair—is there a hostile implementation expected to be refused or flagged? (9.4)
- Does the trace distinguish the ten failure classes of Table 9.3, and name the stage that set aside each candidate? (9.5)
- Are false-success conditions declared: the visible success criterion and the governing failure criterion? (9.6)
- Are perturbation and metamorphic tests included where an invariance is easier to state than an expected output? (9.8)
- Is there a runtime identity check against the frozen hash manifest? (9.9, 9.12)
- Is expected output banked for regression, with the test count and versions? (9.10)
- Where historical reconstruction is claimed, is there a replay test that uses only decision-time material? (9.11)

E.3 Benchmark Specification

A benchmark specification should answer each of the following before it runs. The list condenses the benchmark-design guidance on which Chapter 9 draws.

Table E.1 — Benchmark specification

Item	What to record
Bounded claim	The proposition, in a sentence that could be contradicted
Model source	The scenario and model packet and its version
Boundary and horizon	Actors, entities, decision points, episode length
State, perception, objectives	$S_i(t)$; $P_i(t)$ and $O_i(t)$ where relevant
Capability and value	PP, enabling resources, and where PV is actually realized
Governing conditions	Domains, thresholds, constraints, authority, recovery

Item	What to record
Corridors	Normal, failure, recovery, substitution, and degradation paths as relevant
Adequacy and false success	What makes an outcome adequate; what counts as false success
Variables	Fixed structure; controlled perturbations; policy or baseline variation; observed outcomes
Comparators	Task baseline, constraint baseline, legitimate scalar baseline, ablations, hostile variants
Integration mode	Runtime-integrated, standalone teaching, or hybrid; runtime surfaces under test
Observables	Ordinary task metrics and Productive Value–Productive Power (PV-PP) observables, kept apart
Acceptance criteria	Invariants or conditional behaviors, written before the run
Reproducibility	Versions, hashes, configuration, seeds, trial counts
Limitations	What the benchmark does not establish, and excluded surfaces

E.4 Programs of Benchmarks

- Is the architecture frozen for the program, with a rule that failures are recorded before any change is considered? (9.14)
- Is the selection of environments or cases pre-registered, with no substitution of easier cases and no omission of unfavorable ones? (9.14)
- Is every case given a disposition from a fixed vocabulary—scalar containment, multidimensional preservation, adequacy or constraint distinction, fallback or infeasibility, formalization gap, architecture failure? (9.14)
- Are benchmark defects, fixture errors, and source-fidelity limitations recorded separately from architecture failures? (9.14)
- Does the program end with a closeout that states what it does not establish? (9.14, 9.15)

E.5 Reproducibility and Reporting

- Does every run bind code version, framework and runtime version, runtime hash, configuration, seeds, initial state, perturbation, baseline, expected invariants, and trace destination? (9.9)
- Does the package include specifications, the test suite and its expected result, reproduction commands, raw or reconstructable outputs, and a limitations statement? (9.9)
- Are copied logs identified as copies rather than reported as reruns? (9.10)
- Does the report place each result in the correct column: framework, runtime version, or application? (9.16)

E.6 Errors to Rule Out

Table E.2 — Testing and benchmarking errors to rule out (9.18)

Area	Errors to rule out
Claims	Citing a regression suite as validation; reporting a benchmark win as general superiority; attributing framework-level or v0.70 evidence to a later runtime, or supervision-layer evidence to the decision core
Design	Declaring success without a governing failure criterion; a single success flag; weak comparators or omitted strong ones; changing rules between arms or during a program
Selection and disposition	Choosing environments after results; dropping unfavorable ones; confusing fixture or source defects with architecture failures
Implementation and records	Reimplementing runtime behavior while claiming to test the runtime; treating copied logs as reruns; inserting later knowledge into replay

Appendix F — Sources and Open Items

This appendix says what controls the claims in this book, where its evidence comes from, and what remains open.

F.1 What Controls What

- **Framework meaning.** The Productive Value–Productive Power (PV-PP) framework is defined by its V2.1 specification set, maintained by the author. This Guide explains that framework; where the Guide is less precise than a specification, the specification controls.
- **Runtime behavior.** The frozen PV-PP Runtime v2.1.167 source (Framework 2.1, build 167; development identifier v0.167) and its tests, published in the repository, control what the runtime does; its decision core is the source of Runtime V2.1 (build v0.141), unchanged. The PV-PP Programmer's Reference states that behavior exactly; this Guide explains it.
- **Scalar-reduction results.** Direction 1 is published as *When Is One Number Enough? A Containment Theorem for Scalar Comparison in Staged Decision Architectures* (Amundsen, 2026; DOI 10.5281/zenodo.22715202). Direction 2 and the program's closure rest on the author's theorem packet and closure records; the paper describes Direction 2 but does not prove it. Chapter 11 states both results and their limits.

F.2 Evidence Sources

The benchmark records named in Table F.1 are held by the author; the benchmark problems themselves come from the public sources named in Chapter 1 and Chapter 9.

Table F.1 — Benchmark and evidence sources

Program	Record	Runtime
Runtime V2.1 test suite	Release notes; hardening record; file-hash manifest	v0.141
Runtime v2.1.167 test suite and hardening	Release notes; twelve hostile-review records; candidate-freeze audit; file-hash manifest	v2.1.167
Small Economy V2.1 objective benchmark	Benchmark record (v0.140); hardening rerun (v0.141)	v0.140, v0.141
Small Economy stress test	Internal development summary and milestone findings	v0.70
AI Safety Gridworlds (PB-001–PB-010)	Gridworld closeout; PB-009H closeout	Frozen architecture; version not recorded
MO-Gymnasium (MB-001–MB-016)	Complete-suite charter, selection audit, closeout, white paper	Frozen architecture; version not recorded

Program	Record	Runtime
MOEA RealWorldBenchmarks (RWB-001–RWB-003)	Selection-protocol freeze; three-case assessment; white paper	Frozen architecture; version not recorded
Safe Margin versus Fast Gain	Teaching-benchmark specification; no banked results	Standalone
The Guide's programs	Appendix G	v0.141 (Chapters 6–10); v2.1.167 (Chapter 12)

F.3 Known Gaps and Open Items

Table F.2 — Known gaps and open items

Item	Kind	Where
No public operation to cancel or expire an active recovery corridor	Runtime-interface limitation, found in historical testing (v0.70), confirmed in v0.141, unchanged in v2.1.167	7.18
Direct Layer-1 handoff builder maps outcomes differently from the ϵ path	Runtime-interface inconsistency	7.8
No intrinsic size limit on supervised observations	Deployment boundary in v2.1.167; hosts cap size before admission	12.10
Decision core synchronous; supervision cannot govern a native call after entry, act on its own, or stop anything physically	Runtime scope limitation; further extension prospective	7.15–7.17, 12.13
Mixed or randomized policies (PB-009H)	Open formalization gap in the framework	9.15, 11.10
Essential domains that do not decay under baseline continuation	Open modeling question	11.9
Scalar classes left open by the proofs	Open research	11.4
Layer 2 as an explicit dynamical system	Candidate research question	11.10

Appendix G — Code Files

The Guide's programs are small, complete, and run against the unmodified frozen runtime. Each has a test file that checks the behavior its chapter describes. Exact copies are in the repository's `programmers-guide/programs` folder; this appendix records what they are and how to run them.

G.1 The Files

Table G.1 — Programs and tests

File	Ch.	What it demonstrates	Tests
coffee_decision.py	6	One decision cycle through Σ , four cases	<code>test_coffee_decision.py</code> (5)
coffee_execute.py	7	The execution lineage and the four outcome classes; stale authority	<code>test_coffee_execute.py</code> (7)
coffee_app.py	8	A complete application: perception, store, Layer 1, intake, dependencies, reconciliation, remedy	<code>test_coffee_app.py</code> (7)
test_coffee_hostile.py; coffee_day_expected.txt	9	Boundary, hostile, metamorphic, regression, and runtime-identity tests	16
agent_ops.py	10	An operations agent: reachability refresh, regime shaping, controlled admission, withdrawn authority	<code>test_agent_ops.py</code> (8)
ops_model.py	12	The operations agent reduced to the failover decision	—
supervised_failover.py	12	Supervised execution: monitor intake, fan-out, checkpoint, governed stop, effect reconciliation, re-governance	<code>test_supervised_failover.py</code> (9)

`coffee_execute.py` imports `coffee_decision.py`, and the Chapter 9 tests import `coffee_app.py` and read `coffee_day_expected.txt`, so keep all files in one folder. `supervised_failover.py` imports `ops_model.py`.

G.2 Running Them

- Requirements: Python 3 (the programs were run with 3.11) and `pytest`.
- Clone or download the repository, github.com/AmundsenLance/pvpp-runtime-api, and note the path of its `runtime-v2.1.167` folder, the one that contains `pvpp_runtime`.
- Set the environment variable `PVPP_RUNTIME_PATH` to that folder.
- Run any program directly, for example: `python3 coffee_app.py`. Its output should match the listing in its chapter.
- Run the tests from the folder containing the files: `python3 -m pytest -q`. Expected result: 52 passed—the 43 tests of Chapters 6–10 and the 9 of Chapter 12.

G.3 Checking the Runtime

Results are claimed for frozen v2.1.167 only. Two checks confirm that the runtime in use is that build. First, from the runtime-v2.1.167 folder, verify every file against the shipped manifest with `sha256sum -c V0167_FILE_HASHES.sha256`; any mismatch means the runtime is not the frozen build. Second, run the runtime's own suite with `python3 -m pytest -q`; the frozen result is 1,320 passed. The Chapter 9 test `test_runtime_is_the_frozen_v0141_build` checks the decision core automatically against the build-141 manifest that the folder also carries; it passes on v2.1.167 because the core is unchanged, and it does not check the supervision layer.

G.4 What the Programs Are Not

The programs are teaching examples. Their domain numbers are illustrative, their perception is simplified, and their stores are local files. They show how an application should call the runtime and where host responsibilities lie; they are not production components, and passing their tests is evidence about these programs on this runtime, nothing more (Chapter 9).